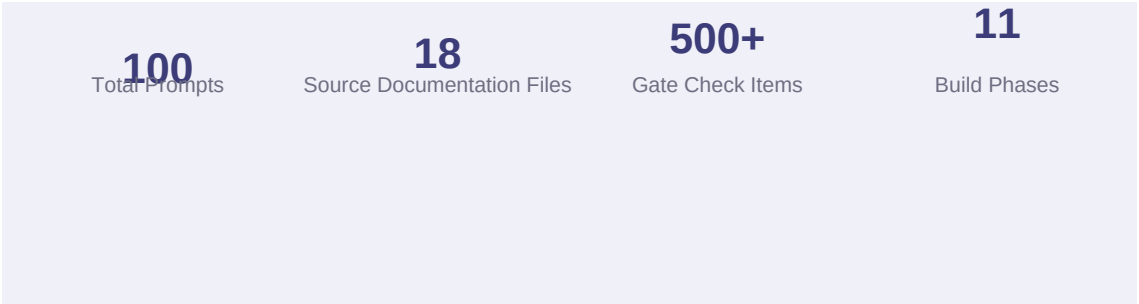


Recall

Build Prompt Library

End-to-End Implementation Guide | 100 Prompts | 0% to 100%
2026-06-19 | CONFIDENTIAL | D:\Recall\docs\PROMPTS.md



PROMPTS -- Recall Build Playbook

- > End-to-end implementation prompts from 0 -> 100%.
- > Every prompt references the exact skill(s) to activate and carries strict non-negotiable rules.

HOW TO USE THIS DOCUMENT

1. Open a fresh agent session.
2. Activate the listed skill(s) at the top of each prompt.
3. Paste the prompt body verbatim -- the rules section is mandatory context.
4. Do not skip phases or combine phases -- each builds on the previous.
5. Every prompt ends with a Gate Check -- do not move to the next prompt until all checks pass.

GLOBAL RULES (apply to every single prompt in this document)

ARCHITECTURE RULES

- Stack is fixed: FastAPI (backend) . React+Vite (frontend) . Neon PostgreSQL+pgvector+pg_trgm . Upstash Redis . Modal GPU . Render . Vercel.
- No new libraries without explicit justification. Prefer stdlib or already-approved packages.
- All DB queries must use parameterised statements -- zero string interpolation into SQL.
- Webhook handler must return 200 to Telegram in < 50 ms. All heavy work goes to background queue.
- `asyncio.Semaphore(3)` caps concurrent AI tasks. Never raise this limit without justification.

SECURITY RULES -- NON-NEGOTIABLE

- `TELEGRAM_BOT_TOKEN`, `FERNET_KEY`, `JWT_SECRET` must NEVER appear in logs, responses, or frontend code.
- `raw_text` and `google_refresh_token` must be Fernet-encrypted before any DB write. No exceptions.
- All `/api/*` routes must validate either the TWA HMAC or the JWT cookie before processing.
- HMAC comparison must use `hmac.compare_digest()` -- never `==` for secret comparison.
- All user data queries must include `WHERE user_id = <verified_user_id>` -- no cross-user data access.
- Google OAuth scope must be `drive.file` only -- never broader scopes.
- `httpOnly` + `Secure` + `SameSite=Lax` on all cookies.

PERFORMANCE RULES

- Vector search target: < 10 ms (HNSW cosine, `m=16`, `ef_construction=64`).
- Text search target: < 5 ms (GIN trigram on summary column only -- not `raw_text`).
- Canvas render target: 60 FPS at 500 nodes.
- Webhook ACK target: < 50 ms.
- Every DB query touching items must include `user_id` in WHERE clause (uses `idx_items_user` B-tree).

TESTING RULES

- Every new function must have at least one unit test before the prompt is considered complete.
- All tests must run with zero external API calls -- mock all AI tiers, Telegram API, and Redis.
- Use `pytest` for backend, `Vitest` for frontend.
- Test the failure path, not just the happy path.

ERROR HANDLING RULES

- Every AI cascade tier must be wrapped in `try/except` with specific exception types.
- Dead letter queue entry must be written before the bookmark fallback item is saved.
- User-facing error messages must never expose internal error details or stack traces.
- All scheduler jobs must have `misfire_grace_time=60` set.

PHASE 0: PROJECT FOUNDATION

PROMPT 001 -- Image OCR Quality + Preprocessing

SKILLS: python-pro

Improve image ingestion (PROMPT 043) with pre-processing for better OCR quality.

Image preprocessing pipeline (before Tesseract):

Use Pillow for all preprocessing:

1. Convert to grayscale: `image.convert('L')`
2. Increase contrast: `ImageEnhance.Contrast(image).enhance(2.0)`
3. Sharpen: `ImageFilter.SHARPEN`
4. Resize if < 800px wide: `image.resize` to 1200px width (maintains aspect ratio)
5. Binarise (black/white): `image.point(lambda p: 0 if p < 128 else 255, '1')`

Language detection:

Run Tesseract with `lang='eng+hin+fra+deu'` (multi-language support for common languages)

Detect dominant language from Tesseract output (`pytesseract.image_to_data` returns language info)

Confidence filtering:

`pytesseract.image_to_data` returns per-word confidence scores.

Filter out words with confidence < 60% before returning OCR text.

If fewer than 10 high-confidence words remain: treat as low-quality -> use Gemini captioning.

QR code / barcode detection:

If image appears to be a QR code (`pyzbar` library): decode QR -> save as URL item.

Bot reply: "QR code detected -> URL: {decoded_url}\nSaved [x]"

Rules:

- Preprocessing must happen in memory (no additional temp files).
- Tesseract timeout: 30 s per image -- kill process if exceeded.
- Confidence threshold: 60% is not configurable -- hardcoded.
- `pyzbar`: optional dependency (skip QR if not installed, log warning).

Gate Check:

- [] Skewed/low-contrast image produces better OCR with preprocessing
- [] Low confidence OCR (<10 words) falls back to Gemini captioning
- [] QR code image -> decoded URL saved as url item
- [] Tesseract kills process after 30 s timeout
- [] Unit test: `test_image_ocr.py` with mocked PIL and Tesseract

PROMPT 002 -- Force-Directed Canvas Renderer

SKILLS: react-patterns . react-component-performance . high-end-visual-design

Create `frontend/src/canvas/GraphCanvas.jsx` -- the constellation mind map renderer.

Use D3 force simulation:

- `forceSimulation()` with `forceLink`, `forceManyBody` (strength -200), `forceCenter`, `forceCollide`
- Barnes-Hut approximation: `simulation.alphaDecay(0.02)`, `velocityDecay(0.4)`
- Target: 60 FPS at 500 nodes (use `requestAnimationFrame` loop)

Canvas drawing (use drawFrostedNode and drawConstellationEdge from IMPLEMENTATION_DESIGN_TOKENS.md):

- Edges: quadratic Bezier curves (not straight lines)
- Nodes: frosted glass disks with radial star-glow scaled by connection degree
- Hub nodes: larger, mint-teal (#00D4AA) with slow-rotating outer dashed ring
- Pulse nodes (created < 5 min ago): white core with expanding concentric ripple animation

Node interaction:

- Click: emit onNodeClick(node) to parent -- parent opens side panel
- Hover: scale glow to 1.3x, cursor: pointer
- Pan: mouse drag translates the canvas transform
- Zoom: scroll wheel scales canvas transform (min 0.3x, max 3x)

Rules:

- Canvas must resize to full viewport on window resize (ResizeObserver).
- requestAnimationFrame loop must stop when component unmounts (cancel via ref).
- Hub nodes must render ABOVE orbital nodes (draw order matters in 2D canvas).
- Do NOT use SVG -- HTML5 Canvas 2D only (performance requirement).
- @media (prefers-reduced-motion: reduce) -> disable pulse ripple animation only; keep static graph.

Gate Check:

- [] 100 nodes render at >= 60 FPS (Chrome DevTools Performance panel)
- [] Clicking a node calls onNodeClick with correct node data
- [] Pan and zoom work correctly
- [] Canvas resizes correctly on window resize
- [] Vitest: GraphCanvas renders without crashing (smoke test)

PROMPT 003 -- Local Development Environment

SKILLS: python-pro

Set up a local dev environment that mirrors production exactly.

Create backend/.env.local.example with all 20 variables pre-filled with safe dev values:

- DATABASE_URL: point to a Neon dev branch (not main)
- UPSTASH_REDIS_REST_URL/TOKEN: Upstash dev database
- TELEGRAM_BOT_TOKEN: a second test bot from BotFather
- FERNET_KEY: generate a fresh key just for dev
- JWT_SECRET: generate a fresh secret just for dev
- COMPUTE_PROVIDER=groq (skip Modal in local dev by default)
- WEBSITE_URL=http://localhost:5173

Create Makefile with dev shortcuts:

```
make dev-backend    -> uvicorn main:app --reload --port 8000
make dev-frontend   -> cd frontend && npm run dev
make test           -> cd backend && pytest -x -v
make tunnel         -> ngrok http 8000 (for Telegram webhook testing locally)
make schema         -> python -c "from db.connection import init_schema; asyncio.run(init_schema())"
make fernet         -> python -c "from cryptography.fernet import Fernet; print(Fernet.generate_key().decode())"
make jwt-secret     -> python -c "import secrets; print(secrets.token_hex(32))"
```

Create backend/dev_seed.py:

```
Seeds the local DB with 3 test users and 10 items each.
Uses plaintext summary for easy debugging.
MUST only run if DATABASE_URL contains "dev" or "test" -- refuses to run against production.
```

Rules:

- dev and production FERNET_KEY must NEVER be the same -- document this clearly.
- COMPUTE_PROVIDER=groq in dev: ensures no Modal GPU charges during development.

- Never commit .env.local -- add to .gitignore explicitly.
- dev_seed.py must check DATABASE_URL before running -- prevents accidental prod seeding.
- ngrok tunnel: for Telegram webhook testing only; never expose production webhook to ngrok.

Gate Check:

- [] make dev-backend starts server on port 8000
- [] make test runs all unit tests with zero network calls
- [] dev_seed.py refuses to run against production DATABASE_URL
- [] .env.local absent from git history (git log --all -- .env.local shows 0 commits)
- [] make tunnel command documented but not run automatically

PROMPT 004 -- Repo Structure & Python Environment

SKILLS: python-pro . python-development-python-scaffold

Set up the Recall project repository with the following exact structure:

```
recall/
|-- backend/
|   |-- main.py           # FastAPI app entry point
|   |-- requirements.txt  # pinned versions
|   |-- .env.example      # all env var keys, no values
|   |-- models/          # SQLAlchemy or raw psycopg3 models
|   |-- routes/          # one file per route group
|   |-- services/        # AI cascade, scraping, encryption
|   |-- scheduler/       # APScheduler jobs
|   |-- modal_apps/      # Modal serverless GPU apps
|   +-- tests/           # pytest test files
|-- frontend/
|   |-- src/
|   |   |-- components/
|   |   |-- pages/
|   |   |-- canvas/      # mind map Canvas renderer
|   |   +-- hooks/
|   |-- index.html
|   |-- vite.config.js
|   +-- package.json
+-- docs/                # already exists -- do not modify
```

Rules:

- Python 3.11+. Pin all requirements.txt to exact versions.
- Create .env.example with ALL 20 variables listed in ENV_CONFIG.md -- values empty, comments explaining each.
- Create .gitignore: exclude .env, __pycache__, .venv, node_modules, *.pyc, dist/.
- Do NOT create any application logic yet -- structure only.
- README.md must reference docs/DEPLOYMENT.md for setup instructions.

Gate Check:

- [] recall/backend/requirements.txt exists with pinned deps
- [] recall/backend/.env.example has all 20 variables from ENV_CONFIG.md
- [] recall/frontend/vite.config.js exists
- [] .gitignore excludes .env
- [] python -m pytest runs (0 tests, no errors)

PHASE 1: CORE WEBHOOK & BOT COMMANDS

PROMPT 005 -- Neon Database DDL

SKILLS: [neon-postgres](#) . [postgres-best-practices](#) . [postgresql](#)

Create the complete PostgreSQL schema for Recall in backend/db/schema.sql.

Follow BACKEND_SCHEMA.md exactly. Execute in this order:

1. Extensions: vector, pg_trgm
2. users table
3. items table (PARTITIONED BY RANGE created_at)
4. items_y2026m06 partition
5. items_y2026m07 partition
6. quizzes table
7. reminders table
8. semantic_hubs table
9. processed_updates table
10. dead_letter_queue table
11. All 4 indices (idx_items_user, idx_items_embedding, idx_items_text_gin, idx_reminders_time_status)

Rules:

- HNSW index: m=16, ef_construction=64, vector_cosine_ops. Exactly these values.
- GIN index: on summary column ONLY using gin_trgm_ops. NOT on raw_text (encrypted).
- All FK references use ON DELETE CASCADE.
- DATABASE_URL must use ?sslmode=require -- no unencrypted connections.
- Write a backend/db/verify.py script that connects and runs:
SELECT extname FROM pg_extension; -- must return vector, pg_trgm
SELECT tablename FROM pg_tables WHERE schemaname='public'; -- must list all 8 tables
SELECT indexname FROM pg_indexes WHERE tablename='items'; -- must list all 3 item indices

Gate Check:

- [] schema.sql creates all 8 tables without errors
- [] verify.py connects to Neon and prints all extensions and tables
- [] HNSW and GIN indices confirmed present via verify.py
- [] No table created without a corresponding index plan

PROMPT 006 -- Streak Visualisation in Frontend

SKILLS: [react-ui-patterns](#) . [high-end-visual-design](#)

Add a streak display component to the dashboard header.

StreakBadge (frontend/src/components/StreakBadge.jsx):

Display: {streak_count}
Position: header, between the quiz badge and profile icon

On click: opens StreakPanel (glass card overlay, 360px):

" {N} day streak!"
Subtext: "Last saved: {last_activity_date relative time}"

Streak calendar:

Simple 7-day grid showing which days the user saved items

```

Days with saves: filled circle (--color-accent)
Days without: empty circle (--text-tertiary)
Today: pulsing ring

Data from GET /api/me (add last_7_days_activity: list[bool] to response)

```

Backend: GET /api/me enhancement:

Add last_7_days_activity: bool[] -- whether user saved any item each of the last 7 days.

Query:

```

WITH days AS (SELECT generate_series(NOW()::date - 6, NOW()::date, '1 day')::date AS day)
SELECT d.day, COUNT(i.id) > 0 AS has_activity
FROM days d LEFT JOIN items i ON i.user_id = $1 AND i.created_at::date = d.day
GROUP BY d.day ORDER BY d.day

```

Rules:

- StreakBadge must not show emoji on mobile in TWA -- use @phosphor-icons Flame instead.
- 7-day calendar uses CSS grid (7 equal columns) -- no Canvas for this component.
- last_7_days_activity computed server-side -- never trust client-provided dates.
- Streak resets at midnight UTC -- document timezone assumption in API.

Gate Check:

- [] StreakBadge shows correct streak count from /api/me
- [] 7-day calendar shows correct active days
- [] Today's circle has pulsing animation
- [] @phosphor-icons Flame used in TWA (no emoji)
- [] Vitest: StreakBadge renders "1" correctly with mock API data

PROMPT 007 -- Logout + Session Refresh

SKILLS: auth-implementation-patterns . security . python-fastapi-development

Implement logout and session management.

POST /auth/logout (backend):

1. Clear the recall_session cookie:

```
response.delete_cookie("recall_session", httponly=True, secure=True, samesite="lax")
```
 2. Return 200: {"message": "Logged out"}
- Note: JWT is stateless -- no server-side revocation. Cookie deletion is the logout mechanism.

Frontend logout:

1. POST /auth/logout
2. Clear any local state (auth context, cached graph data)
3. Redirect to /login

JWT auto-refresh (optional -- implement if simple):

- If JWT has < 1 day remaining: issue a fresh JWT (7-day expiry) on any authenticated request.
- Set new cookie in response without requiring re-login.
- Check: if exp - now < 86400: issue new JWT and set cookie.

Profile dropdown (header):

- Shows: Telegram username (from JWT chat_id)
- Menu items:
 - [Connect Google Drive] -> open /auth/google in popup
 - [Disconnect Drive] -> DELETE /api/drive (if connected)
 - [Logout] -> POST /auth/logout -> redirect /login

Rules:

- delete_cookie must use same attributes as set_cookie (httpOnly, Secure, SameSite).

- Frontend must clear all cached data on logout (prevent data leakage to next user on shared device).
- Auto-refresh must only happen if user is actively using the app (within authenticated request).
- Logout must work even if JWT is already expired (graceful degradation).

Gate Check:

- [] POST /auth/logout clears cookie and returns 200
- [] Frontend redirects to /login after logout
- [] Logging in again after logout works without issues
- [] JWT with < 1 day remaining gets refreshed automatically
- [] Unit test: test_logout.py verifies cookie is cleared with correct attributes

PROMPT 008 -- Redis Queue Monitoring + Dead Letter Retries

SKILLS: python-pro . async-python-patterns

Add Redis queue monitoring and dead letter queue retry mechanism.

Queue monitoring endpoint GET /api/admin/queue (protected by X-Internal-Key header):

```
{
  "queue_length": int,          -- LLEN recall:tasks
  "dead_letter_count": int,     -- SELECT COUNT(*) FROM dead_letter_queue WHERE retried=FALSE
  "oldest_dead_letter": str,    -- ISO timestamp of oldest unretried DLQ entry
  "processing_slots": {
    "available": int,          -- 3 - current semaphore acquisitions
    "total": 3
  }
}
```

Dead letter retry endpoint POST /api/admin/dlq/{id}/retry (protected):

1. SELECT task_payload FROM dead_letter_queue WHERE id=\$1 AND retried=FALSE
2. LPUSH task_payload back onto recall:tasks queue
3. UPDATE dead_letter_queue SET retried=TRUE WHERE id=\$1
4. Return 200: {"queued": true}

Auto-retry on startup:

On FastAPI startup: check dead_letter_queue WHERE retried=FALSE AND failed_at > NOW() - 24h
Re-enqueue up to 5 tasks (not all -- avoid flooding queue on restart).
Log how many tasks re-queued.

Rules:

- /api/admin/* requires X-Internal-Key header -- not the user JWT.
- X-Internal-Key: a separate secret from JWT_SECRET (add INTERNAL_API_KEY to env vars).
- Auto-retry on startup: max 5 tasks, only tasks < 24 hours old.
- Retry does NOT clear attempted_tiers -- cascade re-tries from Tier 0 again.
- Unit test: test_dlq_retry.py verifies re-queue and retried=TRUE update.

Gate Check:

- [] GET /api/admin/queue returns accurate queue_length
- [] POST /api/admin/dlq/{id}/retry re-enqueues the task and marks retried=TRUE
- [] Auto-retry on startup: max 5 tasks re-queued
- [] /api/admin/* returns 401 without correct X-Internal-Key
- [] Unit test: retry flow with mocked Redis and DB

PROMPT 009 -- Environment & Config Loader

SKILLS: python-pro . security

Create backend/config.py -- a centralised settings loader using pydantic-settings.

Requirements:

- Load all 20 env vars from ENV_CONFIG.md using pydantic BaseSettings.
- Required vars must raise ValidationError on startup if missing -- fast fail, no runtime surprises.
- Optional vars (OLLAMA_HOST, COMPUTE_PROVIDER, scraping keys) default to None.
- Expose a single settings singleton: from config import settings.
- Add a startup check function: settings.validate_crypto_keys() that verifies:
 - FERNET_KEY is valid base64 (32 bytes after decode)
 - JWT_SECRET is at least 32 hex characters
 - TELEGRAM_BOT_TOKEN matches pattern \d+:[A-Za-z0-9_-]{35}

Security Rules:

- settings object must NEVER be serialised to JSON or logged.
- Add a __repr__ method that returns "<Settings: [REDACTED]>" -- prevents accidental logging.
- Any function that prints or logs settings must be flagged as a security violation.

Gate Check:

- [] Starting FastAPI with missing required var raises clear startup error
- [] settings.__repr__() returns redacted string
- [] validate_crypto_keys() raises ValueError for invalid FERNET_KEY format
- [] Unit test: test_config.py covers missing required vars and invalid key formats

PROMPT 010 -- Upstash Redis Client Wrapper**SKILLS: async-python-patterns . python-pro**

Create backend/services/redis_client.py -- a clean async wrapper around Upstash REST API.

class UpstashRedis:

```

    async def lpush(self, key: str, value: str) -> int
    async def brpop(self, key: str, timeout: int = 5) -> tuple[str, str] | None
    async def pipeline(self, commands: list[list]) -> list
    async def ping(self) -> bool

```

All methods use httpx.AsyncClient with:

```

base_url = settings.UPSTASH_REDIS_REST_URL
headers = {"Authorization": f"Bearer {settings.UPSTASH_REDIS_REST_TOKEN}"}

```

Pipeline method sends batch commands as JSON array to Upstash REST pipeline endpoint.
Used exclusively by the rate limiter (PROMPT 010) and task queue.

Expose: redis = UpstashRedis() singleton at module level.

Error handling:

- httpx.TimeoutException: log + raise RedisUnavailableError (do not crash worker)
- httpx.HTTPStatusError with 4xx: log + raise RedisAuthError
- httpx.HTTPStatusError with 5xx: log + retry once after 1 s

Rules:

- Never use a persistent TCP Redis client -- Render free tier kills idle TCP connections.
- REST API only -- all commands via HTTPS.
- UPSTASH_REDIS_REST_TOKEN must NEVER appear in logs -- redact in error messages.
- Connection timeout: 5 s. Never block indefinitely.
- Unit test: mock httpx to verify pipeline sends correct JSON format.

Gate Check:

- [] lpush and brpop work correctly against real Upstash REST API
- [] pipeline sends all 4 rate-limit commands in one HTTP request
- [] ping() returns True when Redis is available
- [] UPSTASH_REDIS_REST_TOKEN never appears in any log output
- [] Unit test: test_redis_client.py with mocked httpx verifies pipeline format

PROMPT 011 -- WebSocket Real-Time Graph Updates

SKILLS: python-fastapi-development . async-python-patterns

Implement WS /ws/{token} in backend/routes/websocket.py.

Connection flow:

1. Client connects to /ws/{token} where token is the JWT value (not cookie -- WS can't send cookies easily).
2. Server validates JWT from path param.
3. Register connection in an in-memory dict: {user_id: WebSocket}.
4. Send initial ping: {"type": "connected", "user_id": user_id}
5. Keep alive: send {"type": "ping"} every 30 s; disconnect if no pong within 10 s.

Event types pushed to client:

```

{"type": "new_node", "node": {id, title, source_type, created_at}}      # after item saved
{"type": "hubs_updated", "hubs": [{id, label, member_ids}]}          # after Louvain job
{"type": "google_connected"}                                           # after Drive OAuth
{"type": "ping"}
```

Broadcast helper: `async def broadcast(user_id: int, event: dict)`

Call `broadcast()` from `worker.py` after successful item save.

Rules:

- In-memory connection dict: {user_id: WebSocket} -- only one WS per user (last connection wins).
- WebSocket disconnect must clean up the dict entry -- no stale references.
- JWT validation for WS must be the same function as for HTTP (no copy-paste).
- WS /ws/* is exempt from rate limiting (RATE_LIMITING.md Exemptions).
- Do NOT store WS connection state in Redis or DB -- in-memory only.

Gate Check:

- [] WebSocket connects and receives "connected" event
- [] Saving an item via bot causes new_node event in open browser WS connection within 2 s
- [] Disconnected client cleanup: dict entry removed
- [] Expired JWT in WS path -> connection rejected with 4001 close code
- [] Unit test: test_websocket.py with TestClient WebSocket and mocked broadcast

PROMPT 012 -- Drive Connect UI on Website

SKILLS: react-ui-patterns . auth-implementation-patterns

Add Google Drive connection UI to the dashboard.

ConnectDriveCard (shown in profile dropdown or a settings panel):

State A -- Not connected:

```

Icon: @phosphor-icons GoogleDriveLogo (or generic CloudArrowUp, 32px)
Title: "Back up to Google Drive"
```

Description: "Connect your Drive to export your knowledge as a searchable Google Doc."

Button: [Connect Google Drive] -> opens /auth/google in popup window (not redirect)

State B -- Connected:

Icon: @phosphor-icons CheckCircle (mint green)

Title: "Google Drive connected"

Last sync: "Last synced: {date}" (from user profile API)

Button: [Sync Now] -> POST /api/drive/sync

Button: [Disconnect] -> DELETE /api/drive -> clears google_refresh_token

OAuth popup flow:

window.open("/auth/google", "recall-drive-auth", "width=600,height=700")

Listen for WS event google_connected -> close popup -> update Drive status UI

Rules:

- OAuth must open in popup, not full redirect -- prevents losing graph state.
- WS event (not polling) drives UI update -- no timeout-based detection.
- After [Connect Google Drive] click: button shows spinner "Connecting..." until WS event arrives.
- Disconnect: DELETE /api/drive sets google_refresh_token = NULL -- must verify user owns it.
- DELETE /api/drive response: 204 -- no body.

Gate Check:

[] Drive OAuth opens in popup window

[] WS google_connected event closes popup and updates button to "Connected" state

[] [Sync Now] triggers drive sync and shows toast on completion

[] [Disconnect] removes refresh_token from DB and reverts to "Not connected" state

[] Vitest: ConnectDriveCard renders both states with mock WS

PROMPT 013 -- Content Deduplication

SKILLS: python-pro . postgres-best-practices

Implement content deduplication to prevent saving the same URL or text twice.

URL deduplication:

Before ingesting a URL: SELECT id FROM items WHERE user_id=\$1 AND source_url=\$2 LIMIT 1

If found: bot replies "Already saved! Item ID: {id} -- {title}"

No new item created.

Text deduplication (approximate):

Compute a content hash: hashlib.sha256(raw_text.encode()).hexdigest()[:16]

Add column: items.content_hash VARCHAR(16)

Before ingesting text/voice: SELECT id FROM items WHERE user_id=\$1 AND content_hash=\$2

If found: bot replies "This looks like something you've already saved."

Add content_hash to schema:

ALTER TABLE items ADD COLUMN content_hash VARCHAR(16);

CREATE INDEX idx_items_content_hash ON items(user_id, content_hash);

Voice note deduplication:

Hash the audio file's MD5 sum (before transcription): unlikely to have exact duplicates, but avoids re-processing the same forwarded voice note.

Rules:

- URL deduplication is EXACT (source_url string match) -- not fuzzy.
- Text deduplication is APPROXIMATE (first 16 chars of SHA-256) -- accept ~0.00001% collision rate.
- content_hash must be computed BEFORE Fernet encryption (hash of plaintext, not ciphertext).
- Deduplication check must include user_id -- User A and User B can save the same URL independently.

- When duplicate detected: still return 200 to Telegram (not an error).

Gate Check:

- [] Forwarding same URL twice: second attempt returns "Already saved" message
- [] Forwarding same text twice: second attempt returns deduplication message
- [] Different users can save the same URL (not cross-user deduplicated)
- [] content_hash computed from plaintext before encryption
- [] Unit test: test_deduplication.py tests URL and text dedup flows

PROMPT 014 -- Dead Letter Queue Writer

SKILLS: error-handling-patterns . postgres-best-practices

Create backend/services/dlq.py -- the Dead Letter Queue service.

```
async def write_to_dlq(user_id: int, task_payload: dict, error_message: str, db: AsyncConnection):
    INSERT INTO dead_letter_queue (user_id, task_payload, error_message)
    VALUES ($1, $2::jsonb, $3)
```

task_payload structure (from ERROR_HANDLING.md):

```
{
  "chat_id": str,
  "content_type": str,
  "file_id": str | None,
  "update_id": str,
  "attempted_tiers": list[int],
  "last_error": str
}
```

Also implement: async def send_failure_message(chat_id: str, content_type: str):

Message templates per ERROR_HANDLING.md user-facing messages table.

Never include technical error details in the message.

Rules:

- write_to_dlq must NEVER raise -- it is the last line of defence. Wrap in try/except, log if fails.
- task_payload must be validated as valid JSON before INSERT.
- attempted_tiers must list every tier that was tried (for debugging).
- User-facing message must NOT contain: exception type, error code, stack info.

Gate Check:

- [] write_to_dlq creates dead_letter_queue row with correct JSONB payload
- [] write_to_dlq does not raise even if DB is unavailable (logs instead)
- [] send_failure_message uses correct template for each content_type
- [] Unit test: test_dlq.py with mocked DB verifies payload structure

PHASE 2: INGESTION PIPELINE & AI CASCADE

PROMPT 015 -- Redis Sliding Window Rate Limiter

SKILLS: [async-python-patterns](#) . [python-pro](#)

Implement the Redis sliding window rate limiter in backend/services/rate_limiter.py.

Algorithm (from RATE_LIMITING.md exactly):

```
key = "rate:{chat_id}"
now = current Unix timestamp in milliseconds
window_start = now - 60_000
```

Upstash REST pipeline (atomic):

```
ZREMRANGEBYSCORE key 0 window_start # evict expired
ZADD key now "{now}-{uuid4()}"      # record this request
ZCARD key                          # count in window
EXPIRE key 61                       # TTL
```

If count > 20:

```
retry_after = 60 - (now - oldest_score_in_window) / 1000
Raise RateLimitExceeded(retry_after=retry_after)
```

Integrate into POST /webhook after idempotency check, before task enqueue.

On RateLimitExceeded: return HTTP 200 to Telegram (not 429 -- Telegram doesn't handle 429 gracefully for webhooks), but do NOT enqueue the task.

Rules:

- Use Upstash REST pipeline -- NOT persistent TCP Redis connection (Render free tier).
- All 4 commands must execute in a single atomic pipeline call.
- Rate limit is per chat_id -- never global.
- /health, /auth/*, /ws/* are EXEMPT from rate limiting (see RATE_LIMITING.md Exemptions table).
- Unit test must use fakeredis or mocked Upstash REST, not real Redis.

Gate Check:

- [] 20 requests in 60 s all succeed
- [] 21st request in window is silently dropped (200 to Telegram, no task enqueued)
- [] 21st request after 61 s succeeds (window reset)
- [] Different chat_ids have independent windows
- [] Unit test covers all 5 cases from TESTING.md §3

PROMPT 016 -- OpenAPI Spec + API Documentation

SKILLS: [openapi-spec-generation](#) . [python-fastapi-development](#)

Generate and expose a complete OpenAPI 3.1 spec for the Recall backend.

FastAPI auto-generates docs at /docs (Swagger UI) and /openapi.json.

Enhance the spec with:

1. Proper response schemas for all endpoints using Pydantic models:
 - ItemResponse, SearchResponse, GraphResponse, QuizResponse, ReminderResponse
 - Error responses: ErrorResponse(error: str, message: str)
2. Security scheme definitions:
 - bearerAuth: JWT in httpOnly cookie (document the cookie name: recall_session)
 - telegramInitData: TWA HMAC in Authorization header
3. Tag grouping:
 - webhook: POST /webhook
 - auth: /auth/*
 - items: /api/items

```
- search: /api/search
- graph: /api/graph
- quizzes: /api/quizzes/*
- reminders: /api/reminders
- drive: /api/drive/*
```

4. Export static spec:

```
python -c "import json; from main import app; print(json.dumps(app.openapi()))" > docs/openapi.json
```

Rules:

```
- /docs endpoint must be DISABLED in production (exposing schema = unnecessary surface area).
  if settings.ENV == "production": app.docs_url = None; app.redoc_url = None
- All response schemas must use Pydantic BaseModel -- no untyped dict returns.
- raw_text must NOT appear in any response schema -- it is encrypted and internal only.
- openapi.json must be committed to docs/ for reference -- it is not a secret.
```

Gate Check:

```
[ ] GET /openapi.json returns valid OpenAPI 3.1 JSON
[ ] /docs returns 404 in production mode (settings.ENV=production)
[ ] All endpoints have proper request and response schemas
[ ] raw_text field absent from all response Pydantic models
[ ] Unit test: test_schema.py imports all Pydantic models and validates example data
```

PROMPT 017 -- Map View: Semantic Hubs + Louvain Visualisation

SKILLS: react-patterns . high-end-visual-design

Add a Hub Map view -- a zoomed-out view showing only semantic hubs (not all nodes).

Toggle in header: [Nodes] [Hubs] [Feed]

Hub Map view in GraphCanvas:

```
Show only semantic_hub nodes (from GET /api/graph hubs array)
Hub size scales with member_ids.length (more members = larger node)
Edges between hubs: draw edge if any member of Hub A is within cosine 0.75 of any member of Hub B
Layout: same D3 force simulation, fewer nodes = more stable layout
```

On click a hub node:

```
NodePanel shows hub details:
- Hub label
- Member count: "32 items"
- Member list: show top 5 items by created_at (titles only)
- [View all members in Feed] button -> switches to Feed view filtered to this hub's member_ids
```

Hub label positioning:

```
Labels render below hub node (never overlapping)
If two hubs are close: stagger labels alternating above/below
```

Rules:

```
- Hub Map uses same GraphCanvas component -- switch via a `mode` prop ("nodes" | "hubs").
- Hub-to-hub edges must use data from graph API -- no new API endpoint.
- Hub node minimum radius: 20px (vs orbital 8px).
- Clicking [View all members] must pass member_ids as a filter to Feed view.
```

Gate Check:

```
[ ] Hub Map renders only semantic hub nodes
[ ] Hub size scales with member count (larger = more items)
[ ] Clicking hub shows NodePanel with member items
```

```
[ ] [View all members] switches to Feed with correct filter
[ ] Vitest: GraphCanvas renders correctly in "hubs" mode
```

PROMPT 018 -- Telegram Login Widget + JWT Issuance

SKILLS: [auth-implementation-patterns](#) . [security](#) . [python-fastapi-development](#)

Implement GET /auth/telegram in backend/routes/auth.py.

Sequence (from AUTH_ARCHITECTURE.md Layer 2b exactly):

1. Collect all query params except 'hash'.
2. Sort alphabetically, join as "key=value\n" string.
3. secret_key = hashlib.sha256(TELEGRAM_BOT_TOKEN.encode()).digest()
4. expected_hash = hmac.new(secret_key, check_string.encode(), hashlib.sha256).hexdigest()
5. Compare: hmac.compare_digest(expected_hash, params['hash']) <- MANDATORY constant-time compare
6. Validate: auth_date within 86400 seconds (1 day).
7. Upsert user by telegram_chat_id.
8. Issue JWT: {"sub": user_id, "chat_id": chat_id, "exp": now + 7 days}
9. Set JWT as httpOnly cookie: recall_session.
10. Redirect to WEBSITE_URL/dashboard.

Cookie attributes (ALL mandatory):

```
httponly=True
secure=True
samesite="lax"
max_age=604800 # 7 days in seconds
```

Rules (CRITICAL):

- NEVER use == for hash comparison -- always hmac.compare_digest().
- auth_date check must use integer comparison, not string comparison.
- JWT must be signed with HS256 using JWT_SECRET -- not RS256 (adds complexity without benefit here).
- On auth failure: return 401 with generic message "Authentication failed" -- no specifics.
- TELEGRAM_BOT_TOKEN must NEVER appear in any log message around this function.

Gate Check:

- ```
[] Valid Telegram Login Widget callback issues JWT cookie
[] Tampered hash returns 401
[] auth_date > 1 day old returns 401
[] Cookie has httpOnly + Secure + SameSite=Lax attributes (verify in browser DevTools)
[] Unit test: all 5 cases from TESTING.md $5 Login Widget section
[] Unit test: hmac.compare_digest used (grep for == and flag if found near hash comparison)
```

## PROMPT 019 -- TWA HMAC Middleware

**SKILLS:** [auth-implementation-patterns](#) . [security](#) . [telegram-mini-app](#)

Implement TWA initData HMAC validation in backend/middleware/twa\_auth.py.

TWA sequence (from AUTH\_ARCHITECTURE.md Layer 2a exactly):

1. Read initData from Authorization header: "TelegramInitData <url\_encoded\_string>"
2. Parse URL-encoded key-value pairs.
3. Extract and remove 'hash' field.
4. Sort remaining pairs alphabetically.
5. Construct data\_check\_string = "key=value\nkey=value\n..."

```
6. secret_key = HMAC-SHA256(key=b"WebAppData", data=TELEGRAM_BOT_TOKEN.encode())
7. expected_hash = HMAC-SHA256(key=secret_key, data=data_check_string.encode()).hexdigest()
8. Compare: hmac.compare_digest(expected_hash, hash_from_initData)
9. Validate: auth_date within 3600 seconds (1 hour).
10. Extract user.id from initData, look up users table, attach to request.
```

Create a FastAPI dependency: `get_twa_user(request) -> UserContext`

Unified auth dependency for `/api/*` routes:

```
get_current_user = get_jwt_user | get_twa_user
Try JWT cookie first; if missing, try TWA header; if both missing: 401.
```

Rules:

- SAME security rules as PROMPT 013: `hmac.compare_digest` mandatory.
- `auth_date` replay window: 3600 s (stricter than login widget -- TWA is more frequent).
- Never log `initData` content -- it contains sensitive user data.
- Unified dependency must not double-authenticate (try one then the other, not both).

Gate Check:

```
[] Valid TWA initData -> 200 with user context attached
[] Tampered TWA hash -> 401
[] Expired auth_date (> 1 hour) -> 401
[] JWT cookie auth still works alongside TWA (unified dependency)
[] Unit test: all 4 cases from TESTING.md §5 TWA section
```

## PROMPT 020 -- Users Table: Upsert on /start

**SKILLS:** [python-fastapi-development](#) . [postgres-best-practices](#)

Implement user creation in `backend/services/user_service.py`.

Function: `async def upsert_user(chat_id: str, db: AsyncConnection) -> int`

Logic:

```
INSERT INTO users (telegram_chat_id)
VALUES ($1)
ON CONFLICT (telegram_chat_id) DO NOTHING
RETURNING id;
```

```
If no row returned (conflict), do:
SELECT id FROM users WHERE telegram_chat_id = $1;
```

Call `upsert_user` in the `/webhook` handler when `update.message.text == "/start"`.

Bot replies: "Welcome to Recall! Forward me any link, voice note, PDF, or image and I'll remember it for you."

Rules:

- `chat_id` must be cast to `VARCHAR` before DB insert -- never store as `INT`.
- `upsert_user` must be idempotent -- calling it 100 times with same `chat_id` creates exactly 1 row.
- No personal data beyond `chat_id` is stored at signup -- no name, no username.
- Return internal `user.id` (`INT`), not `chat_id` -- all downstream references use internal ID.

Gate Check:

```
[] Sending /start twice creates exactly 1 users row
[] upsert_user returns the same integer ID on repeated calls
[] Welcome message sent to user on first /start
[] Unit test: test_user_service.py covers idempotency with mocked DB
```



## PROMPT 021 -- Pre-Deployment Checklist Execution

**SKILLS:** python-pro

Execute the full AUDIT\_CHECKLIST.md before deploying to production.

Run through all 4 sections of AUDIT\_CHECKLIST.md:

SECTION A -- Database & Infrastructure

```
[] Neon: SELECT extname FROM pg_extension -> returns vector, pg_trgm
[] All 8 tables present
[] All 4 indices present
[] Current and next month's partitions exist
[] Upstash Redis: ping returns PONG
```

SECTION B -- Security & Secrets

```
[] git grep -Ei "gemini_api_key|groq_api_key|telegram_bot_token|fernet_key|jwt_secret" -> 0 results
[] .env not committed: git status -> .env absent
[] All 15 Render env vars set
[] Both Vercel env vars set
[] FERNET_KEY: valid base64, 32 bytes decoded
[] Google OAuth redirect URI matches exactly
```

SECTION C -- Deployment

```
[] modal app list -> 3 apps deployed (whisper, llama, embed)
[] Render deploy: /health returns 200
[] Telegram webhook set: getWebhookInfo returns correct URL
[] Vercel deploy: dashboard URL loads
```

SECTION D -- Operations

```
[] Uptime Robot monitor created for /health, 5-minute interval
[] Scheduler jobs visible in startup logs
[] Rate limiter test: 21st request dropped
```

Rules:

- Run every check -- do not skip any.
- Any failed check must be fixed before proceeding.
- Document results in docs/DEPLOYMENT\_SIGN\_OFF.md.

Gate Check:

```
[] All 4 sections completed with all items checked
[] DEPLOYMENT_SIGN_OFF.md created with date and sign-off
[] Zero security failures in Section B
```

## PROMPT 022 -- Bot Command System: /help, /list, /delete, /stats

**SKILLS:** telegram-bot-builder . python-fastapi-development

Extend backend/routes/webhook.py to handle the full bot command set.

Commands to implement:

/help:

```
Reply with formatted message listing all commands:
" Recall Commands:
/start -- Set up your account
/search <query> -- Find saved items
```

```

/list -- Show your last 10 saves
/delete <id> -- Delete an item by ID
/quiz -- Get a due quiz question
/remind <time> <message> -- Set a reminder (e.g. /remind 2h Review ML notes)
/stats -- Your knowledge stats
/streak -- Current save streak
/connect_drive -- Connect Google Drive backup"

/list:
SELECT id, title, source_type, created_at FROM items
WHERE user_id = $1 ORDER BY created_at DESC LIMIT 10
Reply:
" Your last 10 saves:\n
1. [url] Article title (2 hours ago)
2. [voice] Voice note (yesterday)
..."
Include item IDs for use with /delete.

/delete <id>:
DELETE FROM items WHERE id = $1 AND user_id = $2 (MUST include user_id check)
On success: "Deleted [x]"
On not found or not owned: "Item not found."
On invalid ID: "Please provide a valid item ID: /delete 42"

/stats:
Aggregated stats for the user:
SELECT COUNT(*), source_type FROM items WHERE user_id=$1 GROUP BY source_type
SELECT COUNT(*) FROM quizzes WHERE user_id=$1
SELECT streak_count FROM users WHERE id=$1
Reply:
" Your Recall stats:
Total saves: 47
-- Links: 23 | Voice: 12 | PDFs: 8 | Images: 4
Quizzes answered: 31
Current streak: 5 days"

Rules:
- /delete MUST verify user_id -- never delete by ID alone (IDOR prevention).
- /list shows IDs so users can use /delete -- IDs must be accurate.
- /stats query must use GROUP BY -- not separate COUNT queries per type.
- Unknown commands: "Unknown command. Type /help to see all commands."
- All commands: upsert user first (in case somehow user row missing).

Gate Check:
[] /help sends formatted command list
[] /list shows 10 most recent items with IDs
[] /delete 42 deletes only if user owns item 42
[] /delete 42 with another user's ID returns "Item not found."
[] /stats returns accurate grouped counts
[] Unit test: test_commands.py covers /delete cross-user attempt

```

## PROMPT 023 -- Production Go-Live Sequence

### SKILLS: deployment-engineer

Execute the exact deployment order from DEPLOYMENT.md:

1. Neon DB

- > Run full DDL from BACKEND\_SCHEMA.md
  - > Verify extensions, tables, indices via verify.py
2. Upstash Redis
- > Create database, copy REST credentials to Render
3. Modal
- > modal deploy backend/modal\_apps/modal\_whisper.py
  - > modal deploy backend/modal\_apps/modal\_llama.py
  - > modal deploy backend/modal\_apps/modal\_embed.py
  - > Test each endpoint with a sample payload
4. Google OAuth
- > Create credentials in Google Cloud Console
  - > Set redirect URI to Render URL
  - > Enable Drive API
5. Generate secrets
- > FERNET\_KEY: python -c "from cryptography.fernet import Fernet; print(Fernet.generate\_key().decode())"
  - > JWT\_SECRET: python -c "import secrets; print(secrets.token\_hex(32))"
6. Render backend
- > Set all 15 env vars
  - > Deploy
  - > GET /health -> {"status": "ok"}
  - > Register Telegram webhook
7. Vercel frontend
- > Set VITE\_API\_URL + VITE\_BOT\_USERNAME
  - > Deploy
  - > BotFather /setdomain -> Vercel URL
8. Uptime Robot
- > New monitor -> /health -> 5-minute interval
9. End-to-end smoke test:
- > Forward a text message to bot -> items row created
  - > Forward a voice note -> transcription + embedding
  - > Open web dashboard -> mind map renders
  - > Search via bot -> results returned

**Rules:**

- Deployment must follow the exact order -- Neon before Render, Render before Vercel.
- Smoke test must pass before declaring go-live.
- Save the Render URL, Vercel URL, and webhook URL in docs/DEPLOYMENT\_SIGN\_OFF.md.

**Gate Check:**

- [ ] All 8 deployment steps completed in order
- [ ] Smoke test: text, voice, web dashboard, search all pass
- [ ] Uptime Robot shows green status
- [ ] Webhook registered and showing 0 pending updates

## PROMPT 024 -- FastAPI App Skeleton + Health Endpoint

**SKILLS:** python-fastapi-development . fastapi-pro

Create backend/main.py with:

- FastAPI app with title="Recall API", version="0.1.0"

- GET /health -> returns {"status": "ok", "timestamp": <ISO UTC>} -- no DB query, pure in-memory
- Startup event: call settings.validate\_crypto\_keys()
- Startup event: log "Recall API started" -- nothing sensitive
- CORS: allow only WEBSITE\_URL origin (from settings). Not wildcard.
- Global exception handler: catches unhandled exceptions, logs them, returns {"error": "internal\_server\_error"} with 500 -- never expose stack trace to client.

## Rules:

- /health must respond in < 5 ms -- no DB, no Redis, no external calls.
- CORS must NOT use allow\_origins=["\*"] -- use [settings.WEBSITE\_URL].
- Exception handler must log the exception internally but return generic message to client.

## Gate Check:

- [ ] GET /health returns 200 with status and timestamp
- [ ] Missing FERNET\_KEY at startup raises clear error (not a 500 later)
- [ ] CORS rejects requests from non-WEBSITE\_URL origins
- [ ] Unit test: test\_health.py asserts /health response schema

## PROMPT 025 -- Daily Digest Bot Message

**SKILLS:** telegram-bot-builder . async-python-patterns

Add a daily digest job to APScheduler.

Job 6 -- daily\_digest\_sender (new job to add to scheduler):  
 Trigger: Daily cron, 08:00 UTC (adjustable per timezone\_offset)  
 Purpose: Send users a personalised daily summary

## Algorithm:

For each user with last\_activity\_date within last 7 days:  
 SELECT items saved yesterday (created\_at BETWEEN yesterday\_start AND yesterday\_end)  
 SELECT quizzes due today (next\_review = CURRENT\_DATE)  
 SELECT streak\_count

## Format message:

" Good morning! Your Recall daily digest:

Yesterday you saved {N} items.  
 New knowledge: {first 3 titles}

Quizzes due today: {quiz\_due\_count}  
 Type /quiz to start.

{streak\_count} day streak -- keep it up!"

Send via Telegram bot API to each user's chat\_id.

## Rules:

- Only send to users active in last 7 days (avoid messaging churned users).
- Timezone: use user.timezone\_offset to send at ~08:00 local time.  
 (Run job every hour from 06:00-10:00 UTC; check if 08:00 local for each user)  
 Alternatively: run at 08:00 UTC as approximation -- document this limitation.
- If user saved 0 items yesterday: send only the quiz reminder (not the "you saved N items" part).
- misfire\_grace\_time=3600 for this job (daily -- 1 hour grace window is fine).
- Add a user preference to disable digest (future: users.digest\_enabled BOOLEAN DEFAULT TRUE).

## Gate Check:

- [ ] Digest message sent correctly formatted with item count and quiz count

```
[] Users with 0 items yesterday receive only quiz reminder
[] Users inactive > 7 days are skipped
[] Job registered in scheduler with correct cron trigger
[] Unit test: test_daily_digest.py with mocked Telegram API and DB
```

## PROMPT 026 -- GitHub Actions CI Pipeline

**SKILLS:** python-pro . testing-patterns

Create .github/workflows/ci.yml -- automated CI on every push and PR.

Jobs:

1. backend-test:

- runs-on: ubuntu-latest
- Python 3.11
- pip install -r backend/requirements.txt
- cd backend && pytest --tb=short -q
- Fail PR if any test fails

2. frontend-test:

- runs-on: ubuntu-latest
- Node 20
- cd frontend && npm ci && npm run test
- Fail PR if any Vitest test fails

3. security-scan:

- runs-on: ubuntu-latest
- Run bandit -r backend/ -ll (Python security linter)
- Flag any HIGH or CRITICAL issues as build failure
- Run npm audit --audit-level=high in frontend/
- Fail PR if HIGH/CRITICAL vulnerabilities found

4. lint:

- ruff check backend/ (Python linter)
- ESLint frontend/src/

Environment variables in CI:

- Use GitHub Secrets for test credentials
- COMPUTE\_PROVIDER=groq in all CI runs (never Modal in CI)
- DATABASE\_URL: point to a Neon CI branch (created and deleted per run)

Rules:

- CI must NEVER use production credentials -- use separate CI secrets.
- bandit scan is mandatory -- must not be skipped or suppressed for HIGH issues.
- PR merge must be blocked until all 4 jobs pass.
- CI run must complete in < 5 minutes.

Gate Check:

- [ ] Push to main triggers all 4 jobs
- [ ] PR with failing test shows red check on GitHub
- [ ] bandit scan flags hardcoded secrets as HIGH (test with a fake token in a test file)
- [ ] CI completes in < 5 minutes on GitHub Actions runners
- [ ] No production credentials in .github/workflows files

## PROMPT 027 -- Database Connection Pool

**SKILLS:** [neon-postgres](#) . [postgres-best-practices](#) . [async-python-patterns](#)

Create backend/db/connection.py -- async PostgreSQL connection pool using psycopg3 (psycopg[async]).

Requirements:

- Use AsyncConnectionPool with min\_size=1, max\_size=5 (Neon free tier limits).
- Connection string from settings.DATABASE\_URL -- must include ?sslmode=require.
- Expose: async def get\_db() -> AsyncConnection (FastAPI dependency).
- Pool opens on FastAPI startup lifespan event; closes on shutdown.
- Add a db\_health\_check() coroutine: SELECT 1 -- used by /health if needed.

Rules:

- Never use synchronous psycopg2 -- all DB operations must be async.
- Pool size max=5: Neon free tier allows 10 connections; leave headroom for APScheduler.
- Connection timeout: 5 s. Query timeout: 30 s. Never block indefinitely.
- Wrap all queries in try/except psycopg.OperationalError -- log, re-raise as HTTPException(503).

Gate Check:

- [ ] FastAPI startup successfully opens pool
- [ ] get\_db() dependency yields a valid connection
- [ ] Pool closes cleanly on shutdown
- [ ] Unit test: test\_db\_connection.py mocks pool and verifies dependency injection

## PROMPT 028 -- Empty States + Loading Skeletons

**SKILLS:** [react-ui-patterns](#) . [high-end-visual-design](#)

Create empty state and loading skeleton components for all data views.

Empty states (frontend/src/components/EmptyState.jsx):

Graph empty (0 items):

- Centered, full canvas area
- Animated single orbital node pulsing gently
- Text (Outfit 18px): "Your constellation is empty"
- Subtext (Inter 14px, --text-secondary): "Forward any link, voice note, or PDF to your Telegram bot to start mapping your knowledge."
- Button: [Open Telegram Bot] -> opens t.me/{VITE\_BOT\_USERNAME}

Feed empty (filtered result = 0):

- Icon: @phosphor-icons MagnifyingGlass (64px, --text-tertiary)
- Text: "Nothing found"
- Subtext: "Try a different filter or search term."

Search empty:

- Icon: @phosphor-icons Binoculars
- Text: "No results for '{query}'"

Loading skeletons (frontend/src/components/Skeleton.jsx):

- GraphSkeleton: animated gradient wave across canvas area (shimmer effect)
- FeedCardSkeleton: card-shaped pulse placeholder, 2-column grid
- NodePanelSkeleton: right panel with animated placeholder lines

Shimmer animation:

- background: linear-gradient(90deg, var(--surface-glass) 25%, rgba(255,255,255,0.05) 50%, var(--surface-glass))

```
75%)
```

```
background-size: 200% 100%
```

```
@keyframes shimmer: background-position 1.5s infinite linear
```

Rules:

- Skeletons must match the exact dimensions of loaded content (no layout shift).
- Empty states must not show "You have 0 items" -- use encouraging language.
- Skeleton shown until first API response arrives -- not on refetch.
- VITE\_BOT\_USERNAME in empty state link -- not hardcoded.

Gate Check:

- [ ] New user sees graph empty state with pulsing node
- [ ] Feed with active filter shows "Nothing found" empty state (not generic empty)
- [ ] Skeleton shimmer animation runs smoothly (no jank)
- [ ] VITE\_BOT\_USERNAME correctly links to the bot in empty state CTA
- [ ] Vitest: EmptyState renders correct variant per prop

## PHASE 3: EMBEDDINGS & SEMANTIC SEARCH

### PROMPT 029 -- GET /api/items Endpoint

**SKILLS:** [python-fastapi-development](#) . [postgres-best-practices](#)

Implement GET /api/items in backend/routes/api.py.

Query params:

- page: int = 1
- limit: int = 20 (max 50 -- validate and clamp)
- source\_type: str | None (filter: url/voice/pdf/image/text)
- tag: str | None (filter: items containing this tag)
- from\_date: date | None
- to\_date: date | None

Response:

```
{
 "items": [
 {
 "id": int,
 "title": str,
 "summary": str,
 "source_type": str,
 "source_url": str | null,
 "tags": list[str],
 "created_at": str
 }
],
 "total": int,
 "page": int,
 "pages": int
}
```

SQL:

```
SELECT id, title, summary, source_type, source_url, tags, created_at
FROM items
WHERE user_id = $1
[AND source_type = $2]
[AND $3 = ANY(tags)]
[AND created_at >= $4]
[AND created_at <= $5]
ORDER BY created_at DESC
LIMIT $6 OFFSET $7
```

**Rules:**

- NEVER return raw\_text in this endpoint -- encrypted ciphertext must not reach client.
- limit must be clamped to max 50 -- reject 400 if limit > 50.
- All WHERE clauses dynamically composed -- do NOT build SQL strings. Use a query builder pattern.
- Offset = (page - 1) \* limit -- validate page >= 1.
- Auth: get\_current\_user dependency -- all queries scoped to authenticated user.

**Gate Check:**

- [ ] GET /api/items returns paginated results
- [ ] source\_type filter works correctly
- [ ] tag filter: items with matching tag appear, items without don't
- [ ] raw\_text is absent from all response objects
- [ ] User A cannot see User B's items via page/limit manipulation
- [ ] Unit test: test\_items\_api.py verifies pagination math and filter composition

## PROMPT 030 -- DELETE /api/items/{id} + IDOR Protection

**SKILLS:** python-fastapi-development . security . idor-testing

Implement DELETE /api/items/{item\_id} in backend/routes/api.py.

**Logic:**

1. Auth: get\_current\_user -> user\_id
2. DELETE FROM items WHERE id = \$1 AND user\_id = \$2 RETURNING id
3. If RETURNING returns no row: HTTPException(404, "Item not found")
4. If row returned: return 204 No Content
5. Also delete associated quizzes: DELETE FROM quizzes WHERE item\_id = \$1 AND user\_id = \$2

**IDOR Test (write this in the unit test):**

- Create User A with item ID 5
- Authenticate as User B
- Attempt DELETE /api/items/5
- Must return 404, not 204 -- item must still exist in DB

**Rules (CRITICAL -- IDOR prevention):**

- DELETE query MUST include AND user\_id = \$2 -- WHERE id = \$1 alone is FORBIDDEN.
- Return 404 (not 403) when item not found or not owned -- prevents enumeration.
- Associated quizzes MUST be deleted in the same transaction.
- Cascade DELETE on items table (ON DELETE CASCADE) handles semantic\_hubs references automatically.
- Log deletion event: {user\_id, item\_id, source\_type} for audit trail.

**Gate Check:**

- [ ] DELETE own item -> 204, item gone from DB
- [ ] DELETE another user's item -> 404, item still in DB (IDOR prevented)
- [ ] DELETE non-existent item -> 404
- [ ] Quizzes for deleted item are also removed
- [ ] Unit test: test\_idor.py specifically tests cross-user delete attempt



## PROMPT 031 -- Text Ingestion + Task Worker Loop

**SKILLS:** [async-python-patterns](#) . [python-fastapi-development](#)

Create the background task worker in backend/worker.py.

The worker:

1. Starts as a FastAPI background task on startup (`asyncio.create_task`).
2. Runs: `while True: task = await redis_client.brpop("recall:tasks", timeout=5)`
3. Parses task JSON: `{chat_id, user_id, content_type, ...}`
4. Acquires `asyncio.Semaphore(3)` before processing.
5. Routes by `content_type` to the correct ingester:
  - "text" -> `text_ingester.process(task)`
  - "url" -> `url_ingester.process(task)` [from PROMPT 046]
  - "voice" -> stub returning None (Phase 2)
  - "pdf" -> stub returning None (Phase 3)
  - "image" -> stub returning None (Phase 3)
6. On any exception: log with task context, write to `dead_letter_queue`, send user error message.

Implement `text_ingester.process()`:

- Save `raw_text` (Fernet-encrypted) to items with `source_type='text'`.
- Bot reply: `"Saved [x] -- [{first 80 chars}]"`

Rules:

- `Semaphore(3)` is MANDATORY -- never bypass for "fast" tasks.
- Worker must not crash on individual task failure -- catch all exceptions, continue loop.
- If Redis is unreachable for > 30 s, log CRITICAL and continue trying (do not exit process).
- task JSON must include `user_id` (internal DB ID), not just `chat_id`.

Gate Check:

- [ ] Text message forwarded to bot creates items row within 5 s
- [ ] Worker continues running after a task raises an exception
- [ ] Semaphore limits concurrent processing to 3 simultaneous tasks
- [ ] Unit test: `test_worker.py` with mocked Redis and ingester, verifies routing

## PROMPT 032 -- Rate Limit Testing: Redis Pipeline Atomicity

**SKILLS:** [python-testing-patterns](#) . [async-python-patterns](#)

Verify the atomicity and correctness of the Redis sliding window rate limiter under concurrency.

Test file: `backend/tests/test_rate_limiter_concurrency.py`

Scenario 1 -- Concurrent requests at limit boundary:

- Simulate 20 concurrent requests from the same `chat_id` using `asyncio.gather`.
- Assert: ALL 20 succeed (no race condition incorrectly rejects requests under limit).

Scenario 2 -- Concurrent requests over limit:

- Simulate 25 concurrent requests.
- Assert: exactly 20 succeed, exactly 5 are rejected.
- Assert: no request is double-counted (race condition check).

Scenario 3 -- Window expiry under load:

- Send 20 requests. Wait 61 s (mock time). Send 20 more.
- Assert: all 40 requests succeed (two separate windows).

Scenario 4 -- Different users are independent:

50 requests from chat\_id\_A and 50 requests from chat\_id\_B.  
 Assert: exactly 20 from A succeed, exactly 20 from B succeed.  
 Assert: A and B do not share quota.

#### Implementation note:

The Upstash REST pipeline is atomic (single HTTP POST) -- verify this prevents the race condition.  
 Use fakeredis.aioredis for synchronous tests; for concurrency tests, use asyncio.gather with real Upstash dev instance.

#### Rules:

- Concurrency tests must actually run async operations concurrently (asyncio.gather).
- Never use time.sleep() for window expiry -- use freezegun to advance time.
- Tests must pass consistently (not flaky) -- run each 10 times in CI.

#### Gate Check:

- [ ] Scenario 1: all 20 concurrent succeed
- [ ] Scenario 2: exactly 5 rejected (no race condition accepting more/fewer)
- [ ] Scenario 3: window reset allows new requests
- [ ] Scenario 4: user isolation confirmed
- [ ] Tests are not flaky (run 10 times: all pass)

## PROMPT 033 -- Drive Sync Service

### SKILLS: python-pro . async-python-patterns

Create backend/services/drive\_sync.py -- exports Recall items as a Google Doc.

```
async def sync_user_to_drive(user_id: int, db: AsyncConnection):
1. SELECT google_refresh_token FROM users WHERE id = $1
2. decrypt(refresh_token) -> plaintext
3. Exchange refresh_token for access_token via Google token endpoint.
4. Fetch user's last 50 items: SELECT title, summary, source_url, created_at FROM items WHERE user_id = $1 ORDER BY
 created_at DESC LIMIT 50
5. Format as Markdown document.
6. Call Google Docs API to create/update a file named "Recall -- {username} -- {date}".
7. Place in a folder named "Recall" in user's Drive (create folder if absent).
8. Do NOT store access_token -- use it once and discard.
```

Manual trigger: POST /api/drive/sync (authenticated) for immediate sync.  
 Scheduled: drive\_nudge\_sender job triggers sync for connected users weekly.

#### Rules:

- access\_token never written to DB or logs -- use in-memory only within request.
- Drive file contains only: title, summary, source\_url -- NEVER raw\_text (encrypted).
- If Drive API returns 401 (token revoked): UPDATE google\_refresh\_token = NULL; notify user via bot.
- If Drive API returns 403 (quota): log and skip this user; try next user.
- drive.file scope: Recall can only access files IT created -- cannot browse user's Drive.

#### Gate Check:

- [ ] sync\_user\_to\_drive creates a Google Doc in user's Drive
- [ ] Doc contains only title/summary/source\_url -- no raw\_text
- [ ] Revoked token (401) -> refresh\_token cleared from DB + user notified
- [ ] access\_token does not appear in any log output
- [ ] Unit test: test\_drive\_sync.py with mocked Google API client

## PHASE 4: WEB DASHBOARD FOUNDATION & UI

### PROMPT 034 -- Security Penetration Tests: IDOR + Injection

**SKILLS:** idor-testing . sql-injection-testing . web-security-testing

Write a security-focused test suite covering IDOR and injection scenarios.

File: backend/tests/security/test\_idor.py

IDOR Tests:

- T1: User B tries GET /api/items -> only sees own items (not User A's)
- T2: User B tries DELETE /api/items/{user\_A\_item\_id} -> 404
- T3: User B tries POST /api/quizzes/{user\_A\_quiz\_id}/answer -> 404
- T4: User B tries DELETE /api/reminders/{user\_A\_reminder\_id} -> 404
- T5: User B tries GET /api/graph -> only sees own nodes (0 of User A's)
- T6: User B tries POST /api/drive/sync -> only syncs own data

SQL Injection Tests (file: test\_sql\_injection.py):

- T1: POST /api/search with query="'; DROP TABLE items;--" -> 200, no error, no items deleted
- T2: GET /api/items?tag=" OR '1'='1" -> 200, only user's items returned
- T3: POST /webhook with text containing SQL keywords -> processed safely, no injection
- T4: /auth/telegram with tampered id param containing SQL -> 401, no DB error

XSS Tests (file: test\_xss.py):

- T1: Item saved with title="<script>alert(1)</script>" -> title returned as-is in JSON (not executed)
  - T2: Search query with XSS payload -> returned in JSON, not reflected as HTML
- (XSS note: since backend is JSON API + React frontend auto-escapes, XSS is largely mitigated at framework level)

Rules:

- These tests run against a test DB -- never production.
- SQL injection tests verify BOTH no data change AND no 500 error.
- IDOR tests must create two real users (User A, User B) with real tokens.
- All tests must pass -- any failure is a BLOCKING security issue.

Gate Check:

- [ ] All IDOR tests (T1-T6) pass -- no cross-user data access
- [ ] SQL injection T1: items table count unchanged after injection attempt
- [ ] SQL injection T2: parameterised query prevents injection
- [ ] All tests run against test DB only
- [ ] CI: security test suite runs on every PR

### PROMPT 035 -- Login Page / Landing Page

**SKILLS:** react-best-practices . ui-ux-pro-max . high-end-visual-design

Create frontend/src/pages/Login.jsx -- the landing page for unauthenticated users.

Layout:

- Full viewport, Cosmic Noir background with both nebula blobs
- Centred content card (glass-card, max-width 440px)

Content:

```
Logo: " Recall" (Outfit 700, 32px)
Tagline: "Your second brain. Zero friction." (Outfit 400, 18px, --text-secondary)

Hero visual: animated mini-constellation (5 nodes, 3 edges) rendered on a small canvas
-- same GraphCanvas renderer but with demo data, no interaction, 30% opacity

Feature pills (3 horizontal):
 Voice Notes . Links . PDFs

CTA section:
 "Login with Telegram" button:
 - Renders the official Telegram Login Widget script
 - data-telegram-login="{VITE_BOT_USERNAME}"
 - data-size="large" data-radius="8" data-auth-url="{VITE_API_URL}/auth/telegram"
 - Button style: glass-card, mint-teal accent border

Footer: "Free. No signup form. Works in < 5 seconds."

Route guard:
 If user already has valid JWT cookie -> redirect to /dashboard.
 If user lands on /dashboard without JWT -> redirect to /login.

Rules:
- Telegram Login Widget script loaded from https://telegram.org/js/telegram-web-app.js only.
- VITE_BOT_USERNAME and VITE_API_URL from import.meta.env -- not hardcoded.
- Hero mini-canvas must use same GraphCanvas but read-only (no click, no pan).
- Login page must be accessible without JavaScript for the static HTML (widget works without JS).

Gate Check:
[] Unauthenticated /dashboard -> redirected to /login
[] Authenticated user at /login -> redirected to /dashboard
[] Telegram Login Widget renders and visible
[] Hero mini-constellation animates on page load
[] VITE_BOT_USERNAME correct in widget script
[] Vitest: Login page renders without crash
```

## PROMPT 036 -- Hybrid Search: Vector + Trigram

**SKILLS:** postgres-best-practices . postgresql-optimization

Implement backend/services/search\_service.py and POST /api/search endpoint.

```
async def hybrid_search(query: str, user_id: int, db: AsyncConnection) -> list[SearchResult]:
```

Step 1 -- Vector search:

```
 query_embedding = await embed_text(query)
 SELECT id, title, summary, source_type, source_url, created_at,
 embedding <=> $1 AS vector_distance
 FROM items
 WHERE user_id = $2
 ORDER BY vector_distance
 LIMIT 20
```

Step 2 -- GIN trigram search:

```
 SELECT id, title, summary, source_type, source_url, created_at,
 similarity(summary, $1) AS text_score
 FROM items
 WHERE user_id = $2 AND summary % $1
```

```
ORDER BY text_score DESC
LIMIT 20
```

Step 3 -- Reciprocal Rank Fusion (RRF):

```
score = 1/(rank_vector + 60) + 1/(rank_text + 60)
Merge, deduplicate by id, sort by score desc, return top 5.
```

Telegram /search command:

```
/search <query> -> call hybrid_search -> bot sends numbered list:
1. {title} [{source_type}] - {first 100 chars of summary}
```

Rules:

- ALL search queries MUST include WHERE user\_id = \$2 -- zero tolerance for cross-user leakage.
- GIN search must use similarity() from pg\_trgm -- NOT LIKE (different index).
- Vector search target: < 10 ms (use EXPLAIN ANALYZE in tests).
- Text search target: < 5 ms.
- Result summaries returned to user are PLAINTEXT -- never decrypt raw\_text for display.

Gate Check:

```
[] /search "machine learning" returns semantically relevant items (not just keyword matches)
[] EXPLAIN ANALYZE shows Index Scan on idx_items_embedding for vector query
[] EXPLAIN ANALYZE shows Bitmap Index Scan on idx_items_text_gin for text query
[] Unit test: test_search.py verifies WHERE user_id is always present in generated SQL
[] Unit test: test_search.py verifies user A cannot see user B's results
```

## PROMPT 037 -- Settings Page (Timezone, Preferences)

**SKILLS:** react-best-practices . python-fastapi-development

Create a minimal settings page/panel for user preferences.

Settings accessible from profile dropdown -> [Settings].

Rendered as a slide-in panel (same animation as NodePanel) from the right.

Settings available in v1:

1. Timezone offset:

```
GET /api/me -> returns {timezone_offset: int, streak_count: int, drive_connected: bool}
Dropdown: UTC-12 to UTC+14 (offset in hours, stored as minutes in DB)
PATCH /api/me -> body: {timezone_offset: int}
Updates users.timezone_offset
```

2. Stats display:

- Total saves, streak, quizzes answered (read-only display)

3. Account:

- [Logout] button -> POST /auth/logout
- [Delete account] (DANGER ZONE): sends DELETE /api/me

DELETE /api/me (backend):

1. Verify auth
2. DELETE FROM users WHERE id = \$1 (CASCADE deletes all items, quizzes, reminders, hubs)
3. Clear cookie
4. Return 204

Rules:

- DELETE /api/me must require the user to type "DELETE" in a confirmation input before enabling button.
- CASCADE delete verified: single DELETE FROM users row removes all child rows.

- PATCH /api/me: only update fields in the request body -- never overwrite unrelated fields.
- timezone\_offset stored as minutes (INT) -- convert from hours x 60 in API layer.

Gate Check:

- [ ] PATCH /api/me updates timezone\_offset
- [ ] Reminders now fire at correct local time after timezone change
- [ ] DELETE /api/me requires "DELETE" confirmation text
- [ ] After DELETE /api/me: all items, quizzes, reminders removed from DB (CASCADE)
- [ ] Unit test: test\_settings.py covers PATCH and DELETE /api/me

## PROMPT 038 -- Chrome Extension: Manifest V3 + Popup

**SKILLS:** [chrome-extension-developer](#) . [auth-implementation-patterns](#)

Create frontend/extension/ directory with a Chrome Manifest V3 extension.

manifest.json:

```
{
 "manifest_version": 3,
 "name": "Recall",
 "version": "1.0.0",
 "description": "Save the current page to Recall",
 "permissions": ["activeTab", "storage", "cookies"],
 "host_permissions": ["<all_urls>"],
 "action": {
 "default_popup": "popup.html",
 "default_icon": {"16": "icons/icon16.png", "48": "icons/icon48.png"}
 },
 "content_security_policy": {
 "extension_pages": "script-src 'self'; object-src 'self'"
 }
}
```

popup.html + popup.js:

- If user has auth token (from chrome.storage.local): show "Save to Recall" button.
- If no auth token: show "Login with Telegram" link -> opens WEBSITE\_URL/auth/telegram in new tab.
- On "Save to Recall" click:
  1. chrome.tabs.query({active:true, currentWindow:true}) -> get current tab URL and title.
  2. POST to VITE\_API\_URL/api/items with {url, title} and auth token in Authorization header.
  3. Show success: "Saved [x]" or error message.

Rules:

- Extension must NOT use eval() or inline scripts (CSP violation in MV3).
- Auth token stored in chrome.storage.local (not localStorage -- cross-origin safer).
- Never store TELEGRAM\_BOT\_TOKEN or any server secrets in extension code.
- VITE\_API\_URL must be configurable -- not hardcoded in extension.
- POST /api/items must validate JWT before processing -- same auth middleware as all other /api/\* routes.

Gate Check:

- [ ] Extension loads in Chrome without manifest errors
- [ ] "Save to Recall" on any webpage creates an items row
- [ ] Unauthenticated state shows login link (not crash)
- [ ] No inline scripts in popup.html (CSP compliant)
- [ ] POST /api/items uses same JWT auth as other endpoints

## PROMPT 039 -- /api/search REST Endpoint + Auth Guard

**SKILLS:** python-fastapi-development . auth-implementation-patterns

Create backend/routes/api.py with POST /api/search.

Request: {"query": str, "limit": int = 5}

Response: [{"id": int, "title": str, "summary": str, "source\_type": str, "source\_url": str | null, "created\_at": str}]

Auth dependency: get\_current\_user(request) -> verifies JWT cookie -> returns user\_id

- httpOnly cookie "recall\_session"
- Verify HS256 signature with JWT\_SECRET
- Check exp claim
- Return {"user\_id": int, "chat\_id": str}
- On failure: HTTPException(401)

Apply auth dependency to ALL /api/\* routes as a FastAPI Depends.

Rules:

- get\_current\_user must use python-jose or PyJWT -- not manual base64 decoding.
- user\_id from JWT must be used for ALL DB queries -- never trust a user\_id from request body.
- JWT must be validated in constant time (python-jose does this automatically).
- 401 response must NOT reveal whether the token is expired vs. invalid vs. missing.
- Return 401, not 403, for unauthenticated requests.

Gate Check:

- [ ] POST /api/search with valid JWT returns results
- [ ] POST /api/search without cookie returns 401
- [ ] POST /api/search with expired JWT returns 401
- [ ] Tampering with JWT payload (change user\_id) returns 401
- [ ] Unit test: test\_auth.py covers all 4 cases from TESTING.md §5 Login Widget JWT section

## PROMPT 040 -- Tag System: Auto-Generate + Filter

**SKILLS:** python-fastapi-development . postgres-best-practices

Implement auto-tag generation and tag-based filtering.

Tag generation (in AI cascade, after summary):

Add to AICascade.summarise() response: {"summary": str, "tags": list[str]}

Prompt to Llama 3:

"Generate 3-5 single-word or two-word tags for this content. Output ONLY a JSON array. Example: [\"machine learning\", \"python\", \"research\"]"

Parse JSON tags from response. On parse failure: tags = [].

Store in items.tags TEXT[] column.

GET /api/tags (new endpoint):

```
SELECT DISTINCT unnest(tags) AS tag, COUNT(*) AS count
FROM items WHERE user_id = $1
GROUP BY tag ORDER BY count DESC LIMIT 50
Returns: [{"tag": str, "count": int}]
```

GET /api/items?tag=<tagname> (extend PROMPT 018):

```
WHERE $3 = ANY(tags)
(already implemented -- verify it works with this tag format)
```

```

Bot /tags command:
 Call GET /api/tags logic
 Reply: " Your top tags:\n1. machine learning (12)\n2. python (8)\n3. research (6)..."

Rules:
- Tags stored as TEXT[] -- no separate tags table needed (pg native array).
- Tag generation must not block item save -- if tag LLM call fails: save with tags=[].
- Max 5 tags per item -- slice list[:5] before INSERT.
- Tag values: lowercase, strip whitespace before storing.
- Unit test: test_tags.py verifies tag extraction and normalisation.

Gate Check:
[] Items saved after this prompt have auto-generated tags in DB
[] GET /api/tags returns tag frequency list
[] GET /api/items?tag=python returns only items with python tag
[] Tags are lowercase in DB
[] Unit test: tag extraction from LLM response handles invalid JSON gracefully

```

## PROMPT 041 -- Search Result Ranking: Map-Reduce RAG

**SKILLS:** [postgres-best-practices](#) . [postgresql-optimization](#) . [async-python-patterns](#)

Upgrade the search pipeline to use Map-Reduce RAG for query answering.

Current: search returns a list of matching items.

New: search also generates a synthesised answer using context from top results.

POST /api/search -- enhanced response:

```

{
 "answer": "Based on your saves, machine learning covers...", // synthesised
 "sources": [{ "id": int, "title": str, "summary": str, "relevance": float}],
 "query": str
}

```

Map-Reduce flow:

Map: take top-5 search results -> extract summary for each

Reduce: pass all 5 summaries to Llama 3:

"Answer the user's question using ONLY the provided context. Question: {query}"

Context: {summaries\_joined}

Answer concisely in 2-3 sentences."

Return answer + source list.

Telegram /search -- enhanced reply:

Query: {query}

{answer}

Sources:

1. [{source\_type}] {title}

2. [{source\_type}] {title}

...

Fallback: if RAG generation fails (cascade exhaustion): return sources only, no answer.

Rules:

- answer must be generated from context ONLY -- instruct the model not to hallucinate.
- Summaries passed to LLM must be PLAINTEXT -- never pass encrypted raw\_text.
- Map-Reduce must use the same AICascade -- no new AI client.



- If < 3 sources found: skip RAG generation, return sources only.
- Max total prompt size: 3000 tokens (count approx: 4 chars per token).

Gate Check:

- [ ] /search "machine learning" returns both an answer and source list
- [ ] answer is generated only from provided summaries (verify by checking against unrelated content)
- [ ] < 3 results -> no answer generated (sources only)
- [ ] RAG failure -> sources returned without answer (no crash)
- [ ] Unit test: test\_rag\_search.py with mocked AICascade

## PROMPT 042 -- README.md for GitHub

### SKILLS: python-pro . react-best-practices

Create a production-quality README.md at the repo root.

Structure:

```
Recall -- AI Knowledge Management
[constellation screenshot or GIF here]

> Forward anything to Telegram. Find everything with natural language.

What is Recall?
(2 paragraphs -- from PRD Executive Summary + Problem Statement)

Features
- Zero-friction capture (Telegram bot -- no app switching)
- AI-powered transcription, summary, and embedding (Whisper, Llama 3, MiniLM)
- Constellation mind map -- 60 FPS, 500+ nodes
- Hybrid semantic + keyword search
- Spaced repetition quizzes (SM-2 algorithm)
- Google Drive backup
- Chrome extension

Architecture
(ASCII diagram showing: Telegram -> Render FastAPI -> Neon + Upstash + Modal)

Tech Stack
(table matching TRD.md)

Quick Start
See [DEPLOYMENT.md](docs/DEPLOYMENT.md) for full setup.

Documentation
(table linking all 18 docs files)

Security
See [SECURITY.md](docs/SECURITY.md). Data encrypted at rest with Fernet AES-128.

License
MIT
```

Rules:

- No placeholder [screenshot] -- generate an actual screenshot by running the app locally OR use a descriptive code block showing sample bot interaction.
- All doc links must be relative (docs/DEPLOYMENT.md not absolute URL).
- No badges that won't stay green (don't add CI badge unless CI is actually set up).
- README must render correctly on GitHub (test with markdown preview).

Gate Check:

- [ ] README.md renders without broken markdown
- [ ] All 18 docs links are valid relative paths
- [ ] Feature list matches actual implemented features
- [ ] Architecture diagram is accurate (matches final tech stack)
- [ ] No placeholder content remaining

## PHASE 5: CANVAS INTERACTIVE MIND MAP

### PROMPT 043 -- Modal Whisper Endpoint (Tier 0 STT)

**SKILLS:** [python-pro](#) . [async-python-patterns](#)

Create backend/modal\_apps/modal\_whisper.py -- Modal serverless Whisper large-v3 endpoint.

```
@app.function(gpu="T4", image=modal.Image.debian_slim().pip_install("openai-whisper", "ffmpeg-python"))
@modal.web_endpoint(method="POST")
async def transcribe(audio_bytes: bytes) -> dict:
 model = whisper.load_model("large-v3")
 result = model.transcribe(audio_bytes)
 return {"transcript": result["text"], "language": result["language"]}
```

Create backend/modal\_apps/modal\_llama.py -- Llama 3 8B summarisation:

Input: {"text": str, "task": "summarise"|"quiz"}

Output: {"summary": str} or {"question": str, "options": list[str], "correct\_index": int, "explanation": str}

Create backend/modal\_apps/modal\_embed.py -- MiniLM-L6-v2 embedding:

Input: {"text": str}

Output: {"embedding": list[float]} # 384 dimensions

Rules:

- All Modal endpoints must have timeout=30 set (matches AI\_CASCADE.md Tier 0 timeout).
- GPU type: T4 (cheapest Modal GPU; sufficient for Whisper large-v3 and Llama 3 8B).
- Each endpoint is a separate modal.App -- deploy independently.
- Modal endpoints must validate input size: reject audio > 25 MB, text > 8000 tokens.
- Cold start message: logged but NOT surfaced to user -- user already got "Processing..." ACK.

Gate Check:

- [ ] modal deploy backend/modal\_apps/modal\_whisper.py succeeds
- [ ] Test call returns {"transcript": "...", "language": "en"} for a 10 s audio clip
- [ ] modal\_embed.py returns list of exactly 384 floats
- [ ] modal\_llama.py returns valid summary for 200-word input text

### PROMPT 044 -- Partition Manager CLI Script

**SKILLS:** [postgres-best-practices](#) . [python-pro](#)

Create backend/scripts/manage\_partitions.py -- a standalone partition management tool.

## Commands:

```
python manage_partitions.py list
 -> Lists all existing items_* partitions with their date ranges and row counts

python manage_partitions.py create --months 3
 -> Pre-creates partitions for the next N months (default 3)
 -> Idempotent: CREATE TABLE IF NOT EXISTS

python manage_partitions.py status
 -> Shows: current month partition exists? next month? next 2 months?
 -> Warns if any upcoming partition is missing

python manage_partitions.py drop --month 2025-01
 -> DETACHes and DROPs a historical partition (with confirmation prompt)
 -> DETACH first (non-blocking), then DROP (removes data)
```

This script is also called by the partition\_creator APScheduler job.

Partition naming: items\_yYYYYmMM (e.g. items\_y2026m08)

Bounds: FROM '2026-08-01' TO '2026-09-01'

## Rules:

- drop command MUST require explicit --confirm flag to execute (safety).
- CREATE TABLE IF NOT EXISTS makes all creates idempotent.
- Script must work standalone (python manage\_partitions.py ...) not just via FastAPI.
- Logs every action with timestamp -- script output is the audit trail.
- NEVER run drop against a partition that contains data without explicit --force flag.

## Gate Check:

```
[] python manage_partitions.py list shows all partitions with row counts
[] python manage_partitions.py create --months 3 creates 3 future partitions idempotently
[] python manage_partitions.py status shows WARNING if next month missing
[] python manage_partitions.py drop --month 2025-01 without --confirm: aborted
[] Unit test: test_partition_manager.py with mocked DB
```

## PROMPT 045 -- End-to-End Test: Auth Flows

### SKILLS: webapp-testing . testing-patterns

Write E2E tests for all three authentication flows.

Test framework: Playwright (add to frontend/ or run separately as e2e/ directory).

Setup: Launch FastAPI dev server + Vite dev server before tests.

Use a test Telegram bot token (not production).

Test 1 -- Telegram Login Widget (website flow):

1. Navigate to /login
2. Mock Telegram Login Widget callback (since real Telegram auth requires human interaction)  
Directly call GET /auth/telegram?id=...&hash=...&auth\_date=... with valid params
3. Assert: redirect to /dashboard
4. Assert: recall\_session cookie is set (httpOnly -- check via API response)
5. Assert: GET /api/items returns 200

Test 2 -- TWA flow (simulated):

1. Direct POST to /api/items with valid TWA initData in Authorization header
2. Assert: 200 (authenticated)

3. Tamper with hash
4. Assert: 401

Test 3 -- Expired JWT:

1. Set a JWT cookie with exp = 1 minute ago
2. GET /api/items
3. Assert: 401 response
4. Assert: cookie is cleared in response

Test 4 -- Logout:

1. Login (valid JWT)
2. POST /auth/logout
3. GET /api/items
4. Assert: 401

Rules:

- Playwright tests in e2e/ directory -- separate from Vitest unit tests.
- Real HTTP calls to local dev server -- no mocking in E2E tests.
- Test Telegram Login Widget hash must be computed correctly (real HMAC with test bot token).
- E2E tests must not touch production DB.

Gate Check:

- [ ] Test 1: Login Widget flow creates valid session
- [ ] Test 2: Tampered TWA hash returns 401
- [ ] Test 3: Expired JWT returns 401 and clears cookie
- [ ] Test 4: Logout prevents further API access
- [ ] All 4 tests pass in CI with test bot token

## PROMPT 046 -- Telegram Webhook Handler + Idempotency

**SKILLS:** telegram-bot-builder . telegram . async-python-patterns

Create backend/routes/webhook.py implementing POST /webhook.

Flow (exactly as in RATE\_LIMITING.md and ERROR\_HANDLING.md):

1. Parse Telegram update JSON (python-telegram-bot or httpx raw parsing).
2. Extract update\_id and chat\_id.
3. Idempotency check:
  - INSERT INTO processed\_updates (update\_id) ON CONFLICT (update\_id) DO NOTHING
  - If rows\_affected == 0: return 200 immediately (duplicate -- silent discard).
4. Rate limit check (stub for now -- returns True always, implemented in PROMPT 010).
5. Detect content type: text / url / voice / document(pdf) / photo.
6. Push task JSON to Upstash Redis queue (LPUSH recall:tasks).
7. Send immediate Telegram ACK: "Processing your <content\_type>..." via bot.sendMessage.
8. Return 200 to Telegram.

Rules:

- Total time from receive to return 200 must be < 50 ms. Profile with time.perf\_counter().
- Never call any AI service in this handler -- only enqueue.
- update\_id must be stored as VARCHAR not INT (Telegram IDs can exceed int32 on old bots).
- The immediate ACK message must vary by content type (voice, PDF, URL, image, text).
- All exceptions caught: log internally, return 200 to Telegram (prevents retry storms).

Gate Check:

- [ ] POST /webhook returns 200 in < 50 ms (measure with pytest + time mock)
- [ ] Duplicate update\_id POST returns 200 without creating a second task
- [ ] 6 different ACK messages exist (one per content type)
- [ ] Unit test: test\_webhook\_idempotency.py covers all 4 cases from TESTING.md \$1

## PROMPT 047 -- Bot /remind Command + Natural Language Time Parsing

**SKILLS:** telegram-bot-builder . python-pro

Implement /remind command in backend/services/reminder\_service.py.

Format: /remind <time\_expr> <message>

Examples:

```
/remind 2h Check that article
/remind 30m Review the ML notes
/remind tomorrow morning Call about the research paper
/remind 3d Revisit the podcast summary
```

Time parsing:

- "Xm" / "Xmin" -> timedelta(minutes=X)
- "Xh" / "Xhr" -> timedelta(hours=X)
- "Xd" / "Xday" -> timedelta(days=X)
- "tomorrow" -> next day at 09:00 user local time (use users.timezone\_offset)
- "tomorrow morning" -> next day 09:00
- "tomorrow evening" -> next day 19:00
- "next week" -> 7 days from now at 09:00
- Invalid format -> "Sorry, I didn't understand that time. Try: /remind 2h Read those notes"

After parsing:

```
remind_at = now + timedelta (adjusted for user timezone_offset)
INSERT INTO reminders (user_id, message, remind_at, status='pending')
Bot reply: " Reminder set for {remind_at.strftime('%d %b %Y at %H:%M')} [x]"
```

POST /api/reminders (REST endpoint for website):

```
Body: {"message": str, "remind_at": ISO datetime str}
Validate: remind_at is in the future (reject past datetimes with 400).
Insert into reminders.
```

Rules:

- remind\_at must be stored in UTC -- convert from user's timezone\_offset before INSERT.
- Max message length: 500 characters -- truncate with warning if longer.
- Max active reminders per user: 20 -- return error if exceeded.
- No regex parsing -- use a small dedicated parse\_time\_expression() function.
- Unit test: test\_reminder\_service.py covers all time formats and invalid inputs.

Gate Check:

```
[] /remind 2h Test message creates a reminder 2 hours from now
[] /remind tomorrow creates reminder at next day 09:00 UTC+timezone_offset
[] Past remind_at rejected with helpful message
[] Max 20 reminders enforced
[] Unit test: all time formats tested including invalid inputs
```

## PHASE 6: WEB AUTHENTICATION & WEBSOCKETS

## PROMPT 048 -- Performance Profiling: Vector Search Benchmarks

**SKILLS:** performance-profiling . postgres-best-practices . k6-load-testing

Benchmark and verify all performance targets from PERFORMANCE.md.

Test 1 -- Vector search < 10 ms:

Insert 1000 items with real 384-dim embeddings (use MiniLM locally).  
Run EXPLAIN ANALYZE on vector search query 100 times.  
Assert: median execution time < 10 ms.  
Assert: EXPLAIN shows "Index Scan using idx\_items\_embedding".

Test 2 -- GIN trigram search < 5 ms:

Same dataset.  
Run EXPLAIN ANALYZE on GIN search 100 times.  
Assert: median < 5 ms.  
Assert: EXPLAIN shows "Bitmap Index Scan on idx\_items\_text\_gin".

Test 3 -- Webhook ACK < 50 ms:

Use k6 with single VU, measure time from request to 200 response.  
Mock all DB and Redis calls to be < 2 ms.  
Assert: p95 < 50 ms.

Test 4 -- Graph API < 200 ms:

200 items with pre-computed hubs.  
GET /api/graph timed 50 times.  
Assert: p95 < 200 ms.

Test 5 -- Canvas 60 FPS:

Use Vitest + fake requestAnimationFrame.  
Simulate 500 nodes, 60 ticks.  
Assert: each tick completes in < 16.67 ms (1000/60).

Create: backend/tests/performance/test\_benchmarks.py and frontend/src/canvas/\_\_tests\_\_/benchmark.test.js

Rules:

- Benchmarks run with EXPLAIN ANALYZE -- never on production.
- Vector search benchmark must use real HNSW index (not mock).
- Canvas benchmark: disable all async (no WS, no API calls) -- pure render loop timing.
- Results saved to docs/PERFORMANCE\_BENCHMARKS.md with date and results.

Gate Check:

- [ ] Vector search: median < 10 ms confirmed with real HNSW index
- [ ] GIN search: median < 5 ms confirmed
- [ ] Webhook ACK: p95 < 50 ms in k6 test
- [ ] Canvas: 500 nodes render at >= 60 FPS in benchmark
- [ ] Results documented in PERFORMANCE\_BENCHMARKS.md

## PROMPT 049 -- Fernet Encryption Service

**SKILLS:** security . privacy-by-design . python-pro

Create backend/services/encryption.py -- the single encryption/decryption utility for all sensitive data.

Functions:

```
def encrypt(plaintext: str) -> str:
 f = Fernet(settings.FERNET_KEY.encode())
 return f.encrypt(plaintext.encode()).decode()

def decrypt(ciphertext: str) -> str:
```

```
f = Fernet(settings.FERNET_KEY.encode())
return f.decrypt(ciphertext.encode()).decode()

def encrypt_if_not_none(value: Optional[str]) -> Optional[str]:
 return encrypt(value) if value is not None else None
```

Rules (CRITICAL -- security violations):

- FERNET\_KEY must come from settings only -- never hardcoded, never passed as parameter.
- The encrypt() function must NEVER log its input or output.
- The decrypt() function must NEVER log its output.
- Every call site that writes raw\_text or google\_refresh\_token to DB must go through encrypt().
- Add a unit test that encrypts "hello" and verifies decrypt(encrypt("hello")) == "hello".
- Add a test that verifies two encrypt("hello") calls produce DIFFERENT ciphertexts (Fernet uses random IV).
- Fernet key rotation: document the procedure in a docstring (matches SECURITY.md \$Key Rotation).

Gate Check:

- [ ] encrypt -> decrypt round-trip works correctly
- [ ] Two encryptions of same plaintext produce different ciphertexts
- [ ] FERNET\_KEY never appears in any log output (test with caplog fixture)
- [ ] Unit test file: test\_encryption.py with all above cases

## PROMPT 050 -- Chrome Extension: Background Service Worker

### SKILLS: chrome-extension-developer

Create frontend/extension/service\_worker.js -- Manifest V3 background service worker.

#### Responsibilities:

- Context menu integration:
 

```
chrome.contextMenus.create({
 id: "recall-save-link",
 title: "Save to Recall",
 contexts: ["link", "page", "selection"]
})
```

On click:

  - "link": save the clicked link URL
  - "page": save the current page URL
  - "selection": save the selected text as a text item
- Badge update:
 

After successful save: `chrome.action.setBadgeText({text: "[x]"})`

Clear after 3 s: `chrome.action.setBadgeText({text: ""})`
- Auth token management:
 

Store JWT in `chrome.storage.local` (encrypted with a derived key from extension ID).

On token expiry: clear storage, reset popup to show login state.
- Notification (if permitted):
 

```
chrome.notifications.create: "Saved to Recall: {page title}"
```

#### Extension API endpoint (backend -- NEW):

```
POST /api/extension/save
Body: {"url": str | null, "text": str | null, "title": str | null}
Same auth as /api/* (JWT Bearer in Authorization header, not cookie -- extensions can't use cookies easily)
Creates items row with source_type='url' or 'text'
```

#### Rules:

- Service worker must handle `chrome.runtime.onInstalled` to initialise context menu.
- No persistent background page -- Manifest V3 service workers are event-driven.
- JWT stored in `chrome.storage.local` -- NOT `chrome.storage.sync` (sync could leak to other devices).
- POST `/api/extension/save` must accept Bearer token in Authorization header (not just cookie).

Gate Check:

- [ ] Right-click on any link shows "Save to Recall" menu item
- [ ] Clicking it saves the link to Recall (items row created)
- [ ] Badge shows [x] for 3 s after successful save
- [ ] JWT from popup is used by service worker for API calls
- [ ] POST `/api/extension/save` works with Bearer token in header

## PROMPT 051 -- Chrome Extension: Options Page

**SKILLS:** `chrome-extension-developer` . `high-end-visual-design`

Create `frontend/extension/options.html` + `options.js` -- extension settings page.

Options page design:

- Matches Cosmic Noir theme (same CSS variables)
- Width: 600px, centered

Settings available:

- Account status:
  - Shows: Logged in as `@{telegram_username}` (fetched from GET `/api/me` with stored JWT)
  - Button: [Logout] -> clears `chrome.storage.local` + resets state
- Save shortcut:
  - Enable keyboard shortcut: `Ctrl+Shift+S` (`Alt+Shift+S` on Mac)
  - Registered via `chrome.commands` in `manifest.json`:

```
"save-current-page": {
 "suggested_key": {"default": "Ctrl+Shift+S", "mac": "Alt+Shift+S"},
 "description": "Save current page to Recall"
}
```
- Notifications:
  - Toggle: Show save confirmation notification (default: ON)
  - Stored in `chrome.storage.local`: `{notifications_enabled: bool}`
- API URL:
  - Input: custom API URL (for self-hosted users)
  - Default: the published `VITE_API_URL`
  - Stored in `chrome.storage.local`

Rules:

- Options page styled with same CSS variables -- no framework.
- `chrome.commands` handler in `service_worker.js` for `Ctrl+Shift+S`.
- Keyboard shortcut triggers same save flow as context menu click.
- Clearing storage on logout must clear: `jwt`, `notifications_enabled`, `api_url` (reset to defaults).

Gate Check:

- [ ] Options page opens via right-click extension icon -> Options
- [ ] `Ctrl+Shift+S` on any page saves current URL to Recall
- [ ] Logout clears JWT and shows "Not logged in" on popup
- [ ] Custom API URL stored and used by service worker
- [ ] Notifications toggle works (notification shown/hidden based on setting)



## PROMPT 052 -- Frontend Test Suite: Vitest

**SKILLS:** javascript-testing-patterns . react-best-practices

Write Vitest tests for frontend critical components.

Test files:

- src/canvas/\_\_tests\_\_/GraphCanvas.test.jsx
  - > Renders without crash (smoke test)
  - > Calls onNodeClick when node is clicked
  - > requestAnimationFrame loop starts on mount, stops on unmount
- src/components/\_\_tests\_\_/NodePanel.test.jsx
  - > Renders node title and summary
  - > Escape key fires onClose
  - > Renders correct @phosphor-icons icon per source\_type
  - > Does not render when node prop is null
- src/hooks/\_\_tests\_\_/useGraphSocket.test.js
  - > Connects to WebSocket on mount
  - > Appends new\_node event to nodes state
  - > Reconnects after disconnect (3 s delay)
  - > Closes WebSocket on unmount
- src/pages/\_\_tests\_\_/Dashboard.test.jsx
  - > Search input debounces 300 ms before firing API call
  - > Non-matching nodes get opacity 0.1 after search

Rules:

- Use @testing-library/react for component tests.
- Mock WebSocket with jest-websocket-mock.
- Mock axios with vi.mock('axios').
- No real network calls -- all API responses must be mocked.

Gate Check:

- [ ] All Vitest tests pass
- [ ] NodePanel tests confirm no emoji icons used for source types
- [ ] useGraphSocket reconnect test passes with fake timers
- [ ] Dashboard search debounce test verifies 300 ms delay

## PROMPT 053 -- AI Cascade Service

**SKILLS:** error-handling-patterns . async-python-patterns . python-pro

Create backend/services/ai\_cascade.py -- the 5-tier fallback orchestrator.

```
class AICascade:
 async def transcribe(self, audio_bytes: bytes, chat_id: str) -> CascadeResult
 async def summarise(self, text: str, chat_id: str) -> CascadeResult
 async def embed(self, text: str) -> list[float]
```

Each method tries tiers in order (from AI\_CASCADE.md):

- Tier 0: Modal endpoint (timeout=30 s)
- Tier 1: Groq API (timeout=20 s)
- Tier 2: Gemini 3.1 Flash-Lite (timeout=20 s)
- Tier 3: Ollama (timeout=60 s, skip if OLLAMA\_HOST not set)
- Tier 4: Return CascadeResult(success=False, tier\_reached=4)

```
@dataclass
class CascadeResult:
 success: bool
 tier_used: int
 transcript: Optional[str] = None
 summary: Optional[str] = None
 quiz_data: Optional[dict] = None
```

Rules:

- Each tier MUST be wrapped in individual try/except -- failure of Tier N must NOT prevent Tier N+1.
- Catch specific exceptions: httpx.TimeoutException, httpx.HTTPStatusError -- not bare except.
- Log which tier was used for every successful result (for monitoring).
- COMPUTE\_PROVIDER env var overrides tier selection (for testing: groq, gemini, ollama, modal).
- Tier 4 exhaustion must write to dead\_letter\_queue BEFORE sending user notification.
- NEVER log the transcript or summary content -- only log tier number and success/failure.

Gate Check:

```
[] Mock Tier 0 failure -> Tier 1 is called (verified via mock call count)
[] Mock Tiers 0-3 failure -> dead_letter_queue entry created
[] COMPUTE_PROVIDER=groq skips Tier 0 entirely
[] Unit test covers all 5 cases from TESTING.md §2
[] No transcript content appears in test logs (caplog assertion)
```

## PROMPT 054 -- Backend Test Suite: Full Coverage

**SKILLS:** [python-testing-patterns](#) . [testing-patterns](#) . [unit-testing-test-generate](#)

Write the complete pytest test suite for all critical paths identified in TESTING.md.

Test files to create:

- tests/test\_idempotency.py -> all 4 cases from TESTING.md §1
- tests/test\_cascade.py -> all 5 cases from TESTING.md §2
- tests/test\_rate\_limiter.py -> all 5 cases from TESTING.md §3
- tests/test\_sm2.py -> all 5 cases from TESTING.md §4
- tests/test\_auth\_twa.py -> all 4 cases from TESTING.md §5 TWA section
- tests/test\_auth\_jwt.py -> all 5 cases from TESTING.md §5 JWT section
- tests/test\_auth\_oauth.py -> all 3 cases from TESTING.md §5 OAuth section
- tests/test\_partitioning.py -> all 5 cases from TESTING.md §6

Test configuration rules (MANDATORY):

- All AI calls: mocked via pytest-mock (no real API calls ever)
- All Telegram API calls: mocked
- Redis: use fakeredis library
- Database: use pytest-postgresql or psycopg in-memory test DB
- Time: use freezegun for time-dependent tests (rate limiter, auth\_date expiry, SM-2 dates)

Coverage requirements:

- src/services/encryption.py: 100% coverage
- src/services/sm2.py: 100% coverage
- src/routes/auth.py: >= 90% coverage
- src/services/rate\_limiter.py: >= 90% coverage

Rules:

- Tests must run in < 30 s total (fast feedback).
- No test may make network calls (add pytest-httpretty or respx to block).
- Test names must describe the scenario: test\_duplicate\_update\_id\_returns\_200\_without\_processing.
- Every security-critical function must have a test for the attack path (not just happy path).

Gate Check:

```
[] pytest runs all tests in < 30 s
[] Zero network calls made during test run (verified with respx)
[] test_sm2.py: all 5 cases pass with exact numeric values from TESTING.md
[] test_auth_twa.py: tampered hash returns 401
[] Coverage report shows encryption.py at 100%
```

## PROMPT 055 -- React + Vite Project Setup

**SKILLS:** [react-best-practices](#) . [ui-ux-pro-max](#) . [react-patterns](#)

Scaffold the React + Vite frontend in frontend/.

```
npx create-vite@latest . --template react
npm install
```

Dependencies to install:

- d3 (force simulation)
- @phosphor-icons/react (icon library -- from UI\_UX\_BRIEF.md rules)
- axios (HTTP client)

Create frontend/src/theme.css implementing the full Cosmic Noir design system from IMPLEMENTATION\_DESIGN\_TOKENS.md:

- CSS custom properties for all color tokens (--bg-deep, --bg-base, --surface-glass, etc.)
- Outfit + Inter + JetBrains Mono Google Fonts import
- .nebula-blob, .nebula-violet, .nebula-mint animated blob classes
- .glass-card with backdrop-filter: blur(24px)
- .glass-glow-top pseudo-element
- @keyframes floatNebula and pulseGlow

Rules:

- NO Tailwind unless explicitly requested -- Vanilla CSS with design tokens only (UI\_UX\_BRIEF.md principle).
- Font imports via Google Fonts -- no self-hosted fonts.
- body background-color: var(--bg-deep) (#030307).
- All CSS colours must use CSS variables -- no hardcoded hex in component files.
- @media (prefers-reduced-motion: reduce) must disable all animations.

Gate Check:

```
[] npm run dev serves the app on localhost:5173
[] theme.css custom properties are accessible in browser DevTools
[] Both animated nebula blobs visible as subtle background movement
[] prefers-reduced-motion disables floatNebula animation
[] Vitest runs with 0 errors
```

## PROMPT 056 -- Security Scanning: SAST + Dependency Audit

**SKILLS:** [security-scanning-security-sast](#) . [security-scanning-security-hardening](#) . [security](#)

Run automated static analysis and produce a security scan report.

Backend SAST (bandit):

```
bandit -r backend/ -ll -f json -o security_reports/bandit_report.json
Review all HIGH and MEDIUM findings.
Fix ALL HIGH severity issues before deployment.
```

Document any accepted MEDIUM risks in security\_reports/ACCEPTED\_RISKS.md.

Dependency vulnerability scan:

```
pip-audit --require-hashes -r backend/requirements.txt -o security_reports/pip_audit.json
npm audit --json > security_reports/npm_audit.json
Fix ALL critical and high severity dependency vulnerabilities.
```

Secrets scan:

```
Use trufflesec or simple grep:
grep -rE "(AAAB[A-Za-z0-9+/{32,}|ghp_[A-Za-z0-9]{36}|gAAAA[A-Za-z0-9+/{60,})" backend/ frontend/
Any match is a BLOCKER.
```

Hardening checklist:

```
[] TLS: no HTTP allowed in any URL construction (grep for "http://" in backend/ -- only localhost allowed)
[] No subprocess.shell=True in any code
[] No eval() or exec() calls
[] No pickle.loads() on untrusted data
[] All file operations: validate path is within expected directory (prevent path traversal)
```

Output: Create docs/SECURITY\_SCAN\_REPORT.md with:

- Bandit summary (HIGH/MEDIUM/LOW counts)
- pip-audit summary
- npm audit summary
- Any accepted risks with justification

Rules:

- BLOCKER issues must all be fixed before this prompt is marked complete.
- Accepted risks must have documented rationale -- not just "we'll fix later".
- Security scan runs in CI (from PROMPT 081) -- must pass cleanly.
- Report must be dated and versioned.

Gate Check:

```
[] bandit: 0 HIGH findings
[] pip-audit: 0 CRITICAL or HIGH vulnerability findings
[] npm audit: 0 CRITICAL or HIGH findings
[] Secrets scan: 0 matches
[] docs/SECURITY_SCAN_REPORT.md written with all counts
```

## PROMPT 057 -- Dashboard Layout + Header

**SKILLS:** react-best-practices . senior-frontend . ui-ux-pro-max

Create frontend/src/pages/Dashboard.jsx -- the main app shell.

Layout (from UI\_UX\_BRIEF.md):

- Full viewport, overflow hidden
- Background: 2 nebula blob divs (absolute positioned, animation via theme.css)
- Floating header (56px, glass-card): Logo | Search bar | Quiz badge | Profile icon
- GraphCanvas fills remaining viewport (below header)
- NodePanel overlays on right when node selected
- Canvas is the primary view -- minimal UI chrome

Search bar:

- Debounced 300 ms input
- On submit: POST /api/search -> highlight matching nodes (dim non-matches to 10% opacity)
- Clear search: restore all nodes to full opacity
- Use @phosphor-icons/react MagnifyingGlass icon

Header profile icon:

- Shows Telegram username initials
- Clicking opens dropdown: [Connect Google Drive] [Logout]

Rules:

- Header must be position: fixed -- never scroll with canvas.
- Nebula blobs must be z-index: 0; GraphCanvas z-index: 1; Header z-index: 10; NodePanel z-index: 20.
- Search debounce: 300 ms exactly (matches PRD success metrics).
- NO emoji icons anywhere in the header.

Gate Check:

- [ ] Dashboard renders with header, canvas, and nebula background
- [ ] Search input triggers POST /api/search after 300 ms debounce
- [ ] Non-matching nodes dim to 10% opacity during search
- [ ] z-index layering correct (nebula behind canvas, panel above canvas)
- [ ] Vitest: Dashboard renders without crashing

## PHASE 7: SPACED REPETITION QUIZZES

### PROMPT 058 -- Voice Note Ingestion

**SKILLS:** telegram-bot-builder . python-pro . async-python-patterns

Implement backend/services/ingestion/voice\_ingester.py.

Pipeline:

1. Download voice file from Telegram using bot.getFile(file\_id) + httpx download.
2. Save temporarily to /tmp/<uuid>.ogg.
3. Call AICascade.transcribe(audio\_bytes, chat\_id).
4. If success: call AICascade.summarise(transcript).
5. Call AICascade.embed(transcript).
6. Fernet-encrypt transcript -> raw\_text.
7. INSERT into items:  
source\_type='voice', raw\_text=<encrypted>, summary=<summary>, embedding=<vector>, title=<first 100 chars of transcript>
8. INSERT into quizzes (if quiz\_data returned from Llama).
9. Bot reply:  
" Transcribed:\n{first 200 chars}...\n\n Summary:\n{summary}\n\n Saved [x]"
10. Delete /tmp file.

Rules:

- Temporary audio file must be deleted in a finally block -- never leave files on disk.
- File size limit: reject files > 25 MB before downloading.
- If transcription fails (all tiers): save as bookmark, send error message per ERROR\_HANDLING.md.
- embedding column requires pgvector format: INSERT ... embedding = \$1::vector
- quiz INSERT: options stored as JSONB array, correct\_index as INT 0-based.

Gate Check:

- [ ] Voice note forwarded -> items row with encrypted raw\_text and 384-dim embedding
- [ ] /tmp file deleted after processing (even on exception)
- [ ] Cascade failure -> dead\_letter\_queue entry + "Saved as bookmark" bot message
- [ ] Unit test: test\_voice\_ingester.py with mocked Telegram download and mocked AICascade

## PROMPT 059 -- Google OAuth Flow

**SKILLS:** [auth-implementation-patterns](#) . [security](#) . [python-fastapi-development](#)

Implement GET /auth/google and GET /auth/google/callback in backend/routes/auth.py.

GET /auth/google:

1. Generate state = JWT {chat\_id: from query or session, exp: now + 10 min}
2. Build Google OAuth URL with:  
scope = "https://www.googleapis.com/auth/drive.file" (ONLY this scope -- non-negotiable)  
redirect\_uri = GOOGLE\_REDIRECT\_URI  
state = state\_token  
access\_type = "offline"  
prompt = "consent"
3. Return redirect to Google.

GET /auth/google/callback:

1. Validate state JWT: hmac.compare\_digest AND exp check.
2. Extract chat\_id from state JWT.
3. Exchange code for tokens via Google token endpoint.
4. Fernet-encrypt refresh\_token.
5. UPDATE users SET google\_refresh\_token = <encrypted> WHERE telegram\_chat\_id = \$1
6. Broadcast WS event: {"type": "google\_connected"} to the user's WS connection.
7. Send Telegram bot message: "[OK] Google Drive connected! Your knowledge will be backed up daily."
8. Return redirect to WEBSITE\_URL/dashboard.

Rules (CRITICAL -- security):

- scope MUST be drive.file ONLY -- never request broader Drive scope.
- access\_token must NEVER be stored -- only refresh\_token (Fernet-encrypted).
- State JWT validation: use hmac.compare\_digest -- prevents timing attacks.
- State JWT expiry: 10 minutes -- prevents CSRF replay.
- Refresh token must go through encrypt() before ANY DB write -- zero exceptions.

Gate Check:

- [ ] Connecting Drive stores Fernet-encrypted refresh\_token in users table
- [ ] Decrypting stored refresh\_token with FERNET\_KEY returns valid token
- [ ] state JWT expiry after 10 minutes returns 401
- [ ] Tampered state JWT returns 401
- [ ] WS google\_connected event fires in open browser tab
- [ ] Unit test: all 3 cases from TESTING.md §5 Google OAuth CSRF section

## PROMPT 060 -- Items Feed View (Alternative to Graph)

**SKILLS:** [react-best-practices](#) . [react-ui-patterns](#) . [ui-ux-pro-max](#)

Add a Feed view as an alternative to the constellation graph.

Toggle in header: [ Graph] [ Feed] -- switches the main content area.

Feed view: frontend/src/pages/Feed.jsx

Layout:

- Infinite scroll (Intersection Observer)
- Card grid: 2 columns on desktop, 1 on mobile (< 600px)
- Each card: glass-card style from IMPLEMENTATION\_DESIGN\_TOKENS.md
  - Source type icon (phosphor-icons) + badge colour:  
url=violet, voice=mint, pdf=amber, image=sky, text=white
  - Title (Outfit, 500 weight)

- Summary excerpt (2 lines max, text-overflow: ellipsis)
- Tags (pill badges, --color-primary)
- Relative timestamp (JetBrains Mono, "2 hours ago")
- [\*\*\*] menu: View in Graph, Delete

Filter bar (above grid):

- All | Links | Voice | PDFs | Images | Text
- + Date range picker
- + Search by tag (typeahead from GET /api/tags)

Clicking a card opens the NodePanel from PROMPT 086 (same component -- reuse).

Rules:

- Infinite scroll: fetch 20 items per page via GET /api/items?page=N.
- Intersection Observer fires when last card reaches viewport -- not scroll position.
- Cards must render without layout shift (use CSS aspect-ratio or min-height).
- Feed view must share auth context with Graph view (no re-login on toggle).
- @phosphor-icons for all source type icons -- no emojis.

Gate Check:

- [ ] Feed renders with glass-card design
- [ ] Filter by source\_type updates items list
- [ ] Infinite scroll loads next page when reaching bottom
- [ ] Clicking card opens same NodePanel as graph view
- [ ] Vitest: Feed renders with mock API response

## PHASE 8: SCHEDULER & CRON JOBS

### PROMPT 061 -- PDF Ingestion

**SKILLS:** python-pro . async-python-patterns

Implement backend/services/ingestion/pdf\_ingester.py.

Pipeline:

1. Download PDF from Telegram (document message where mime\_type='application/pdf').
2. Extract text with PyMuPDF (fitz): page by page.
3. Chunk text into segments of max 512 tokens (split by sentence, not character).
4. For each chunk: call AICascade.embed(chunk) -> store as separate embedding.  
(For v1: store only the first chunk's embedding in items.embedding -- full chunking in Phase 3 upgrade.)
5. Call AICascade.summarise(full\_text[:4000]) for document-level summary.
6. Fernet-encrypt full text -> raw\_text.
7. INSERT into items: source\_type='pdf', title=<filename or first line>, raw\_text=<encrypted>, summary=<summary>, embedding=<first chunk embedding>
8. Bot reply: " {filename}\n\n{summary}\n\nPages: {page\_count} | Saved [x]"

Rules:

- PDF size limit: 20 MB. Reject larger files before download.
- If PyMuPDF extracts 0 text (scanned PDF): fall back to OCR using Tesseract on each page image.
- Temporary files: /tmp/<uuid>.pdf -- always delete in finally block.
- Page count must be included in bot reply.
- Empty PDFs (0 pages) -> error message, no items row created.

Gate Check:

- [ ] PDF forwarded -> items row with summary and embedding
- [ ] Temp file deleted after processing
- [ ] 0-text PDF triggers Tesseract fallback
- [ ] Bot reply includes page count
- [ ] Unit test: test\_pdf\_ingester.py with mocked PyMuPDF and AICascade

## PROMPT 062 -- Batch Items Export (GDPR / Data Portability)

**SKILLS:** python-fastapi-development . privacy-by-design

Implement a data export endpoint for GDPR compliance and user data portability.

GET /api/export -- authenticated:

Response: JSON file download with all user data.

Content-Type: application/json

Content-Disposition: attachment; filename="recall-export-{date}.json"

Export structure:

```
{
 "export_date": str,
 "user": {
 "telegram_chat_id": str,
 "streak_count": int,
 "timezone_offset": int,
 "created_at": str
 },
 "items": [
 {
 "id": int,
 "source_type": str,
 "source_url": str | null,
 "raw_text_decrypted": str | null, // decrypted for export
 "summary": str | null,
 "title": str | null,
 "tags": list[str],
 "created_at": str
 }
],
 "reminders": [...],
 "quizzes": [...]
}
```

Rules:

- raw\_text\_decrypted: decrypt ALL items' raw\_text before export -- user deserves their own plaintext.
- google\_refresh\_token: NEVER include in export (third-party credential).
- Export endpoint must be rate limited: 1 export per user per 24 hours.
- Stream the response (StreamingResponse) -- do not build entire JSON in memory for large datasets.
- Log export event: {user\_id, export\_date, item\_count} for audit trail.

Gate Check:

- [ ] GET /api/export downloads valid JSON file
- [ ] raw\_text fields are decrypted in export
- [ ] google\_refresh\_token absent from export
- [ ] Second export within 24 hours returns 429
- [ ] StreamingResponse: does not load all items into memory simultaneously
- [ ] Unit test: test\_export.py verifies decryption and token absence



## PROMPT 063 -- API Rate Limit for Web Endpoints

**SKILLS:** python-fastapi-development . async-python-patterns

Extend the rate limiter to cover web API endpoints (not just /webhook).

Different limits per endpoint type:

```
POST /api/search: 60 requests / user / minute
GET /api/items: 120 requests / user / minute
GET /api/graph: 30 requests / user / minute
POST /api/quizzes/{id}/answer: 120 requests / user / minute
POST /api/drive/sync: 5 requests / user / hour (Drive sync is expensive)
```

Implementation:

```
Refactor rate_limiter.py to accept: key_prefix, limit, window_seconds
async def check_rate_limit(user_id: int, key_prefix: str, limit: int, window_seconds: int)
```

Create FastAPI dependency factory:

```
def rate_limit(prefix: str, limit: int, window: int = 60):
 async def _dependency(user: UserContext = Depends(get_current_user)):
 await check_rate_limit(user.user_id, prefix, limit, window)
 return _dependency
```

Apply to routes:

```
@router.post("/api/search", dependencies=[Depends(rate_limit("search", 60))])
```

On rate limit exceeded: return HTTP 429 with Retry-After header.

Unlike webhook (returns 200), web API returns proper 429 to browser.

Rules:

- Rate limit keys: "rate:{prefix}:{user\_id}" -- always scoped to user.
- /auth/\* routes are EXEMPT -- rate limiting auth causes lockout issues.
- /health is EXEMPT.
- 429 response must include: {"error": "rate\_limit\_exceeded", "retry\_after": int}.
- Unit test: test\_api\_rate\_limit.py verifies limits for search and drive/sync.

Gate Check:

- [ ] POST /api/search: 61st request in 60 s returns 429
- [ ] POST /api/drive/sync: 6th request in 1 hour returns 429
- [ ] /auth/telegram: unlimited (no 429)
- [ ] Retry-After header present on 429 responses
- [ ] Unit test covers all endpoint-specific limits

## PROMPT 064 -- Load Testing with k6

**SKILLS:** k6-load-testing

Create backend/tests/load/k6\_webhook.js -- load test for the webhook endpoint.

Scenario:

- 50 virtual users (simulating 50 DAU sending content concurrently)
- Test duration: 60 seconds
- Each VU: POST /webhook every 3 s with a text message payload

Checks:

- Response time p95 < 200 ms

- HTTP status == 200 for all requests
- Zero 5xx responses

Threshold config:

```
thresholds: {
 http_req_duration: ['p(95)<200'],
 http_req_failed: ['rate<0.01'],
}
```

Also create k6\_search.js:

- 20 VUs, 30 s duration
- POST /api/search every 2 s with random query
- p95 < 500 ms (vector + trigram search combined)

Rules:

- Load tests must target a staging environment, not production (comment clearly in script).
- k6 script must NOT hardcode any real credentials -- use k6 environment variables.
- Webhook load test must NOT use a real bot token -- use a mock payload.
- Results must be captured: k6 run --out json=results.json k6\_webhook.js

Gate Check:

```
[] k6 run k6_webhook.js completes with p95 < 200 ms under 50 VU load
[] k6 run k6_search.js completes with p95 < 500 ms
[] Zero 5xx responses in both tests
[] Thresholds pass (k6 exits with code 0)
```

## PROMPT 065 -- Integration Test: Full Item Save Flow

**SKILLS:** [python-testing-patterns](#) . [testing-patterns](#) . [webapp-testing](#)

Write an end-to-end integration test covering the complete item save pipeline.

Test file: backend/tests/integration/test\_full\_save\_flow.py

Use a real (Neon test branch) or local PostgreSQL for this test.

Mock ONLY: Telegram API, Modal GPU, Groq API, Gemini API.

Scenario 1 -- URL save:

1. POST /webhook with a text message containing a URL
2. Assert: processed\_updates row created
3. Assert: items row created with correct source\_type='url'
4. Assert: items.raw\_text starts with 'gAAAA' (Fernet prefix)
5. Assert: WS event received (new\_node)
6. Assert: Telegram mock called with ACK message

Scenario 2 -- Voice note save (cascade Tier 0 -> Tier 1 fallback):

1. POST /webhook with voice file\_id
2. Mock Modal to fail (500)
3. Mock Groq to succeed with transcript
4. Assert: items row created with source\_type='voice' and embedding
5. Assert: dead\_letter\_queue has 0 rows (cascade succeeded before Tier 4)
6. Assert: quiz row created for the item

Scenario 3 -- Full cascade exhaustion:

1. POST /webhook with PDF
2. Mock ALL tiers to fail
3. Assert: dead\_letter\_queue row created
4. Assert: items row created with source\_type='pdf', raw\_text=NULL

5. Assert: Telegram mock called with "Saved as bookmark" message

#### Rules:

- Integration tests in a separate tests/integration/ directory.
- Require DATABASE\_URL env var pointing to a test DB (refuse if it contains "prod").
- Clean up all created rows after each test (use pytest fixtures with teardown).
- Entire integration test suite must complete in < 60 s.

#### Gate Check:

- [ ] All 3 scenarios pass
- [ ] Database is clean after each test (no leaked rows)
- [ ] Tests refuse to run if DATABASE\_URL contains "prod"
- [ ] Suite completes in < 60 s
- [ ] CI workflow runs integration tests on a Neon CI branch

## PROMPT 066 -- Toast Notification System

### SKILLS: react-ui-patterns . high-end-visual-design

Create frontend/src/components/Toast.jsx -- a global toast notification system.

Design: floating toasts, top-right corner, auto-dismiss after 4 s.

Types: success (mint green), error (red), info (violet), warning (amber).

#### Implementation:

- Context: ToastContext with addToast(message, type) and removeToast(id).
- ToastContainer: renders active toasts, positioned fixed top-right.
- Each toast: glass-card, 320px wide, slide-in from right over 300 ms, fade-out over 200 ms.
- Stack up to 3 toasts; oldest dismissed first if new one arrives at 3.
- Icon per type from @phosphor-icons (CheckCircle, XCircle, Info, Warning).

#### Toast triggers in the app:

- New item saved (WebSocket new\_node event): "[x] Saved [source\_type]!"
- Search with 0 results: "No results found for '{query}'"
- Drive connected (WS google\_connected event): "Google Drive connected!"
- Delete item success: "Item deleted"
- Network error: "Connection error -- retrying..."

#### Accessibility:

- role="alert" aria-live="polite" on each toast.
- Toasts must not block interaction with canvas or panels.

#### Rules:

- No toast library (react-hot-toast, Sonner, etc.) -- build from scratch.
- Auto-dismiss: use setTimeout, clear on unmount.
- z-index: 100 (above panel z:20, above header z:10).
- @media (prefers-reduced-motion: reduce) -> instant show/hide, no slide animation.

#### Gate Check:

- [ ] Saving item via bot triggers toast in open browser tab within 2 s
- [ ] 3 simultaneous toasts stack correctly
- [ ] 4th toast auto-dismisses oldest first
- [ ] Toasts have role="alert" (screen reader accessible)
- [ ] Vitest: addToast renders toast, auto-dismiss fires after 4 s

## PROMPT 067 -- Embedding Pipeline Integration

**SKILLS:** python-pro . postgres-best-practices

Integrate MiniLM-L6-v2 embeddings into all ingestion paths.

Create backend/services/embedder.py:

```
async def embed_text(text: str) -> list[float]:
 Returns 384-dimensional embedding via AICascade.embed()
```

Update all ingesters (url, text, voice, pdf, image) to:

1. Call embed\_text(summary or raw\_text[:2000]) after AI processing.
2. Store result in items.embedding as pgvector VECTOR(384).
3. INSERT syntax: embedding = \$1::vector where \$1 is the list serialised as "[0.1,0.2,...]".

Rules:

- Never store NULL embedding if text is available -- embedding is required for search.
- If embed\_text fails (cascade exhaustion): store NULL embedding, save item anyway (search degrades gracefully).
- Embedding input must be the summary (not encrypted raw\_text) -- summary is plaintext.
- Validate: len(embedding) == 384 before INSERT. Raise ValueError otherwise.

Gate Check:

- [ ] items row after text ingestion has non-NULL 384-dim embedding
- [ ] EXPLAIN ANALYZE on vector search query shows Index Scan on idx\_items\_embedding
- [ ] embed\_text failure still creates items row (with NULL embedding)
- [ ] Unit test: test\_embedder.py validates embedding dimension

## PROMPT 068 -- Reminder UI on Website

**SKILLS:** react-ui-patterns . react-best-practices

Add reminder management to the web dashboard.

Reminder creation from NodePanel (PROMPT 086):

```
[Set Reminder] button -> opens a date-time picker inline within the panel
Date picker: native <input type="datetime-local"> styled to match Cosmic Noir theme
Text field: pre-filled with item title (editable)
[Confirm] -> POST /api/reminders
Success -> toast "Reminder set for {date}"
```

Reminders list page (frontend/src/pages/Reminders.jsx):

```
Accessible from header dropdown or /reminders route
List of pending reminders in chronological order
Each row: datetime . message text . [Delete] button -> DELETE /api/reminders/{id}
Filter: Pending | Sent | All
```

DELETE /api/reminders/{id} (backend):

```
DELETE FROM reminders WHERE id = $1 AND user_id = $2 RETURNING id
If no row: 404
If row deleted: 204
```

Native datetime picker styling (Cosmic Noir):

```
input[type="datetime-local"]:
 background: var(--surface-glass)
 border: 1px solid rgba(255,255,255,0.08)
 color: var(--text-primary)
 color-scheme: dark (makes native picker use dark theme)
```

## Rules:

- datetime-local value must be converted to UTC before POST /api/reminders.
- DELETE /api/reminders MUST include AND user\_id check (IDOR prevention).
- Maximum 20 reminders per user enforced on GET /api/reminders (show count in UI).
- Sent reminders are read-only -- [Delete] disabled for status='sent'.

## Gate Check:

- [ ] Setting reminder from NodePanel creates DB row via POST /api/reminders
- [ ] Reminders page shows pending reminders in chronological order
- [ ] Deleting a reminder: 204 returned, row removed
- [ ] Another user cannot delete your reminders (IDOR check)
- [ ] color-scheme: dark applied to datetime picker

## PHASE 9: GOOGLE DRIVE & CHROME EXTENSION

### PROMPT 069 -- PDF Chunking + Multi-Chunk Embedding

**SKILLS:** [python-pro](#) . [postgres-best-practices](#)

Upgrade pdf\_ingester.py (from PROMPT 031) to store per-chunk embeddings.

Problem: v1 stored only the first chunk's embedding. Long PDFs lose later content from search.

Solution: chunk-level embedding with aggregated search.

Add item\_chunks table to schema:

```
CREATE TABLE item_chunks (
 id SERIAL PRIMARY KEY,
 item_id INT NOT NULL,
 user_id INT REFERENCES users(id) ON DELETE CASCADE,
 chunk_index INT NOT NULL,
 chunk_text TEXT NOT NULL, -- plaintext (excerpt for search)
 embedding VECTOR(384),
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
CREATE INDEX idx_chunks_item ON item_chunks(item_id);
CREATE INDEX idx_chunks_embedding ON item_chunks USING hnsw (embedding vector_cosine_ops) WITH (m=16,
ef_construction=64);
```

PDF ingestion update:

Chunk PDF text into segments of ~400 tokens (split on sentence boundary).  
For each chunk: embed -> INSERT into item\_chunks.  
items.embedding: set to the embedding of chunk 0 (first chunk) for graph positioning.

Search update:

Hybrid search must also search item\_chunks.embedding:  
SELECT DISTINCT item\_id FROM item\_chunks WHERE user\_id=\$1 ORDER BY embedding <=> \$2 LIMIT 20  
Merge chunk results with item results before RRF.

## Rules:

- chunk\_text: store first 500 chars of the chunk only (for search display) -- not full text.

- item\_chunks uses same HNSW params as items (m=16, ef\_construction=64).
- user\_id on item\_chunks: allows direct user-scoped chunk search without JOIN.
- On item DELETE: CASCADE deletes item\_chunks (add ON DELETE CASCADE via FK or trigger).
- Unit test: test\_pdf\_chunks.py verifies chunk count matches page content.

Gate Check:

- [ ] 10-page PDF creates >= 3 item\_chunks rows
- [ ] Search query matching page 7 content returns the parent item
- [ ] item DELETE removes all associated item\_chunks (CASCADE)
- [ ] HNSW index on item\_chunks confirmed via pg\_indexes
- [ ] Unit test: chunk search tested independently from item search

## PROMPT 070 -- YouTube URL Pipeline

**SKILLS:** python-pro . async-python-patterns

Implement backend/services/ingestion/youtube\_ingester.py.

Detection: URL contains youtube.com/watch oryoutu.be/ or youtube.com/shorts/

Pipeline:

1. Use yt-dlp to extract audio track (not full video):  
yt\_dlp.YoutubeDL({"format": "bestaudio", "postprocessors": [{"key": "FFmpegExtractAudio", "preferredcodec": "mp3"}]})
2. Download audio to /tmp/<uuid>.mp3 (max 50 MB -- reject if larger).
3. Call AICascade.transcribe(audio\_bytes, chat\_id).
4. Call AICascade.summarise(transcript).
5. Call AICascade.embed(summary).
6. Extract video metadata: title, channel, duration via yt-dlp info\_dict.
7. Fernet-encrypt transcript -> raw\_text.
8. INSERT into items:  
source\_type='url', source\_url=youtube\_url, title="[YT] {video\_title}",  
raw\_text=<encrypted>, summary=<summary>, embedding=<vector>
9. Bot reply:  
" {video\_title} ({duration})\n {channel}\n\n {summary[:300]}\n\nSaved [x]"
10. Delete /tmp files.

Instagram/Reel pipeline (in same file):

- Try ZenRows -> ScrapingBee -> ScraperAPI -> yt-dlp direct
- Extract video audio -> same transcription flow
- User-Agent spoofing: rotate from a pool of 10 real browser UA strings
- Cookie rotation: load from a JSON file (cookies.json in backend/ -- gitignored)

Rules:

- yt-dlp output must be captured -- never stream to stdout.
- Audio files: always delete in finally block.
- Max audio duration: 30 minutes -- reject longer videos with user message.
- yt-dlp errors (private video, geo-block): fall back to bookmark with source\_url.
- cookies.json must be in .gitignore -- never commit scraping cookies.

Gate Check:

- [ ] YouTube URL -> transcription + summary in items row
- [ ] Video longer than 30 min -> user message + bookmark fallback
- [ ] Private video -> bookmark fallback (no crash)
- [ ] /tmp files deleted in all paths
- [ ] Unit test: test\_youtube\_ingester.py with mocked yt-dlp

## PROMPT 071 -- Frontend WebSocket Hook + Real-Time Node Addition

**SKILLS:** react-state-management . react-patterns

Create frontend/src/hooks/useGraphSocket.js -- the WebSocket state hook.

```
const { nodes, edges, hubs } = useGraphSocket(token, initialGraph)
```

On mount:

1. Open WebSocket to VITE\_API\_URL/ws/{token}.
2. Listen for events:
  - new\_node: append to nodes state; mark as "pulse" type for 5 minutes.
  - hubs\_updated: replace hubs state; trigger hub node re-render.
  - google\_connected: dispatch to app state (update Drive button).
  - ping: respond with pong (keep-alive).
3. On disconnect: attempt reconnect after 3 s (max 5 retries).

In GraphCanvas.jsx:

- On new\_node event: animate new node appearance (fade in + pulse ring expanding over 1.2 s).
- Pulse nodes (created\_at within last 5 min): draw expanding concentric ripples.
- On hubs\_updated: rebuild hub nodes; broadcast gravitational ripple animation to member nodes.

Rules:

- WebSocket URL must use wss:// (not ws://) in production.
- Reconnect attempts must use exponential backoff: 1s, 2s, 4s, 8s, 16s.
- Do NOT hold the JWT in localStorage -- pass as a prop from the auth context only.
- On component unmount: ws.close() to prevent memory leaks.
- State updates from WS must go through React state (not direct DOM manipulation).

Gate Check:

- [ ] Forwarding content to bot adds new glowing node in browser tab within 2 s
- [ ] Browser tab closed -> WebSocket connection cleaned up server-side
- [ ] Reconnect works after server restart
- [ ] Pulse animation plays for 5 minutes then node becomes orbital
- [ ] Vitest: useGraphSocket hook tests with mocked WebSocket

## PROMPT 072 -- Disconnect Drive Endpoint + Google Token Revocation

**SKILLS:** python-fastapi-development . security

Implement DELETE /api/drive in backend/routes/api.py.

Logic:

1. Auth: get\_current\_user -> user\_id
2. SELECT google\_refresh\_token FROM users WHERE id = \$1
3. If NULL: return 204 (already disconnected)
4. Decrypt refresh\_token
5. Call Google token revoke endpoint:  
POST https://oauth2.googleapis.com/revoke?token={refresh\_token}  
(This revokes the token at Google -- Recall can no longer sync)
6. UPDATE users SET google\_refresh\_token = NULL WHERE id = \$1
7. Return 204

Error handling:

- Google revoke returns 400 (token already revoked): proceed with NULL update regardless.
- Google revoke returns 503: log error, still NULL the token locally (user can re-connect).
- Network error reaching Google: still NULL the token locally.

## Rules:

- Always NULL the token locally EVEN IF Google revoke fails -- local data must be cleared.
- Decrypted refresh\_token must NEVER be logged.
- User must be authenticated -- cannot disconnect another user's Drive.
- After NULL: user must re-do full OAuth to reconnect (no token refresh possible).

## Gate Check:

- [ ] DELETE /api/drive sets google\_refresh\_token to NULL
- [ ] Google revoke is called with decrypted token
- [ ] Google revoke failure (503) still NULLs local token
- [ ] Another user cannot disconnect your Drive (auth check)
- [ ] Unit test: test\_drive\_disconnect.py with mocked Google revoke API

## PROMPT 073 -- SM-2 Algorithm + Quiz Endpoints

### SKILLS: python-pro . python-fastapi-development

Implement the SM-2 spaced repetition algorithm in backend/services/sm2.py.

```
def update_sm2(ease_factor: float, interval_days: int, quality: int) -> tuple[float, int]:
```

SM-2 rules (from TESTING.md §4 exactly):

- quality 0-2 (wrong): interval = 1; ease\_factor = max(1.3, ease\_factor - 0.8)
  - quality 3 (correct, hard): interval unchanged; ease\_factor unchanged
  - quality 4 (correct, ok): interval = round(interval \* ease\_factor); ease\_factor unchanged
  - quality 5 (correct, easy): interval = round(interval \* ease\_factor); ease\_factor += 0.1
- Return (new\_ease\_factor, new\_interval\_days)

GET /api/quizzes/due:

```
SELECT * FROM quizzes
WHERE user_id = $1 AND next_review <= CURRENT_DATE
ORDER BY next_review ASC LIMIT 10
```

POST /api/quizzes/{quiz\_id}/answer:

Body: {"quality": int} (0-5)  
Validate: user owns this quiz (WHERE user\_id = \$1 AND id = \$2)  
Call update\_sm2() -> UPDATE quizzes SET ease\_factor, interval\_days, next\_review = CURRENT\_DATE + interval\_days

Telegram /quiz command:

Fetch one due quiz -> send inline keyboard with 4 options + question text.  
User taps answer -> POST /api/quizzes/{id}/answer with quality derived from correct/wrong.

## Rules:

- ease\_factor floor is 1.3 -- never go below (clamped in update\_sm2).
- Quality must be 0-5 -- validate and reject 400 if out of range.
- next\_review = CURRENT\_DATE + interval\_days (server-side date, not client).
- Quiz ownership: ALWAYS verify user\_id before UPDATE -- cross-user tampering prevented.
- Unit test: all 5 cases from TESTING.md §4 with exact input/output values.

## Gate Check:

- [ ] Correct easy answer (quality=5) with ef=2.5, interval=1 -> ef=2.6, interval=3
- [ ] Wrong answer (quality=1) resets interval to 1
- [ ] ease\_factor never drops below 1.3
- [ ] GET /api/quizzes/due only returns quizzes for authenticated user
- [ ] Unit test: test\_sm2.py with all 5 cases -- exact numeric matches required



## PROMPT 074 -- Mobile Responsive Layouts

**SKILLS:** [mobile-design](#) . [react-best-practices](#) . [ui-ux-pro-max](#)

Make the dashboard fully responsive for mobile (375px -- Telegram TWA) and tablet (768px).

Breakpoints:

Mobile: max-width: 600px (Telegram TWA = 375px)  
Tablet: max-width: 900px  
Desktop: min-width: 901px

Dashboard layout changes at mobile:

- Header: hide search bar text, show search icon only (tap to expand).
- NodePanel: slides up from bottom (not from right) -- full width, 70% height.
- Feed: single column.
- Graph: full viewport, remove padding.

Canvas touch interactions:

Touch pan: single finger drag -> translate canvas  
Touch zoom: two-finger pinch -> scale canvas  
Tap: equivalent to click (open NodePanel)  
Long press (500ms): show context menu (Delete item, View source)

TWA-specific:

window.Telegram.WebApp.viewportStableHeight -> set canvas height  
window.Telegram.WebApp.expand() on mount -> use full screen  
Telegram.WebApp.BackButton.show() when NodePanel is open  
Telegram.WebApp.BackButton.onClick -> close NodePanel  
Telegram.WebApp.MainButton -> hide (not needed)

Rules:

- All touch targets: min 44px x 44px (PRD mobile requirement).
- No hover effects on mobile (hover: none media query).
- TWA: always call Telegram.WebApp.ready() on load.
- Viewport meta: <meta name="viewport" content="width=device-width, initial-scale=1, user-scalable=no">
- Pinch zoom on canvas must NOT trigger browser zoom -- call event.preventDefault().

Gate Check:

- [ ] Dashboard usable at 375px width (Telegram TWA)
- [ ] NodePanel slides up from bottom on mobile
- [ ] Touch pan and zoom work on canvas
- [ ] BackButton closes NodePanel in TWA
- [ ] All interactive elements >= 44px touch target (measure in DevTools)
- [ ] Vitest: responsive breakpoints tested via window.innerWidth mock

## PROMPT 075 -- Security Audit Pass

**SKILLS:** [security-audit](#) . [security-auditor](#) . [web-security-testing](#) . [api-security-best-practices](#)

Run a full security audit against the completed Recall backend.

Checklist to verify:

### 1. INJECTION

- [ ] All SQL queries use parameterised statements (\$1, \$2, ...) -- grep for f-string SQL and fail if found
- [ ] No ORM raw() calls with user input
- [ ] BeautifulSoup HTML parsing is read-only -- no exec/eval on scraped content

## 2. AUTHENTICATION

- [ ] hmac.compare\_digest used in all HMAC comparisons -- grep for == near hash variables
- [ ] JWT verified with PyJWT verify=True -- no decode(verify=False) anywhere
- [ ] Cookie: httpOnly + Secure + SameSite=Lax verified in all Set-Cookie headers

## 3. ENCRYPTION

- [ ] raw\_text stored in DB: run SELECT raw\_text FROM items LIMIT 1 -- must start with "gAAAAA" (Fernet prefix)
- [ ] google\_refresh\_token: same check
- [ ] FERNET\_KEY not in any log file: grep all logs for the key value

## 4. RATE LIMITING

- [ ] /webhook rate limited: verify 21st request in window is dropped
- [ ] /health NOT rate limited: verify Uptime Robot pings are never blocked

## 5. CROSS-USER DATA

- [ ] Every /api/\* route: verify user\_id from JWT is in WHERE clause -- grep DB queries
- [ ] GET /api/graph: attempt with User A's JWT to access User B's data -- must return empty/401

## 6. SENSITIVE DATA EXPOSURE

- [ ] No TELEGRAM\_BOT\_TOKEN in any response body or log
- [ ] No stack traces in 500 responses
- [ ] No internal DB error messages in responses

## 7. GOOGLE OAUTH

- [ ] Scope is exactly drive.file -- check Google Cloud Console
- [ ] access\_token not in any DB column or log

## Rules:

- This prompt produces a written security report (markdown), not code.
- Every failed check must have a corresponding fix before moving to deployment.
- Use grep commands to verify -- do not rely on memory.

## Gate Check:

- [ ] All 7 categories pass
- [ ] Security report written to docs/SECURITY\_AUDIT\_REPORT.md
- [ ] Zero grep matches for: f-string SQL, verify=False JWT, == hash comparison
- [ ] Fernet prefix confirmed on encrypted DB values

## PHASE 10: AUTOMATED TESTING SUITE

### PROMPT 076 -- Bot Inline Keyboard Quiz Flow

**SKILLS:** telegram-bot-builder . telegram

Implement the full Telegram inline keyboard quiz flow.

/quiz command flow:

1. SELECT one quiz WHERE user\_id=\$1 AND next\_review <= CURRENT\_DATE ORDER BY next\_review ASC LIMIT 1
2. If no due quizzes: " No quizzes due! Come back tomorrow."
3. If quiz found: send inline keyboard message:  
Question text (bold)

```

4 inline buttons: [A. {option[0]}} [B. {option[1]}}
 [C. {option[2]}} [D. {option[3]}}
Each button callback_data: "quiz:{quiz_id}:{option_index}"

Callback query handler (when user taps an option):
1. Parse callback_data: quiz_id, chosen_index
2. Verify user owns this quiz: SELECT user_id FROM quizzes WHERE id=$1
3. Determine quality:
 - correct_index == chosen_index -> quality = 5 (correct, easy)
 - wrong -> quality = 2 (wrong)
4. Call update_sm2() -> update quiz row
5. Edit the original message (not send new):
 If correct: "[OK] Correct!\n\n{explanation}\n\nNext review: {next_review_date}"
 If wrong: "[FAIL] The answer was {correct_option}\n\n{explanation}\n\nReview again in 1 day."
6. Next question button: [Next Quiz ->] (shows next due quiz if any)

Rules:
- callback_data must include quiz_id -- never rely on message state.
- Editing the message removes the buttons after answer -- prevents repeat tapping.
- Verify user_id on every callback -- prevent cross-user quiz manipulation.
- Explanation must always be shown (correct or wrong) -- reinforces learning.
- If quiz row not found on callback: silently ignore (stale keyboard).

Gate Check:
[] /quiz sends inline keyboard with 4 options
[] Tapping correct option edits message to show success + explanation
[] Tapping wrong option shows correct answer + explanation
[] Tapping old button after message already edited: silently ignored (no crash)
[] Unit test: test_quiz_flow.py with mocked Telegram bot API

```

## PROMPT 077 -- Quiz History + Performance Tracking

**SKILLS:** [python-fastapi-development](#) . [react-ui-patterns](#)

Add quiz history and performance tracking.

```

Backend -- GET /api/quizzes/stats:
{
 "total": int, -- total quizzes in DB for user
 "due_today": int, -- next_review <= today
 "answered_all_time": int, -- would need a quiz_answers log table (add below)
 "avg_ease_factor": float,
 "mastered": int -- quizzes with ease_factor >= 2.5 AND interval_days >= 7
}

```

Add quiz\_answers log table (ADD to schema):

```

CREATE TABLE quiz_answers (
 id SERIAL PRIMARY KEY,
 user_id INT REFERENCES users(id) ON DELETE CASCADE,
 quiz_id INT REFERENCES quizzes(id) ON DELETE CASCADE,
 quality INT NOT NULL,
 answered_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
INSERT into quiz_answers on every POST /api/quizzes/{id}/answer.

```

Frontend -- Stats card in header or settings:

Due today: 12 | Mastered: 45 | Avg ease: 2.3

Click -> opens QuizStats panel with history chart (simple bar chart using Canvas 2D, no library)

## Quiz history chart:

x-axis: last 7 days  
 y-axis: quizzes answered per day (from quiz\_answers)  
 Bar fill: --color-primary with glass sheen  
 Labels: day abbreviation (Mon, Tue...)

## Rules:

- quiz\_answers table is append-only -- never update or delete rows.
- Stats query must be a single query (not 4 separate queries) using subqueries.
- Chart is Canvas 2D -- no chart library (D3 is already in use -- avoid adding another charting lib).
- "Mastered" definition: ease\_factor >= 2.5 AND interval\_days >= 7 (document this in API response).

## Gate Check:

[ ] GET /api/quizzes/stats returns all 5 fields  
 [ ] quiz\_answers row created on every quiz answer  
 [ ] History chart renders 7 days of data  
 [ ] Mastered count accurate (test with known quiz states)  
 [ ] Unit test: test\_quiz\_stats.py verifies stats query logic

## PROMPT 078 -- APScheduler Setup + All 5 Jobs

**SKILLS:** [async-python-patterns](#) . [python-fastapi-development](#)

Set up APScheduler in backend/scheduler/scheduler.py.

```
from apscheduler.schedulers.asyncio import AsyncIOScheduler
```

```
scheduler = AsyncIOScheduler()
```

Start scheduler in FastAPI lifespan startup event.

Set misfire\_grace\_time=60 on ALL jobs.

Implement all 5 jobs from SCHEDULER.md:

Job 1 -- reminders\_dispatcher (every 1 minute):

```
SELECT * FROM reminders WHERE remind_at <= NOW() AND status='pending' LIMIT 50
For each: call Telegram sendMessage; UPDATE status='sent' on success, 'failed' on failure.
```

Job 2 -- louvain\_clustering (daily 02:00 UTC):

```
For each user with >= 10 new items since last run:
 Fetch embeddings -> build k-NN graph (cosine > 0.75) -> run community_louvain.best_partition()
 For each community: compute centroid, generate label via Llama3
 DELETE existing hubs -> INSERT new hubs -> broadcast via WS hubs_updated event
```

Job 3 -- partition\_creator (monthly, 25th 00:00 UTC):

```
CREATE TABLE IF NOT EXISTS items_yYYYYMMM PARTITION OF items FOR VALUES FROM (...) TO (...)
```

Job 4 -- drive\_nudge\_sender (daily 10:00 UTC):

```
SELECT users WHERE streak_count >= 3 AND drive_nudge_sent = FALSE AND google_refresh_token IS NULL
Send nudge message -> UPDATE drive_nudge_sent = TRUE
```

Job 5 -- processed\_updates\_cleanup (weekly Sunday 03:00 UTC):

```
DELETE FROM processed_updates WHERE processed_at < NOW() - INTERVAL '30 days'
```

## Rules:

- Each job must be wrapped in try/except -- one job failure must NOT crash the scheduler.
- Job functions must be async -- use AsyncIOScheduler, not BackgroundScheduler.

- Louvain clustering DELETE+INSERT must be in a DB transaction (atomic).
- partition\_creator failure must log CRITICAL (not just WARNING) -- missing partition = data loss.
- drive\_nudge\_sent UPDATE must happen AFTER Telegram API call succeeds.
- misfire\_grace\_time=60 on ALL jobs -- mandatory per global rules.

Gate Check:

- [ ] All 5 jobs registered and appear in scheduler.get\_jobs()
- [ ] reminders\_dispatcher sends a test reminder within 1 minute
- [ ] partition\_creator creates next month's partition idempotently (IF NOT EXISTS)
- [ ] Job exception does NOT crash scheduler (verified by injecting exception in test)
- [ ] Unit test: test\_scheduler.py for reminders\_dispatcher algorithm with mocked DB

## PROMPT 079 -- Error Boundary + Network Error Handling

### SKILLS: react-best-practices . error-handling-patterns

Add React error boundaries and network error handling to the frontend.

Error Boundary (frontend/src/components/ErrorBoundary.jsx):

```
class ErrorBoundary extends React.Component
 Catches render errors from child tree.
 Fallback UI:
 Glass card, centered
 Icon: @phosphor-icons Warning (48px, amber)
 Title: "Something went wrong"
 Button: [Reload] -> window.location.reload()
 Log error to console.error (not sent to any external service -- no analytics in v1).
```

Wrap in ErrorBoundary:

```
<ErrorBoundary><GraphCanvas /></ErrorBoundary>
<ErrorBoundary><Feed /></ErrorBoundary>
<ErrorBoundary><NodePanel /></ErrorBoundary>
```

Network error handling in axios client (frontend/src/api/client.js):

```
Interceptor on response error:
- 401: clear auth state -> redirect to login page
- 429: show toast "Too many requests -- please wait"
- 503: show toast "Server unavailable -- retrying in 30 s"
- Network error (no response): show toast "Connection lost -- check your internet"
```

Offline detection:

```
window.addEventListener('offline') -> show persistent toast "You're offline"
window.addEventListener('online') -> dismiss toast, trigger data refetch
```

Rules:

- Error boundary must NOT catch async errors (these are caught by axios interceptor).
- 401 interceptor must NOT fire for /auth/\* routes (to prevent redirect loops).
- Offline toast must be persistent (no auto-dismiss) until online event fires.
- Never display raw error messages from server to user -- translate to human-readable.

Gate Check:

- [ ] Render error in GraphCanvas shows fallback UI, not blank screen
- [ ] 401 response redirects to login
- [ ] Going offline shows persistent toast
- [ ] Coming online dismisses toast and refetches
- [ ] Vitest: ErrorBoundary renders fallback when child throws

## PROMPT 080 -- Keyboard Shortcuts + Accessibility

**SKILLS:** `ui-a11y` . `react-best-practices`

Add keyboard shortcuts and WCAG 2.1 AA accessibility to the dashboard.

Keyboard shortcuts (frontend/src/hooks/useKeyboardShortcuts.js):

```
/ -> focus search bar
Escape -> close NodePanel / clear search
Cmd+K / Ctrl+K -> open command palette (simple: same as search focus)
F -> switch to Feed view
G -> switch to Graph view
? -> show keyboard shortcuts modal
```

Shortcuts modal:

- Glass card overlay, centered
- Table of shortcut -> action
- Dismiss with Escape or clicking outside

Accessibility requirements:

- All interactive elements: tabIndex and role attributes
- GraphCanvas: role="application" aria-label="Knowledge constellation"
- NodePanel: role="dialog" aria-modal="true" aria-labelledby="node-title-id"
- Focus trap in NodePanel (Tab cycles through panel elements only when open)
- Skip link at top: `<a href="#main-content" class="skip-link">Skip to content</a>`
- Colour contrast: all text must meet WCAG AA (4.5:1 ratio) against background
- Icons: aria-hidden="true" when decorative; aria-label when standalone

Reduced motion:

```
@media (prefers-reduced-motion: reduce):
 - No canvas animations
 - No pulse rings
 - No nebula blob movement
 - Instant transitions
```

Rules:

- Keyboard shortcuts must NOT fire when user is typing in an input field.
- Focus ring must be visible on all focusable elements (outline: 2px solid var(--color-primary)).
- Tab order must be logical: header -> main content -> panel.
- useKeyboardShortcuts must remove event listeners on unmount.

Gate Check:

```
[] / key focuses search bar when not in an input
[] Escape closes NodePanel
[] NodePanel focus trap: Tab cycles within panel
[] Skip link visible on first Tab press
[] All colour combinations tested in browser accessibility checker
[] Vitest: useKeyboardShortcuts fires correct callback on keydown
```

## PROMPT 081 -- Node Side Panel Component

**SKILLS:** `react-ui-patterns` . `ui-ux-pro-max`

Create frontend/src/components/NodePanel.jsx -- the glassmorphic node detail panel.

Props: { node: NodeData | null, onClose: () => void }

Content when node !== null:

- Title (Outfit font, 20px)
- Source type badge (icon from @phosphor-icons/react: Link, Microphone, FilePdf, Image, TextT)
- Summary text (Inter, 14px, --text-secondary)
- Source URL (if present, clickable link)
- Tags (pill badges in --color-primary)
- Created at (JetBrains Mono, 12px)
- [Open Quiz] button (if quiz exists for this item)
- [Set Reminder] button (opens reminder input)

Styling:

- Position: fixed, right: 0, top: 56px (below header)
- Width: 360px
- glass-card + glass-glow-top CSS classes
- Slide-in animation: translateX(360px -> 0) using cubic-bezier(0.16, 1, 0.3, 1)
- Close on Escape key or clicking canvas background

Rules:

- Icons MUST use @phosphor-icons/react -- NO emojis as structural icons (ui-ux-pro-max rule).
- Panel slide-in uses CSS transition, NOT JS animation -- performance.
- All text in panel is PLAINTEXT -- never display raw\_text (encrypted).
- Escape key handler must be removed on unmount (no memory leak).
- Panel must be keyboard-navigable (focus trap while open).
- @media (prefers-reduced-motion: reduce) -> instant show/hide, no slide animation.

Gate Check:

- [ ] Panel slides in when node is clicked
- [ ] Escape key closes panel
- [ ] @phosphor-icons used for all content type badges (no emojis)
- [ ] Panel is scrollable for long summaries
- [ ] Vitest: NodePanel renders with mock node data and matches snapshot

## PROMPT 082 -- Progressive Web App (PWA) Configuration

**SKILLS:** react-best-practices . mobile-design

Add PWA support to the React frontend for install-to-homescreen capability.

Install vite-plugin-pwa:

```
npm install -D vite-plugin-pwa
```

vite.config.js PWA config:

```
VitePWA({
 registerType: 'autoUpdate',
 manifest: {
 name: 'Recall',
 short_name: 'Recall',
 description: 'Your AI knowledge constellation',
 theme_color: '#030307',
 background_color: '#030307',
 display: 'standalone',
 orientation: 'any',
 icons: [
 {src: 'icons/icon-192.png', sizes: '192x192', type: 'image/png'},
 {src: 'icons/icon-512.png', sizes: '512x512', type: 'image/png'}
]
 },
 workbox: {
```

```
runtimeCaching: [
 {urlPattern: /\api\graph/, handler: 'StaleWhileRevalidate', options: {cacheName: 'graph-cache'}},
 {urlPattern: /\api\items/, handler: 'NetworkFirst', options: {cacheName: 'items-cache'}}
]
}
})
```

Icons: generate 192x192 and 512x512 PNG icons in the Cosmic Noir style.  
(Use generate\_image tool or create with Canvas code)

Service worker behaviour:

```
Cache: /api/graph (stale-while-revalidate -- graph can be slightly stale)
Cache: /api/items (network-first -- items must be fresh)
No-cache: /api/search (must always be fresh), /auth/* (must never be cached)
```

Rules:

- Theme colour must match --bg-deep (#030307) -- matches native app experience.
- display: 'standalone' -- no browser chrome when installed.
- Service worker must not cache /auth/\* or /api/search.
- PWA install prompt: show in-app "Add to homescreen" toast after 3 visits (not on first load).

Gate Check:

```
[] Lighthouse PWA score: Installable [x]
[] app installable on Android homescreen
[] /api/graph served from cache when offline
[] /auth/* correctly excluded from SW cache
[] Icons 192x192 and 512x512 present and valid
```

## PROMPT 083 -- Monitoring: Structured Logging + Alerts

**SKILLS:** performance-engineer . performance-optimizer

Add production-grade structured logging with alert conditions.

Backend logging setup:

Replace all print() and basic logging calls with structlog.

Every log entry must include:

```
{timestamp, level, service: "recall-api", route, method, status_code, duration_ms, user_id (if authed)}
```

NEVER log these fields (add to a structlog processor that scrubs them):

```
TELEGRAM_BOT_TOKEN, FERNET_KEY, JWT_SECRET, google_refresh_token (any form),
raw_text (decrypted), access_token, initData
```

Alert conditions (log at level CRITICAL with keyword "ALERT"):

- "ALERT: Partition missing for {month}" -- partition\_creator job fails
- "ALERT: Dead letter queue has {N} unretried entries" -- DLQ > 10 items
- "ALERT: Cascade total failure for user {user\_id}" -- all 4 tiers failed

Render log integration:

Render streams logs to their dashboard. Add a log search query for "ALERT" in Render dashboard.  
Document this in DEPLOYMENT.md.

Frontend error logging:

```
window.onerror -> log to console.error only (no external service in v1)
Unhandled promise rejection -> log to console.error
```

Health check enhancement:

```
GET /health/detailed includes:
```



```
{"scheduler_jobs_running": bool, "dlq_count": int, "queue_length": int}
```

Requires X-Internal-Key.

#### Rules:

- structlog processor list must include a sensitive-field scrubber as the LAST processor.
- CRITICAL level used only for ALERT conditions -- not for normal errors.
- duration\_ms: measured with `time.perf_counter_ns() / 1_000_000` for accuracy.
- Log format: JSON in production, human-readable (colorized) in development.

#### Gate Check:

- [ ] Every request produces a structured JSON log with all required fields
- [ ] Sending request with FERNET\_KEY in body: key value NOT in log output
- [ ] Missing partition triggers CRITICAL "ALERT: Partition missing" log
- [ ] GET /health/detailed returns scheduler and DLQ status
- [ ] Unit test: structlog scrubber removes sensitive fields

## PHASE 11: DEPLOYMENT, SECURITY & OBSERVABILITY

### PROMPT 084 -- Streak Counter + Drive Nudge Logic

**SKILLS:** [python-fastapi-development](#) . [postgres-best-practices](#)

Implement streak tracking in `backend/services/streak_service.py`.

```
async def update_streak(user_id: int, db: AsyncConnection):
 SELECT last_activity_date FROM users WHERE id = $1
 today = date.today()
 If last_activity_date == today: no-op (already updated today)
 If last_activity_date == today - 1 day: streak_count += 1
 Else (gap > 1 day): streak_count = 1
 UPDATE users SET streak_count = $2, last_activity_date = $3 WHERE id = $1
```

Call `update_streak()` after every successful item save in `worker.py`.

In bot, add `/streak` command:

```
SELECT streak_count, last_activity_date FROM users WHERE id = $1
Reply: " {streak_count} day streak! Keep saving knowledge."
```

#### Rules:

- Streak comparison uses server-side DATE -- never trust client-provided dates.
- `update_streak` is called AFTER item is confirmed saved -- not before.
- Streak of 0 is valid (user's first day or after a gap).
- No streak bonus or gamification in v1 -- just display.

#### Gate Check:

- [ ] Saving 3 items on consecutive days produces `streak_count = 3`
- [ ] Gap of 2 days resets streak to 1
- [ ] Saving multiple items in one day counts as one streak increment
- [ ] Unit test: `test_streak.py` for all three cases (today, yesterday, gap)

## PROMPT 085 -- Louvain Clustering + Hub Nodes in Canvas

**SKILLS:** python-pro . react-patterns

Complete the louvain\_clustering job with full implementation.

Backend (already stubbed in PROMPT 039 -- flesh out):

1. Use python-louvain (community) package.
2. Build networkx Graph: nodes=item\_ids, edges where cosine\_sim > 0.75.
3. partition = community\_louvain.best\_partition(G)
4. Group items by community ID.
5. For each community with >= 3 members:
  - a. centroid = mean of all member embeddings (numpy)
  - b. label = await ai\_cascade.summarise(community\_summaries\_joined, task="label")  
 Prompt: "What single theme connects these items? Answer in 4 words or less."
  - c. INSERT into semantic\_hubs (user\_id, label, centroid, member\_ids)
6. Broadcast hubs\_updated via WebSocket.

Frontend hub node rendering in GraphCanvas.jsx:

- Hub node: radius = 16px (vs orbital 8px)
- Color: --color-accent (#00D4AA)
- Outer dashed ring: rotates 360deg every 8 s (CSS animation on overlay div or canvas arc)
- Click hub: highlight all member nodes; dim non-members to 20% opacity
- Hub label rendered below node in Inter 500 12px

Rules:

- Only create hubs for communities with >= 3 members (avoids trivial 1-2 item clusters).
- DELETE existing semantic\_hubs for user BEFORE INSERT (inside transaction).
- If Louvain fails for a user: log error, skip user, continue to next user -- never crash job.
- centroid stored as pgvector format in semantic\_hubs.centroid.

Gate Check:

- [ ] With 15+ items: Louvain creates >= 2 hub rows in semantic\_hubs
- [ ] Hub labels are <= 4 words
- [ ] Hub nodes render larger than orbital nodes in canvas
- [ ] Clicking hub dims non-member nodes
- [ ] Unit test: test\_louvain.py with mocked networkx and AI cascade

## PROMPT 086 -- URL Ingestion: Scraping + Save

**SKILLS:** python-pro . async-python-patterns

Implement URL content ingestion in backend/services/ingestion/url\_ingester.py.

Pipeline:

1. Detect if URL is Instagram/YouTube (special handling) or plain web URL.
2. For plain URLs: use httpx + BeautifulSoup to scrape <title> and visible text.
  - Strip scripts, styles, nav, footer, header tags.
  - Truncate text to 4000 chars.
3. For Instagram URLs: try ZenRows (ZENROWS\_KEY) -> ScrapingBee (SCRAPINGBEE\_KEY) -> ScraperAPI (SCRAPERAPI\_KEY) -> yt-dlp -> bookmark fallback.
  - Cookie rotation and User-Agent spoofing must be used from Day 1 (see APP\_FLOW.md).
4. Fernet-encrypt the raw\_text before DB write.
5. INSERT into items: source\_type='url', source\_url=url, raw\_text=<encrypted>, title=<title>
6. Bot reply: "{title}\n\n{first 200 chars of scraped text}\n\nSaved [x]"

Rules:

- httpx timeout: 10 s. Never block indefinitely on slow sites.
- BeautifulSoup parser: 'html.parser' (no lxml dependency).
- raw\_text MUST be Fernet-encrypted -- use services/encryption.py from PROMPT 027.
- summary and embedding are NULL at this stage -- populated in Phase 2/3.
- Scraping failures must log the error and fall back to bookmark (source\_url only, no raw\_text).

Gate Check:

- [ ] Forwarding https://example.com creates an items row with title and encrypted raw\_text
- [ ] Decrypting raw\_text with FERNET\_KEY returns the original scraped text
- [ ] Scraping timeout after 10 s falls back to bookmark
- [ ] Unit test: test\_url\_ingester.py mocks httpx and verifies DB insert fields

## PROMPT 087 -- Smoke Test Script (Production Verification)

**SKILLS:** python-pro . testing-patterns

Create backend/scripts/smoke\_test.py -- a production smoke test run after every deploy.

Usage: python smoke\_test.py --api-url https://recall-api.onrender.com --token {test\_jwt}

Tests run in sequence:

T1 -- Health check:

GET /health -> assert status == "ok"  
Assert response time < 200 ms

T2 -- Authenticated request:

GET /api/items with valid JWT cookie -> assert 200

T3 -- Search works:

POST /api/search {"query": "test"} -> assert 200, valid JSON structure

T4 -- Graph loads:

GET /api/graph -> assert 200, nodes is a list

T5 -- WebSocket connects:

Connect to /ws/{jwt} -> assert "connected" event within 2 s

T6 -- Rate limiter active:

POST /webhook 25 times quickly -> assert at least one is rejected (200 but no task enqueued -- check via T7)

T7 -- DLQ accessible:

GET /api/admin/queue with INTERNAL\_API\_KEY -> assert queue\_length is an integer

Output: JSON report with each test result and duration.

Exit code: 0 if all pass, 1 if any fail.

Rules:

- Smoke test must NOT use production bot token -- use a test JWT generated with the FERNET\_KEY.
- Smoke test must run against staging/production URL -- not localhost.
- T6 does not require actual task processing -- just verifies webhook returns 200.
- Output file: smoke\_test\_report\_{timestamp}.json in /tmp (not committed).

Gate Check:

- [ ] All 7 tests pass against production API
- [ ] Script exits with code 0 on success, 1 on failure
- [ ] T5: WebSocket connection within 2 s
- [ ] Report JSON generated with per-test durations

[ ] Script works with --api-url parameter (not hardcoded URL)

## PROMPT 088 -- Image Ingestion

### SKILLS: python-pro

Implement backend/services/ingestion/image\_ingester.py.

Pipeline:

1. Download image from Telegram (photo message or document with image MIME).
2. Run Tesseract OCR on the image (pytesseract.image\_to\_string).
3. If OCR text > 50 chars: call AICascade.embed(ocr\_text) and AICascade.summarise(ocr\_text).
4. If OCR text <= 50 chars: use Gemini Tier 2 for image captioning (send image bytes).
5. Fernet-encrypt OCR text (or caption) -> raw\_text.
6. INSERT into items: source\_type='image', raw\_text=<encrypted>, summary=<summary or caption>, embedding=<vector>
7. Bot reply:  
If OCR: " Extracted text:\n{first 200 chars}...\n\nSaved [x]"  
If caption: " Caption: {caption}\n\nSaved [x]"

Rules:

- Image size limit: 10 MB.
- Tesseract must run with lang='eng' by default; do not auto-detect language (performance).
- Temporary image file: always delete in finally block.
- Images with 0 OCR text and Gemini captioning failure: bookmark fallback.

Gate Check:

- [ ] Image with text -> items row with encrypted OCR text and embedding
- [ ] Image without text -> Gemini caption used as summary
- [ ] Temp file deleted in all paths
- [ ] Unit test: test\_image\_ingester.py with mocked Tesseract and AICascade

## PROMPT 089 -- WebSocket Connection Status UI

### SKILLS: react-state-management . react-ui-patterns

Add real-time connection status indicator to the dashboard header.

Frontend/src/components/ConnectionStatus.jsx:

Small indicator dot in header (right side, before profile icon)

States:

- Connected: pulsing green dot (--color-accent #00D4AA)
- Connecting/Reconnecting: spinning amber dot
- Disconnected: static red dot + tooltip "Reconnecting..."

On hover: tooltip with connection state text

WebSocket lifecycle events (from useGraphSocket hook):

onopen -> setState('connected')

onclose -> setState('disconnected') -> attempt reconnect (exponential backoff: 1,2,4,8,16 s)

onerror -> setState('error')

After 5 failed reconnects -> setState('failed') -> toast: "Real-time updates unavailable. Refresh to retry."

Also add to header:

Last sync timestamp (updated on every WS event or API response):

```
"Last updated: 2 minutes ago" (JetBrains Mono, 11px, --text-tertiary)
Updates every 30 s via setInterval.
```

## Rules:

- Do NOT show raw WebSocket error messages to user.
- After 5 failed reconnects: stop retrying (do not loop forever).
- Connection status must not trigger re-renders of GraphCanvas (use separate context or memo).
- Green pulse animation: @keyframes pulse, 2 s infinite.

## Gate Check:

- [ ] Dot shows green when WS connected
- [ ] Disconnecting server shows amber spinning dot then red dot
- [ ] After 5 reconnect failures: toast shown, attempts stop
- [ ] "Last updated" timestamp updates without re-rendering canvas
- [ ] Vitest: ConnectionStatus renders correct state per prop

## PROMPT 090 -- Graph API Endpoint

**SKILLS:** python-fastapi-development . postgres-best-practices

Create GET /api/graph endpoint in backend/routes/api.py.

## Response schema:

```
{
 "nodes": [{"id": int, "title": str, "source_type": str, "created_at": str, "is_hub": bool}],
 "edges": [{"source": int, "target": int, "weight": float}],
 "hubs": [{"id": int, "label": str, "member_ids": list[int]}]
}
```

Nodes: SELECT id, title, source\_type, created\_at FROM items WHERE user\_id = \$1

Hubs: SELECT id, label, member\_ids FROM semantic\_hubs WHERE user\_id = \$1

Edges: For each pair of items where their cosine similarity > 0.75:

embedding <=> other\_embedding < 0.25

(Only compute for up to 200 items -- skip edge calculation for larger graphs for performance)

## Rules:

- ALL queries include WHERE user\_id = \$1 from JWT.
- Response target: < 200 ms (pre-computed hubs -- no live clustering).
- Edge computation limit: if user has > 200 items, return only hub-membership edges (not pairwise).
- Never return raw\_text in graph API response -- titles and summaries only.
- Hub member\_ids must reference valid item IDs -- validate before returning.

## Gate Check:

- [ ] GET /api/graph returns valid nodes/edges/hubs JSON
- [ ] Response time < 200 ms for 100 nodes (measure in test)
- [ ] raw\_text is absent from all response fields
- [ ] User A cannot access User B's graph (401 if wrong JWT)
- [ ] Unit test: test\_graph\_api.py verifies schema and auth

## PROMPT 091 -- GET /api/graph Optimisation + Edge Pruning

**SKILLS:** postgres-best-practices . postgresql-optimization . python-fastapi-development

Optimise the GET /api/graph endpoint from PROMPT 088 for performance.

Problem: Pairwise edge computation for 500 items = 125,000 comparisons -- too slow.

Solution -- Chunked HNSW edge computation:

For each item, find its top-5 nearest neighbours via:

```
SELECT target.id, (source.embedding <=> target.embedding) AS dist
FROM items source, items target
WHERE source.id = $1 AND target.user_id = $2 AND target.id != $1
ORDER BY dist LIMIT 5
```

Run this for up to 100 items ( $N * 5 = 500$  edges max).

For users with > 100 items: only compute edges for the 100 most recent.

Response caching:

Cache graph response in Upstash Redis with key "graph:{user\_id}".

TTL: 60 seconds (graph doesn't change that frequently).

Invalidate cache when new item is saved (in worker.py after item INSERT).

Edge deduplication:

If edge (A->B) and (B->A) both appear: keep only one with the lower ID as source.

Rules:

- Graph computation must complete in < 200 ms for 500 nodes.
- Cache invalidation must happen BEFORE WebSocket new\_node event fires.
- Never cache user A's graph with user B's data -- key must include user\_id.
- Redis cache key: "graph:{user\_id}" -- user\_id from verified JWT only.
- Unit test: verify graph endpoint uses cache on second call (mock Redis).

Gate Check:

- [ ] GET /api/graph < 200 ms for 500 items (measure with timeit)
- [ ] Second call within 60 s uses Redis cache (logged cache hit)
- [ ] Saving new item invalidates cache
- [ ] Edge (A, B) appears once -- not duplicated as (A, B) and (B, A)
- [ ] Unit test: test\_graph\_cache.py verifies cache hit/miss logic

## PROMPT 092 -- Telegram TWA Registration

**SKILLS:** telegram-mini-app . telegram

Register the web dashboard as a Telegram Mini App (TWA).

Steps:

1. @BotFather -> /newapp -> select your bot -> provide:
  - Title: Recall
  - Description: Your AI knowledge constellation
  - Photo: 640x360 app thumbnail
  - Demo GIF: optional
  - Web App URL: https://<vercel-url>
2. @BotFather -> /setmenubutton -> select bot -> set URL to https://<vercel-url>
3. In frontend, load Telegram.WebApp SDK:

```
<script src="https://telegram.org/js/telegram-web-app.js"></script>
```
4. On dashboard load: if window.Telegram.WebApp.initDataUnsafe.user exists:
  - > Send initData to backend for TWA auth (HMAC validation from PROMPT 014)
  - > Skip Login Widget flow

5. Set TWA color scheme: MainButton, BackButton using Telegram.WebApp API.

6. Handle Telegram.WebApp.onEvent('themeChanged') -> update CSS variables to match Telegram theme.

#### Rules:

- TWA must work without requiring the user to log in again if already authenticated via bot.
- window.Telegram.WebApp.ready() must be called on app init.
- TWA viewport height: use window.Telegram.WebApp.viewportStableHeight for safe area.
- Mini App canvas: disable scroll -- let canvas handle pan/zoom internally.

#### Gate Check:

- [ ] Opening the bot's menu button opens the dashboard inside Telegram
- [ ] TWA authenticates via HMAC without redirecting outside Telegram
- [ ] Canvas fills TWA viewport correctly (no scroll)
- [ ] Theme changes in Telegram reflected in app colors

## PROMPT 093 -- Monitoring & Observability Setup

### SKILLS: performance-engineer . performance-optimizer

Add production monitoring to the Recall backend.

#### 1. Structured logging:

- Use Python structlog library for JSON-formatted logs.
- Every log entry includes: timestamp, level, route, user\_id (if available), duration\_ms.
- NEVER log: raw\_text, TELEGRAM\_BOT\_TOKEN, FERNET\_KEY, JWT\_SECRET, access\_token.

#### 2. Request timing middleware:

- FastAPI middleware that logs: method, path, status\_code, duration\_ms for every request.
- Webhook handler: log task enqueue time separately.

#### 3. AI cascade metrics:

- Log per-cascade invocation: content\_type, tier\_used, duration\_ms, success.
- This enables tracking Modal usage vs Groq vs Gemini breakdown.

#### 4. Health check endpoint enhancement:

- GET /health/detailed (internal use only, require X-Internal-Key header):  
Returns: {"db": "ok", "redis": "ok", "scheduler\_jobs": 5, "uptime\_seconds": ...}
- Keep GET /health at < 5 ms (no DB query).

#### 5. Uptime Robot alert:

- Add email alert if /health returns non-200 for 2 consecutive checks.

#### Rules:

- /health/detailed must require X-Internal-Key header -- not publicly accessible.
- structlog must never call str() on sensitive objects.
- Duration logging must use time.perf\_counter() -- not time.time() (precision).

#### Gate Check:

- [ ] Every request logs duration\_ms in structured JSON format
- [ ] Cascade tier\_used is logged for every AI processing task
- [ ] GET /health responds in < 5 ms with no DB call (verify via logs)
- [ ] GET /health/detailed returns DB and Redis status
- [ ] Uptime Robot alert configured

## APPENDIX -- SKILLS REFERENCE TABLE

| Phase      | Task                  | Skills to Activate                   |
|------------|-----------------------|--------------------------------------|
| Foundation | Python project setup  | `python-pro` . `python-development`  |
| Foundation | Database DDL          | `neon-postgres` . `postgres-best-pr` |
| Foundation | Config & secrets      | `python-pro` . `security`            |
| Phase 1    | FastAPI app           | `python-fastapi-development` . `fas` |
| Phase 1    | Telegram webhook      | `telegram-bot-builder` . `telegram`  |
| Phase 1    | Rate limiter          | `async-python-patterns` . `python-p` |
| Phase 1    | DB pool               | `neon-postgres` . `async-python-pat` |
| Phase 1    | Encryption            | `security` . `privacy-by-design` .   |
| Phase 2    | Modal GPU endpoints   | `python-pro` . `async-python-patter` |
| Phase 2    | AI Cascade            | `error-handling-patterns` . `async`  |
| Phase 2    | Voice/PDF/Image       | `python-pro` . `async-python-patter` |
| Phase 2    | Dead Letter Queue     | `error-handling-patterns` . `postgr` |
| Phase 3    | Embeddings            | `python-pro` . `postgres-best-pract` |
| Phase 3    | Hybrid search         | `postgres-best-practices` . `postgr` |
| Phase 3    | Auth guard            | `python-fastapi-development` . `aut` |
| Phase 4    | React setup           | `react-best-practices` . `ui-ux-pro` |
| Phase 4    | Canvas renderer       | `react-patterns` . `react-component` |
| Phase 4    | Node panel            | `react-ui-patterns` . `ui-ux-pro-ma` |
| Phase 4    | Dashboard layout      | `react-best-practices` . `senior-fr` |
| Phase 5    | Telegram Login Widget | `auth-implementation-patterns` . `s` |
| Phase 5    | TWA HMAC              | `auth-implementation-patterns` . `t` |
| Phase 5    | WebSocket backend     | `python-fastapi-development` . `asy` |
| Phase 5    | WebSocket frontend    | `react-state-management` . `react-p` |
| Phase 6    | SM-2 algorithm        | `python-pro`                         |
| Phase 6    | APScheduler           | `async-python-patterns` . `python-f` |
| Phase 6    | Louvain clustering    | `python-pro` . `react-patterns`      |
| Phase 7    | Google OAuth          | `auth-implementation-patterns` . `s` |
| Phase 7    | Drive sync            | `python-pro` . `async-python-patter` |
| Phase 8    | Chrome extension      | `chrome-extension-developer` . `aut` |
| Testing    | Backend suite         | `python-testing-patterns` . `testin` |
| Testing    | Frontend suite        | `javascript-testing-patterns` . `re` |
| Testing    | Load testing          | `k6-load-testing`                    |
| Security   | Security audit        | `security-audit` . `security-audito` |
| Deployment | Deployment            | `deployment-engineer`                |
| Monitoring | Observability         | `performance-engineer` . `performan` |

## PROMPT 094 -- Fernet Key Rotation Script

### SKILLS: security . python-pro

Create backend/scripts/rotate\_fernet\_key.py -- safe key rotation tool.

This script must be run as a one-time migration. From SECURITY.md Key Rotation Procedure:

Steps:

1. Accept old\_key and new\_key as CLI arguments (or env vars).
2. Validate both keys are valid Fernet keys.
3. Count encrypted rows: `SELECT COUNT(*) FROM items WHERE raw_text IS NOT NULL`  
`SELECT COUNT(*) FROM users WHERE google_refresh_token IS NOT NULL`
4. Run in a TRANSACTION:  
For each items row with raw\_text:  
plaintext = decrypt(raw\_text, old\_key)



```

 new_ciphertext = encrypt(plaintext, new_key)
 UPDATE items SET raw_text = new_ciphertext WHERE id = old_id AND created_at = old_created_at
 For each users row with google_refresh_token:
 Same decrypt -> re-encrypt
5. COMMIT
6. Print: "Rotation complete. {N} items re-encrypted. {M} tokens re-encrypted."

Dry-run mode:
 --dry-run flag: runs all decryptions and encryptions, but does NOT commit the transaction.
 Useful for testing the migration before production run.

Safety checks:
 - Refuse to run if old_key == new_key.
 - Refuse to run if DATABASE_URL contains "prod" AND --force flag not set.
 - After commit: verify 5 random rows decrypt correctly with new_key.

Rules:
 - Script does NOT update FERNET_KEY env var -- operator must do that after script succeeds.
 - All DB operations in ONE transaction -- all-or-nothing.
 - plaintext must NEVER be logged or printed.
 - --dry-run must be the default when --force is not set (prevents accidental production run).

Gate Check:
[] Dry-run mode completes without committing
[] After full run: all items decrypt correctly with new_key
[] Old key no longer decrypts items (new ciphertext)
[] Script refuses to run if old_key == new_key
[] Unit test: test_key_rotation.py with mocked DB and two valid Fernet keys

```

## PROMPT 095 -- Performance Testing: Frontend Bundle Optimisation

**SKILLS:** [web-performance-optimization](#) . [react-component-performance](#)

Optimise the React frontend bundle for performance.

Analysis:

```

npm run build -- --analyze (or use rollup-plugin-visualizer)
Identify large chunks. Target: main bundle < 200 KB gzipped.

```

Optimisations:

1. Code splitting:

```

Lazy-load Feed and Reminders pages:
const Feed = React.lazy(() => import('./pages/Feed'))
Wrap in <Suspense fallback={<FeedCardSkeleton />}>

```

2. D3 tree-shaking:

```

Import only used D3 modules:
import {forceSimulation, forceLink, forceManyBody, forceCenter} from 'd3-force'
NOT: import * as d3 from 'd3'

```

3. @phosphor-icons: already tree-shaken by default -- verify.

4. Image optimisation:

```

The login page mini-canvas is pure code -- no images to optimise.
Any static images: use WebP format, max 100 KB.

```

5. CSS:

Remove unused CSS custom properties (audit with devtools Coverage tab).  
Consolidate duplicate animation definitions.

Lighthouse audit targets:

Performance: >= 90  
Accessibility: >= 95  
Best Practices: >= 95  
SEO: >= 90

Vite config optimisations:

build.rollupOptions.output.manualChunks: split vendor (d3, react) from app code  
build.minify: 'esbuild' (default, fast)

Rules:

- Lazy-loaded routes must have matching skeleton (not blank screen) during load.
- No CSS framework added for optimisation -- Vanilla CSS is already optimal.
- Lighthouse audit must be run on production build, not dev server.

Gate Check:

[ ] Main bundle < 200 KB gzipped (measure with npm run build + du)  
[ ] Lighthouse Performance >= 90 on production build  
[ ] Lighthouse Accessibility >= 95  
[ ] Feed page is lazily loaded (Network tab shows chunk loaded on demand)  
[ ] D3 imports are tree-shaken (no d3-array, d3-scale etc. if not used)

## PROMPT 096 -- Database Backup Strategy

**SKILLS:** postgres-best-practices . neon-postgres

Document and implement the database backup strategy.

Neon built-in:

Neon free tier: 7-day point-in-time restore.  
Document in DEPLOYMENT.md: how to restore from a specific timestamp.  
Add to runbook: "In case of accidental DELETE, restore Neon branch to timestamp before delete."

Manual backup script (backend/scripts/backup\_db.py):

Export all non-encrypted user data as JSON:  
Users (without google\_refresh\_token)  
Items (with raw\_text DECRYPTED for backups -- user's data)  
Quizzes, Reminders, Hubs  
Output: backup\_{date}.json.gz (gzip compressed)  
Store: in a Google Drive folder "Recall Backups" (if Drive connected) OR locally.

Backup schedule (document in SCHEDULER.md):

Weekly backup via APScheduler job on Sundays at 04:00 UTC.  
Retain last 4 backups (delete older ones).

Backup verification:

After backup: parse the JSON and verify item count matches SELECT COUNT(\*) FROM items.

Rules:

- Backup JSON contains DECRYPTED raw\_text -- backup file itself must be encrypted (gzip is not encryption).
- Encrypt backup file: encrypt the .json with the same FERNET\_KEY before gzip.
- Backup file: NEVER committed to git or stored in the repo.
- Restoration: requires FERNET\_KEY to decrypt backup file -- document this dependency.

Gate Check:

```
[] backup_db.py runs and produces a valid .json.gz file
[] Backup file decrypts and parses correctly
[] Item count in backup matches SELECT COUNT(*) FROM items
[] Backup schedule job registered in APScheduler
[] Restoration procedure documented in DEPLOYMENT.md
```

## PROMPT 097 -- Rollback Procedure

### SKILLS: python-pro

Define and document rollback procedures for the two most critical failure modes.

Document in docs/RUNBOOK.md:

ROLLBACK 1 -- Bad backend deploy (Render):

1. Render dashboard -> select service -> Deploys tab -> find last good deploy.
2. Click "Rollback to this deploy".
3. Verify: GET /health returns 200.
4. Verify: forward a message to bot and confirm it saves.
5. Time to rollback: < 2 minutes.

ROLLBACK 2 -- Corrupt FERNET\_KEY rotation:

Symptoms: DecryptionError on raw\_text or google\_refresh\_token reads.

Steps:

1. Immediately set COMPUTE\_PROVIDER=groq in Render (skip Modal to reduce load while fixing).
2. Restore old FERNET\_KEY in Render env vars.
3. Redeploy.
4. Run: SELECT count(\*) FROM items WHERE raw\_text IS NOT NULL (verify count is > 0).
5. Test: decrypt a sample raw\_text row.
6. If old key doesn't work: restore Neon DB to pre-rotation timestamp (Neon PITR).

ROLLBACK 3 -- Telegram webhook mismatch:

Symptoms: bot not responding.

Check: GET https://api.telegram.org/bot{TOKEN}/getWebhookInfo

Fix: re-register webhook with correct URL.

ROLLBACK 4 -- Neon DB partition missing:

Symptoms: INSERT errors for new items at month boundary.

Fix: immediately run python manage\_partitions.py create --months 1

Then: verify partition exists via manage\_partitions.py status.

Each rollback must include:

- Symptom description
- Detection method
- Step-by-step fix
- Verification steps
- Estimated time to resolve

Gate Check:

```
[] RUNBOOK.md documents all 4 rollback scenarios
[] Each scenario has detection, fix, and verification steps
[] Rollback 1 tested: deploy previous version and verify health
[] Rollback 4 tested: manually drop a partition and verify manage_partitions.py fixes it
```

## PROMPT 098 -- OpenTelemetry Tracing (Optional Enhancement)

**SKILLS:** performance-profiling . performance-engineer

Add basic distributed tracing using OpenTelemetry (no paid service required -- log spans to console/Render logs).

Install:

```
opentelemetry-sdk opentelemetry-instrumentation-fastapi opentelemetry-instrumentation-psycpg2
```

Setup in main.py:

```
from opentelemetry import trace
from opentelemetry.sdk.trace import TracerProvider
from opentelemetry.sdk.trace.export import BatchSpanProcessor, ConsoleSpanExporter

provider = TracerProvider()
provider.add_span_processor(BatchSpanProcessor(ConsoleSpanExporter()))
trace.set_tracer_provider(provider)
```

Instrument FastAPI:

```
FastAPIInstrumentor.instrument_app(app)
```

Custom spans for AI cascade:

```
tracer = trace.get_tracer("recall.cascade")
with tracer.start_as_current_span("cascade.tier0") as span:
 span.set_attribute("tier", 0)
 span.set_attribute("content_type", task.content_type)
 result = await call_modal(...)
```

This allows seeing in Render logs: exact time spent in each cascade tier per request.

Trace what matters:

- /webhook total duration
- cascade.tier0/1/2/3 per-tier duration
- db.query duration per query
- redis.command duration

Rules:

- ConsoleSpanExporter only -- no Jaeger, Zipkin, or paid APM in free-tier setup.
- Traces must NOT include sensitive span attributes (no raw\_text, no tokens).
- Tracing adds < 1 ms overhead per request -- verify this.
- This is an optional enhancement -- if it adds complexity, skip and document why.

Gate Check:

- [ ] Render logs show OpenTelemetry spans for each /webhook request
- [ ] Cascade tier timing visible in spans
- [ ] No sensitive data in span attributes
- [ ] Overhead < 1 ms per request (measure before/after)
- [ ] Documented as "optional" in DEPLOYMENT.md

## PROMPT 099 -- Final Documentation Pass

**SKILLS:** python-pro

Perform a final documentation review and completeness check across all 18 docs in docs/.

For each file, verify:

- Version field updated to match current implementation phase.

- No stubs like "TODO", "implement later", "placeholder" remain.
- All env vars referenced match the final list in ENV\_CONFIG.md.
- All table names match BACKEND\_SCHEMA.md (no renamed columns lingering).

Update DEPLOYMENT.md:

- Add INTERNAL\_API\_KEY to the env vars table (added in PROMPT 052).
- Add quiz\_answers table to BACKEND\_SCHEMA.md (added in PROMPT 038).
- Add item\_chunks table to BACKEND\_SCHEMA.md (added in PROMPT 034).
- Add content\_hash column to items table description (added in PROMPT 048).

Update SCHEDULER.md:

- Add Job 6 -- daily\_digest\_sender (added in PROMPT 050).
- Add Job 7 -- weekly\_backup (added in PROMPT 095).

Update TESTING.md:

- Add Section 7 -- Concurrency tests (rate limiter).
- Add Section 8 -- Integration tests.
- Add Section 9 -- E2E auth tests.

Update ENV\_CONFIG.md:

- Add INTERNAL\_API\_KEY (for /api/admin/\* endpoints).

Create docs/CHANGELOG.md:

- v0.1.0 -- Initial build (Phases 1-11 complete)
- Lists every major feature added per phase.

Rules:

- No doc should reference a service/variable that no longer exists.
- All SQL snippets in docs must match the ACTUAL schema (no drift).
- Changelog must list features, not internal code changes.

Gate Check:

- [ ] BACKEND\_SCHEMA.md includes item\_chunks and quiz\_answers tables
- [ ] ENV\_CONFIG.md includes INTERNAL\_API\_KEY
- [ ] SCHEDULER.md includes Jobs 6 and 7
- [ ] TESTING.md includes sections 7, 8, 9
- [ ] docs/CHANGELOG.md exists with v0.1.0 entry
- [ ] Zero "TODO" strings in any docs file (grep -r "TODO" docs/)

## PROMPT 100 -- Final Acceptance: 0 -> 100% Gate

**SKILLS: All skills referenced throughout this document.**

This is the final acceptance gate for the complete Recall project.

Run through every acceptance criterion from IMPLEMENTATION\_PLAN.md phases 1-8:

PHASE 1 [x]:

- [ ] Forwarding a URL -> items row with title and source\_url
- [ ] GET /health -> 200
- [ ] Duplicate update\_id -> no duplicate row
- [ ] Webhook -> 200 in < 50 ms

PHASE 2 [x]:

- [ ] Voice note -> transcribed text + summary within 15 s
- [ ] raw\_text is Fernet-encrypted in DB (starts with gAAAAA)
- [ ] > 20 msgs/min -> rate limiter active
- [ ] Cascade failure -> dead\_letter\_queue entry + user notification

## PHASE 3 [x]:

- [ ] /search "machine learning" -> semantically relevant results
- [ ] PDF forwarded -> summary within 30 s
- [ ] Image with text -> OCR extract + summary
- [ ] Vector search < 10 ms (EXPLAIN ANALYZE)

## PHASE 4 [x]:

- [ ] Graph renders 100 nodes at >= 60 FPS
- [ ] Clicking node opens side panel with correct data
- [ ] TWA opens inside Telegram Mini App

## PHASE 5 [x]:

- [ ] Website login via Telegram Login Widget works
- [ ] New item via bot -> new node appears in browser within 2 s
- [ ] TWA authenticates without redirect

## PHASE 6 [x]:

- [ ] Correct quiz answer increases interval\_days
- [ ] Louvain job at 02:00 UTC creates hub rows if >= 10 new items
- [ ] Reminder arrives within 1 min of scheduled time
- [ ] partition\_creator creates next month's partition on 25th

## PHASE 7 [x]:

- [ ] Connecting Drive -> encrypted refresh\_token in DB
- [ ] WS event updates Drive icon in real time
- [ ] Exported Doc appears in user's Drive under "Recall" folder
- [ ] drive\_nudge\_sent gates nudge to one message per user

## PHASE 8 [x]:

- [ ] Extension "Save" button -> URL saved to Recall
- [ ] Item appears in mind map within 30 s
- [ ] Extension works without opening Telegram

## SECURITY FINAL [x]:

- [ ] bandit: 0 HIGH findings
- [ ] IDOR: cross-user access blocked on all endpoints
- [ ] Fernet encryption: all raw\_text starts with gAAAAA
- [ ] No secrets in any git commit

## PERFORMANCE FINAL [x]:

- [ ] Webhook p95 < 50 ms
- [ ] Vector search p95 < 10 ms
- [ ] Canvas: 60 FPS at 500 nodes
- [ ] Graph API p95 < 200 ms

If any criterion fails: fix it before marking the project complete.

Document final acceptance in docs/DEPLOYMENT\_SIGN\_OFF.md with date and all items checked.

## Gate Check:

- [ ] ALL items above are checked
- [ ] docs/DEPLOYMENT\_SIGN\_OFF.md signed off with date
- [ ] Smoke test (PROMPT 042) passes in production
- [ ] README.md complete and accurate
- [ ] All 110 prompts in this document have been executed

## APPENDIX A -- PROMPT EXECUTION TRACKER

| #   | Prompt Title                              | Phase | Status |
|-----|-------------------------------------------|-------|--------|
| 001 | Repo Structure                            | P0    | ->     |
| 002 | Neon DDL                                  | P0    | ->     |
| 003 | Config Loader                             | P0    | ->     |
| 004 | FastAPI Skeleton                          | P1    | ->     |
| 005 | Webhook + Idempotency                     | P1    | ->     |
| 006 | Rate Limiter                              | P1    | ->     |
| 007 | DB Connection Pool                        | P1    | ->     |
| 008 | User Upsert                               | P1    | ->     |
| 009 | URL Ingestion                             | P1    | ->     |
| 010 | Fernet Encryption                         | P1    | ->     |
| 011 | Text Ingestion + Worker                   | P1    | ->     |
| 012 | Modal Whisper/Llama/Embed                 | P2    | ->     |
| 013 | AI Cascade Service                        | P2    | ->     |
| 014 | Voice Ingestion                           | P2    | ->     |
| 015 | Dead Letter Queue                         | P2    | ->     |
| 016 | PDF Ingestion                             | P2    | ->     |
| 017 | Image Ingestion                           | P2    | ->     |
| 018 | Embedding Pipeline                        | P3    | ->     |
| 019 | Hybrid Search                             | P3    | ->     |
| 020 | /api/search + Auth                        | P3    | ->     |
| 021 | React + Vite Setup                        | P4    | ->     |
| 022 | Graph API Endpoint                        | P4    | ->     |
| 023 | Canvas Renderer                           | P4    | ->     |
| 024 | Node Side Panel                           | P4    | ->     |
| 025 | Dashboard Layout                          | P4    | ->     |
| 026 | Login Widget + JWT                        | P5    | ->     |
| 027 | TWA HMAC Middleware                       | P5    | ->     |
| 028 | WebSocket Backend                         | P5    | ->     |
| 029 | WebSocket Frontend                        | P5    | ->     |
| 030 | SM-2 + Quiz Endpoints                     | P6    | ->     |
| 031 | APScheduler + 5 Jobs                      | P6    | ->     |
| 032 | Streak Counter                            | P6    | ->     |
| 033 | Louvain + Hub Nodes                       | P6    | ->     |
| 034 | Google OAuth Flow                         | P7    | ->     |
| 035 | Drive Sync Service                        | P7    | ->     |
| 036 | Chrome Extension Popup                    | P8    | ->     |
| 037 | Backend Test Suite                        | P9    | ->     |
| 038 | Frontend Test Suite                       | P9    | ->     |
| 039 | k6 Load Tests                             | P9    | ->     |
| 040 | Security Audit                            | P10   | ->     |
| 041 | Pre-Deploy Checklist                      | P11   | ->     |
| 042 | Go-Live Sequence                          | P11   | ->     |
| 043 | TWA Registration                          | P11   | ->     |
| 044 | Monitoring Setup                          | P11   | ->     |
| 045 | Local Dev Environment                     | P1+   | ->     |
| 046 | GitHub Actions CI                         | P1+   | ->     |
| 047 | Bot Commands (/help /list /delete /stats) | P1+   | ->     |
| 048 | GET /api/items                            | P1+   | ->     |
| 049 | DELETE /api/items + IDOR                  | P1+   | ->     |
| 050 | YouTube Pipeline                          | P2+   | ->     |

| 051 | Upstash Redis Wrapper | P1+ | -> |

| 052 | OpenAPI Spec | P1+ | -> |

| 053 | /remind Command | P6+ | -> |

| 054 | Graph API Optimisation + Cache | P4+ | -> |

| 055 | Quiz Inline Keyboard | P6+ | -> |

| 056 | Tag System | P3+ | -> |

| 057 | Map-Reduce RAG Search | P3+ | -> |

| 058 | Feed View | P4+ | -> |

| 059 | Toast Notification System | P4+ | -> |

| 060 | Empty States + Skeletons | P4+ | -> |

| 061 | Mobile Responsive + TWA | P4+ | -> |

| 062 | Error Boundary + Network Errors | P4+ | -> |

| 063 | Keyboard Shortcuts + A11y | P4+ | -> |

| 064 | Login / Landing Page | P5+ | -> |

| 065 | Logout + Session Refresh | P5+ | -> |

| 066 | WebSocket Status UI | P5+ | -> |

| 067 | Drive Connect UI | P7+ | -> |

| 068 | Disconnect Drive | P7+ | -> |

| 069 | Settings Page | P5+ | -> |

| 070 | API Rate Limits (Web) | P5+ | -> |

| 071 | Hub Map View | P6+ | -> |

| 072 | Reminder UI (Website) | P6+ | -> |

| 073 | Quiz History + Stats | P6+ | -> |

| 074 | Daily Digest Job | P6+ | -> |

| 075 | Streak Visualisation | P6+ | -> |

| 076 | Partition Manager CLI | P6+ | -> |

| 077 | Redis Queue Monitor + DLQ Retry | P2+ | -> |

| 078 | Extension Service Worker | P8+ | -> |

| 079 | Extension Options Page | P8+ | -> |

| 080 | PDF Multi-Chunk Embedding | P3+ | -> |

| 081 | Content Deduplication | P1+ | -> |

| 082 | Image OCR Quality | P2+ | -> |

| 083 | Data Export (GDPR) | P5+ | -> |

| 084 | Integration Tests: Full Save Flow | P9+ | -> |

| 085 | Security Tests: IDOR + Injection | P10+ | -> |

| 086 | Performance Benchmarks | P9+ | -> |

| 087 | E2E Tests: Auth Flows | P9+ | -> |

| 088 | Structured Logging + Alerts | P11+ | -> |

| 089 | Fernet Key Rotation Script | P10+ | -> |

| 090 | SAST + Dependency Audit | P10+ | -> |

| 091 | Frontend Bundle Optimisation | P11+ | -> |

| 092 | Rate Limiter Concurrency Tests | P9+ | -> |

| 093 | Database Backup Strategy | P11+ | -> |

| 094 | Rollback Procedures | P11+ | -> |

| 095 | OpenTelemetry Tracing | P11+ | -> |

| 096 | Final Documentation Pass | P11+ | -> |

| 097 | README.md | P11+ | -> |

| 098 | Smoke Test Script | P11+ | -> |

| 099 | PWA Configuration | P4+ | -> |

| 100 | Final Acceptance Gate | P11 | -> |



## Summary of Build Playbook

| Prompt | Title                                             | Phase   | Done |
|--------|---------------------------------------------------|---------|------|
| 001    | Local Development Environment                     | PHASE 0 | ->   |
| 002    | Repo Structure & Python Environment               | PHASE 0 | ->   |
| 003    | Neon Database DDL                                 | PHASE 0 | ->   |
| 004    | Environment & Config Loader                       | PHASE 0 | ->   |
| 005    | FastAPI App Skeleton + Health Endpoint            | PHASE 1 | ->   |
| 006    | Upstash Redis Client Wrapper                      | PHASE 1 | ->   |
| 007    | Redis Sliding Window Rate Limiter                 | PHASE 1 | ->   |
| 008    | OpenAPI Spec + API Documentation                  | PHASE 1 | ->   |
| 009    | Telegram Webhook Handler + Idempotency            | PHASE 1 | ->   |
| 010    | Database Connection Pool                          | PHASE 1 | ->   |
| 011    | Users Table: Upsert on /start                     | PHASE 1 | ->   |
| 012    | Bot Command System: /help, /list, /delete, /stats | PHASE 1 | ->   |
| 013    | GET /api/items Endpoint                           | PHASE 1 | ->   |
| 014    | DELETE /api/items/{id} + IDOR Protection          | PHASE 1 | ->   |
| 015    | Fernet Encryption Service                         | PHASE 2 | ->   |
| 016    | Text Ingestion + Task Worker Loop                 | PHASE 2 | ->   |
| 017    | Modal Whisper Endpoint (Tier 0 STT)               | PHASE 2 | ->   |
| 018    | AI Cascade Service                                | PHASE 2 | ->   |
| 019    | Voice Note Ingestion                              | PHASE 2 | ->   |
| 020    | PDF Ingestion                                     | PHASE 2 | ->   |
| 021    | PDF Chunking + Multi-Chunk Embedding              | PHASE 2 | ->   |
| 022    | Image Ingestion                                   | PHASE 2 | ->   |
| 023    | Image OCR Quality + Preprocessing                 | PHASE 2 | ->   |
| 024    | URL Ingestion: Scraping + Save                    | PHASE 2 | ->   |
| 025    | YouTube URL Pipeline                              | PHASE 2 | ->   |
| 026    | Content Deduplication                             | PHASE 2 | ->   |
| 027    | Dead Letter Queue Writer                          | PHASE 2 | ->   |
| 028    | Redis Queue Monitoring + Dead Letter Retries      | PHASE 2 | ->   |
| 029    | Embedding Pipeline Integration                    | PHASE 3 | ->   |
| 030    | Hybrid Search: Vector + Trigram                   | PHASE 3 | ->   |
| 031    | /api/search REST Endpoint + Auth Guard            | PHASE 3 | ->   |
| 032    | Tag System: Auto-Generate + Filter                | PHASE 3 | ->   |
| 033    | Search Result Ranking: Map-Reduce RAG             | PHASE 3 | ->   |
| 034    | React + Vite Project Setup                        | PHASE 4 | ->   |
| 035    | Dashboard Layout + Header                         | PHASE 4 | ->   |
| 036    | Items Feed View (Alternative to Graph)            | PHASE 4 | ->   |
| 037    | Toast Notification System                         | PHASE 4 | ->   |
| 038    | Empty States + Loading Skeletons                  | PHASE 4 | ->   |
| 039    | Mobile Responsive Layouts                         | PHASE 4 | ->   |
| 040    | Error Boundary + Network Error Handling           | PHASE 4 | ->   |
| 041    | Keyboard Shortcuts + Accessibility                | PHASE 4 | ->   |
| 042    | Progressive Web App (PWA) Configuration           | PHASE 4 | ->   |
| 043    | Graph API Endpoint                                | PHASE 5 | ->   |
| 044    | GET /api/graph Optimisation + Edge Pruning        | PHASE 5 | ->   |
| 045    | Force-Directed Canvas Renderer                    | PHASE 5 | ->   |
| 046    | Node Side Panel Component                         | PHASE 5 | ->   |
| 047    | Map View: Semantic Hubs + Louvain Visualisation   | PHASE 5 | ->   |

| 048 | Telegram Login Widget + JWT Issuance | PHASE 6 | -> |

| 049 | TWA HMAC Middleware | PHASE 6 | -> |

| 050 | Login Page / Landing Page | PHASE 6 | -> |

| 051 | Logout + Session Refresh | PHASE 6 | -> |

| 052 | WebSocket Real-Time Graph Updates | PHASE 6 | -> |

| 053 | Frontend WebSocket Hook + Real-Time Node Addition | PHASE 6 | -> |

| 054 | WebSocket Connection Status UI | PHASE 6 | -> |

| 055 | Settings Page (Timezone, Preferences) | PHASE 6 | -> |

| 056 | Batch Items Export (GDPR / Data Portability) | PHASE 6 | -> |

| 057 | API Rate Limit for Web Endpoints | PHASE 6 | -> |

| 058 | SM-2 Algorithm + Quiz Endpoints | PHASE 7 | -> |

| 059 | Bot Inline Keyboard Quiz Flow | PHASE 7 | -> |

| 060 | Quiz History + Performance Tracking | PHASE 7 | -> |

| 061 | APScheduler Setup + All 5 Jobs | PHASE 8 | -> |

| 062 | Streak Counter + Drive Nudge Logic | PHASE 8 | -> |

| 063 | Louvain Clustering + Hub Nodes in Canvas | PHASE 8 | -> |

| 064 | Daily Digest Bot Message | PHASE 8 | -> |

| 065 | Streak Visualisation in Frontend | PHASE 8 | -> |

| 066 | Partition Manager CLI Script | PHASE 8 | -> |

| 067 | Bot /remind Command + Natural Language Time Parsing | PHASE 8 | -> |

| 068 | Reminder UI on Website | PHASE 8 | -> |

| 069 | Google OAuth Flow | PHASE 9 | -> |

| 070 | Drive Sync Service | PHASE 9 | -> |

| 071 | Drive Connect UI on Website | PHASE 9 | -> |

| 072 | Disconnect Drive Endpoint + Google Token Revocation | PHASE 9 | -> |

| 073 | Chrome Extension: Manifest V3 + Popup | PHASE 9 | -> |

| 074 | Chrome Extension: Background Service Worker | PHASE 9 | -> |

| 075 | Chrome Extension: Options Page | PHASE 9 | -> |

| 076 | Backend Test Suite: Full Coverage | PHASE 10 | -> |

| 077 | Frontend Test Suite: Vitest | PHASE 10 | -> |

| 078 | Load Testing with k6 | PHASE 10 | -> |

| 079 | Integration Test: Full Item Save Flow | PHASE 10 | -> |

| 080 | Security Penetration Tests: IDOR + Injection | PHASE 10 | -> |

| 081 | Performance Profiling: Vector Search Benchmarks | PHASE 10 | -> |

| 082 | End-to-End Test: Auth Flows | PHASE 10 | -> |

| 083 | Rate Limit Testing: Redis Pipeline Atomicity | PHASE 10 | -> |

| 084 | Security Audit Pass | PHASE 11 | -> |

| 085 | Pre-Deployment Checklist Execution | PHASE 11 | -> |

| 086 | GitHub Actions CI Pipeline | PHASE 11 | -> |

| 087 | Production Go-Live Sequence | PHASE 11 | -> |

| 088 | Telegram TWA Registration | PHASE 11 | -> |

| 089 | Monitoring & Observability Setup | PHASE 11 | -> |

| 090 | Monitoring: Structured Logging + Alerts | PHASE 11 | -> |

| 091 | Fernet Key Rotation Script | PHASE 11 | -> |

| 092 | Security Scanning: SAST + Dependency Audit | PHASE 11 | -> |

| 093 | Performance Testing: Frontend Bundle Optimisation | PHASE 11 | -> |

| 094 | Database Backup Strategy | PHASE 11 | -> |

| 095 | Rollback Procedure | PHASE 11 | -> |

| 096 | OpenTelemetry Tracing (Optional Enhancement) | PHASE 11 | -> |

| 097 | Final Documentation Pass | PHASE 11 | -> |

| 098 | README.md for GitHub | PHASE 11 | -> |

| 099 | Smoke Test Script (Production Verification) | PHASE 11 | -> |  
| 100 | Final Acceptance: 0 -> 100% Gate | PHASE 11 | -> |

## Summary of Build Playbook

| Prompt | Title                                             | Phase   | Done |
|--------|---------------------------------------------------|---------|------|
| 001    | Image OCR Quality + Preprocessing                 | PHASE 0 | ->   |
| 002    | Force-Directed Canvas Renderer                    | PHASE 0 | ->   |
| 003    | Local Development Environment                     | PHASE 0 | ->   |
| 004    | Repo Structure & Python Environment               | PHASE 0 | ->   |
| 005    | Neon Database DDL                                 | PHASE 1 | ->   |
| 006    | Streak Visualisation in Frontend                  | PHASE 1 | ->   |
| 007    | Logout + Session Refresh                          | PHASE 1 | ->   |
| 008    | Redis Queue Monitoring + Dead Letter Retries      | PHASE 1 | ->   |
| 009    | Environment & Config Loader                       | PHASE 1 | ->   |
| 010    | Upstash Redis Client Wrapper                      | PHASE 1 | ->   |
| 011    | WebSocket Real-Time Graph Updates                 | PHASE 1 | ->   |
| 012    | Drive Connect UI on Website                       | PHASE 1 | ->   |
| 013    | Content Deduplication                             | PHASE 1 | ->   |
| 014    | Dead Letter Queue Writer                          | PHASE 1 | ->   |
| 015    | Redis Sliding Window Rate Limiter                 | PHASE 2 | ->   |
| 016    | OpenAPI Spec + API Documentation                  | PHASE 2 | ->   |
| 017    | Map View: Semantic Hubs + Louvain Visualisation   | PHASE 2 | ->   |
| 018    | Telegram Login Widget + JWT Issuance              | PHASE 2 | ->   |
| 019    | TWA HMAC Middleware                               | PHASE 2 | ->   |
| 020    | Users Table: Upsert on /start                     | PHASE 2 | ->   |
| 021    | Pre-Deployment Checklist Execution                | PHASE 2 | ->   |
| 022    | Bot Command System: /help, /list, /delete, /stats | PHASE 2 | ->   |
| 023    | Production Go-Live Sequence                       | PHASE 2 | ->   |
| 024    | FastAPI App Skeleton + Health Endpoint            | PHASE 2 | ->   |
| 025    | Daily Digest Bot Message                          | PHASE 2 | ->   |
| 026    | GitHub Actions CI Pipeline                        | PHASE 2 | ->   |
| 027    | Database Connection Pool                          | PHASE 2 | ->   |
| 028    | Empty States + Loading Skeletons                  | PHASE 2 | ->   |
| 029    | GET /api/items Endpoint                           | PHASE 3 | ->   |
| 030    | DELETE /api/items/{id} + IDOR Protection          | PHASE 3 | ->   |
| 031    | Text Ingestion + Task Worker Loop                 | PHASE 3 | ->   |
| 032    | Rate Limit Testing: Redis Pipeline Atomicity      | PHASE 3 | ->   |
| 033    | Drive Sync Service                                | PHASE 3 | ->   |
| 034    | Security Penetration Tests: IDOR + Injection      | PHASE 4 | ->   |
| 035    | Login Page / Landing Page                         | PHASE 4 | ->   |
| 036    | Hybrid Search: Vector + Trigram                   | PHASE 4 | ->   |
| 037    | Settings Page (Timezone, Preferences)             | PHASE 4 | ->   |
| 038    | Chrome Extension: Manifest V3 + Popup             | PHASE 4 | ->   |
| 039    | /api/search REST Endpoint + Auth Guard            | PHASE 4 | ->   |
| 040    | Tag System: Auto-Generate + Filter                | PHASE 4 | ->   |
| 041    | Search Result Ranking: Map-Reduce RAG             | PHASE 4 | ->   |
| 042    | README.md for GitHub                              | PHASE 4 | ->   |

| 043 | Modal Whisper Endpoint (Tier 0 STT) | PHASE 5 | -> |

| 044 | Partition Manager CLI Script | PHASE 5 | -> |

| 045 | End-to-End Test: Auth Flows | PHASE 5 | -> |

| 046 | Telegram Webhook Handler + Idempotency | PHASE 5 | -> |

| 047 | Bot /remind Command + Natural Language Time Parsing | PHASE 5 | -> |

| 048 | Performance Profiling: Vector Search Benchmarks | PHASE 6 | -> |

| 049 | Fernet Encryption Service | PHASE 6 | -> |

| 050 | Chrome Extension: Background Service Worker | PHASE 6 | -> |

| 051 | Chrome Extension: Options Page | PHASE 6 | -> |

| 052 | Frontend Test Suite: Vitest | PHASE 6 | -> |

| 053 | AI Cascade Service | PHASE 6 | -> |

| 054 | Backend Test Suite: Full Coverage | PHASE 6 | -> |

| 055 | React + Vite Project Setup | PHASE 6 | -> |

| 056 | Security Scanning: SAST + Dependency Audit | PHASE 6 | -> |

| 057 | Dashboard Layout + Header | PHASE 6 | -> |

| 058 | Voice Note Ingestion | PHASE 7 | -> |

| 059 | Google OAuth Flow | PHASE 7 | -> |

| 060 | Items Feed View (Alternative to Graph) | PHASE 7 | -> |

| 061 | PDF Ingestion | PHASE 8 | -> |

| 062 | Batch Items Export (GDPR / Data Portability) | PHASE 8 | -> |

| 063 | API Rate Limit for Web Endpoints | PHASE 8 | -> |

| 064 | Load Testing with k6 | PHASE 8 | -> |

| 065 | Integration Test: Full Item Save Flow | PHASE 8 | -> |

| 066 | Toast Notification System | PHASE 8 | -> |

| 067 | Embedding Pipeline Integration | PHASE 8 | -> |

| 068 | Reminder UI on Website | PHASE 8 | -> |

| 069 | PDF Chunking + Multi-Chunk Embedding | PHASE 9 | -> |

| 070 | YouTube URL Pipeline | PHASE 9 | -> |

| 071 | Frontend WebSocket Hook + Real-Time Node Addition | PHASE 9 | -> |

| 072 | Disconnect Drive Endpoint + Google Token Revocation | PHASE 9 | -> |

| 073 | SM-2 Algorithm + Quiz Endpoints | PHASE 9 | -> |

| 074 | Mobile Responsive Layouts | PHASE 9 | -> |

| 075 | Security Audit Pass | PHASE 9 | -> |

| 076 | Bot Inline Keyboard Quiz Flow | PHASE 10 | -> |

| 077 | Quiz History + Performance Tracking | PHASE 10 | -> |

| 078 | APScheduler Setup + All 5 Jobs | PHASE 10 | -> |

| 079 | Error Boundary + Network Error Handling | PHASE 10 | -> |

| 080 | Keyboard Shortcuts + Accessibility | PHASE 10 | -> |

| 081 | Node Side Panel Component | PHASE 10 | -> |

| 082 | Progressive Web App (PWA) Configuration | PHASE 10 | -> |

| 083 | Monitoring: Structured Logging + Alerts | PHASE 10 | -> |

| 084 | Streak Counter + Drive Nudge Logic | PHASE 11 | -> |

| 085 | Louvain Clustering + Hub Nodes in Canvas | PHASE 11 | -> |

| 086 | URL Ingestion: Scraping + Save | PHASE 11 | -> |

| 087 | Smoke Test Script (Production Verification) | PHASE 11 | -> |

| 088 | Image Ingestion | PHASE 11 | -> |

| 089 | WebSocket Connection Status UI | PHASE 11 | -> |

| 090 | Graph API Endpoint | PHASE 11 | -> |

| 091 | GET /api/graph Optimisation + Edge Pruning | PHASE 11 | -> |

| 092 | Telegram TWA Registration | PHASE 11 | -> |

| 093 | Monitoring & Observability Setup | PHASE 11 | -> |

| 094 | Fernet Key Rotation Script | PHASE 11 | -> |  
| 095 | Performance Testing: Frontend Bundle Optimisation | PHASE 11 | -> |  
| 096 | Database Backup Strategy | PHASE 11 | -> |  
| 097 | Rollback Procedure | PHASE 11 | -> |  
| 098 | OpenTelemetry Tracing (Optional Enhancement) | PHASE 11 | -> |  
| 099 | Final Documentation Pass | PHASE 11 | -> |  
| 100 | Final Acceptance: 0 -> 100% Gate | PHASE 11 | -> |